



ch32fun_zig 技術ドキュメント

Zig 0.16 で CH32V003 (RV32EC) を焼くまで

対象: Zig 0.16 / CH32V003 / RV32EC

全16章 + 付録4本

目次

第I部 ターゲットとツールチェーン

- 第1章 本書の対象とゴール — ch32fun_zig が解決していること
- 第2章 RV32EC というターゲットを正しく指定する — Target.Query で RV32EC を組む
- 第3章 Zig 0.16 のクロスコンパイル基盤 — LLVM + lld + compiler_rt の役割

第II部 リンクと起動

- 第4章 メモリマップとリンクスクリプト — VMA/LMA と .data トリック
- 第5章 起動コードとランタイム初期化 — _start から main() までの段取り
- 第6章 ベクタテーブルと割り込みエントリ — PFIC と SysTick ハンドラ

第III部 ビルドパイプラインと書き込み

- 第7章 build.zig をひと通り歩く — Examples 切替とステップ DAG
- 第8章 ELF から .bin / .hex への変換 — objcopy が抜き出すもの
- 第9章 minichlink で実機に書き込む — SWIO と tools/flash.sh

第IV部 HAL の構造

- 第10章 MMIO とレジスタ抽象化 — extern struct と *volatile T
- 第11章 HAL — GPIO と SysTick — Pin 型と delayMs の作り
- 第12章 HAL — I2C と SSD1306 — ブロッキング送信と 1024B フレームバッファ

第V部 言語仕様と標準ライブラリ

- 第13章 本プロジェクトで使える Zig 言語機能と std — 使える / 使えない / 太る の見分け方

第VI部 永続化

- 第14章 データの永続化 — 内蔵 FLASH に書く HAL — Slot(T) で 1 構造体 = 1 ページ

第VII部 便利な周辺機能

- 第15章 周辺機能の HAL — UART / log / PWM / Tone / ADC / EXTI — ログ・LED調光・音・アナログ・割り込み入力

第VIII部 Zig 言語機能を活かす

- 第16章 Zig 言語機能を活かしたファームウェアパターン — 4 つのサンプルで Zig らしさを実装で示す

付録

付録A. 用語集

付録B. RV32 アセンブリ チートシート

付録C. CH32V003 レジスタ早見表

付録D. トラブルシューティング

はじめに

本プロジェクト `ch32fun_zig` が、どのようにして **追加の RISC-V ツールチェイン無しで CH32V003 (RV32EC) 向けファームウェアをビルド・書き込みできているのか** を、コードベースに沿って解剖した技術書。

対象読者

- ・ C/C++ で組み込み開発の経験があり、リンカスクリプトや objcopy の概念は知っている
- ・ Zig の基本構文と `build.zig` を一度くらい触ったことがある
- ・ RISC-V については「ARM とは別 ISA」程度の認識でも読める

ハイライト

- ・ Zig 0.16 + LLVM の `Target.Query` で **RV32EC** をどう表現するか
- ・ `linker.ld` の `> RAM AT > FLASH` トリック
- ・ `_start` (naked) → `_start_c` の起動シーケンス
- ・ `build.zig` のステップ DAG と `addObjCopy` の役割
- ・ `minichlink` + SWIO による書き込み経路
- ・ レジスタ層 / GPIO HAL / SysTick / I2C / SSD1306 の薄い積み重ね

構成

- ・ `chapters/` — 章ごとのソース (Markdown)
- ・ `notes/` — 用語集 / 早見表 / トラブルシューティング
- ・ `generate-book.py` — 章群を 1 冊の PDF にまとめるスクリプト
- ・ `ch32fun-zig-tutorial.pdf` — 生成済み PDF (生成後に出力される)

PDF の生成

```
python3 -m venv /tmp/ch32pdfgen
/tmp/ch32pdfgen/bin/python -m pip install markdown weasyprint pygments
/tmp/ch32pdfgen/bin/python docs/generate-book.py
```

WeasyPrint は内部で pango / cairo を使う。macOS なら `brew install pango`、Linux なら `apt install libpango-1.0-0 libpangoft2-1.0-0` などが別途必要になる場合がある。

章一覧

第 I 部 — ターゲットとツールチェーン

- ・ 第 1 章: 本書の対象とゴール
- ・ 第 2 章: RV32EC というターゲットを正しく指定する
- ・ 第 3 章: Zig 0.16 のクロスコンパイル基盤

第 II 部 — リンクと起動

- ・ 第 4 章: メモリマップとリンカスクリプト
- ・ 第 5 章: 起動コードとランタイム初期化
- ・ 第 6 章: ベクタテーブルと割り込みエントリ

第 III 部 — ビルドパイプラインと書き込み

- ・ 第 7 章: `build.zig` をひと通り歩く
- ・ 第 8 章: ELF から `.bin` / `.hex` への変換
- ・ 第 9 章: minichlink で実機に書き込む

第 IV 部 — HAL の構造

- ・ 第 10 章: MMIO とレジスタ抽象化
- ・ 第 11 章: HAL — GPIO と SysTick
- ・ 第 12 章: HAL — I2C と SSD1306

第 V 部 — 言語仕様と標準ライブラリ

- ・ 第 13 章: 本プロジェクトで使える Zig 言語機能と std

第 VI 部 — 永続化

- ・ 第 14 章: データの永続化 — 内蔵 FLASH に書く HAL

第 VII 部 — 便利な周辺機能

- ・ 第 15 章: 周辺機能の HAL — UART / log / PWM / Tone / ADC / EXTI

第 VIII 部 — Zig 言語機能を活かす

- ・ 第 16 章: Zig 言語機能を活かしたファームウェアパターン

付録

- ・ 付録 A: 用語集
- ・ 付録 B: RV32 アセンブリ チートシート
- ・ 付録 C: CH32V003 レジスタ早見表
- ・ 付録 D: トラブルシューティング

第I部 ターゲットとツールチェーン

RV32EC を Zig のクロスコンパイル基盤で扱う

第1章 本書の対象とゴール

ch32fun_zig が解決していること

学習目標

- ・ 本プロジェクト `ch32fun_zig` が「何を / どの層で」実現しているのかを掴む
 - ・ CH32V003 という RISC-V MCU の概要を知る
 - ・ Zig 0.16 のクロスコンパイル機構が、なぜ追加の RISC-V GCC を使わずにファームウェアを生成できるのか、その全体像を見渡す
 - ・ 以降の章で深掘りする構成要素（ターゲット指定 / リンカ / ランタイム / objcopy / 書き込み）の役割分担を理解する
-

本プロジェクトが目指すもの

`ch32fun_zig` は、[cnlohr/ch32fun](https://github.com/cnlohr/ch32fun) で確立された CH32V シリーズ向けの軽量な開発体験を、**Pure Zig** で再現することを狙った最小限のフレームワークである。

具体的には次の特徴を持つ。

- ・ **CH32V003 (RISC-V RV32EC, 48MHz, FLASH 16K / SRAM 2K)** を主ターゲットとする
- ・ 追加の RISC-V ツールチェーン (`riscv-none-elf-gcc` など) を必要としない
- ・ Zig 自身が内蔵する LLVM バックエンドが RISC-V コードを直接生成する
- ・ 書き込みのみ、ch32fun リポジトリに同梱の `minichlink` をそのまま使う
- ・ `zig build` ひとつで `.elf` / `.bin` / `.hex` まで生成し、`zig build flash` でそのまま MCU に書き込める
- ・ HAL は GPIO / SysTick / I2C / SSD1306 / 入力ボタンといった「最低限よくある」ものに絞っている

つまり本リポジトリは、「組み込み RISC-V のクロス開発をやろうとすると普通は分厚くなる工程を、Zig の機能だけでどこまで薄くたためるか」の実験でもある。

CH32V003 ざっくり仕様

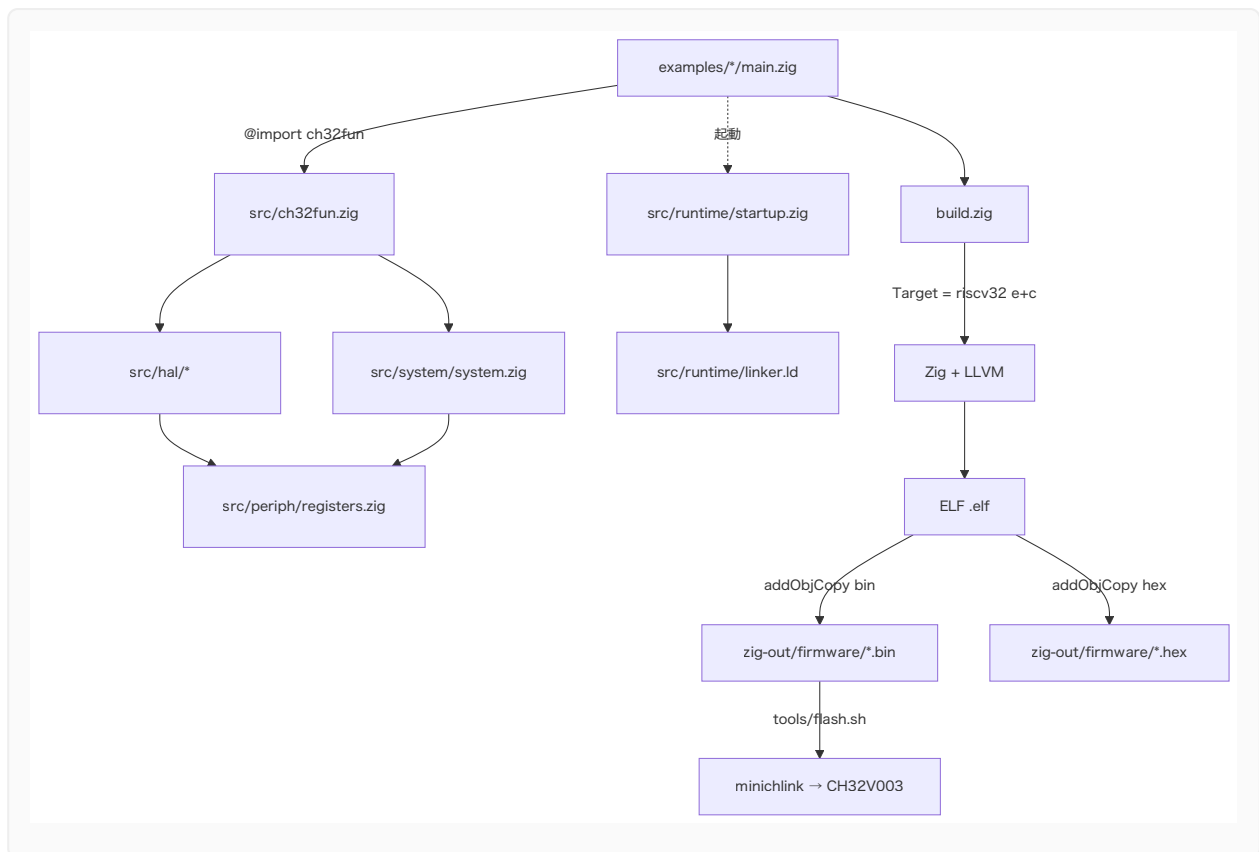
項目	値
MCU	CH32V003 (WCH)

項目	値
CPU コア	QingKe RV32EC (RISC-V)
命令セット	RV32E + C 拡張 (I 拡張は 無い)
動作クロック	最大 48 MHz
内蔵 FLASH	16 KB (0x08000000 開始)
内蔵 SRAM	2 KB (0x20000000 開始)
割り込みコントローラ	PFIC (NVIC 風)
デバッグ	1-wire SWIO
パッケージ	SOP8 / TSSOP20 など

「2KB の RAM で何ができるんだ」というスケール感が重要で、本プロジェクトのコードサイズ・データサイズに対する制約は、ほぼここから来ている。

全体アーキテクチャ

ファームウェアが「書ける」までを上から下に並べると次のようになる。



ポイントは大きく 4 段に分かれる:

1. **アプリ層** — `examples/*/main.zig` が `ch32fun` モジュールを `@import` して HAL を呼ぶ
2. **HAL / システム層** — `src/hal/*.zig` と `src/system/system.zig` が MMIO に対する型付きアクセスを提供
3. **ランタイム / リンク層** — `src/runtime/startup.zig` が `_start` / ベクタテーブル / `.data` コピー / `.bss` ゼロクリアを担当し、`src/runtime/linker.ld` が FLASH / RAM の物理配置を決める
4. **ビルド / 書き込み層** — `build.zig` が `riscv32` ターゲットでコンパイル、`addObjCopy` で `.bin` / `.hex` に変換し、`tools/flash.sh` から `minichlink` を呼んで FLASH に焼く

このうち、本書で特に重点的に扱うのは **2~4 段目** のしくみだ。「アプリの書き方」ではなく、「アプリを CH32V003 のフラッシュに乗るバイナリに変換するまでの道筋」のほうである。

本書のスコープと前提

本書は次のような読者を想定している。

- ・ C/C++ で何らかの組み込み開発をしたことがあり、リンカスクリプトや objcopy の存在は知っている
- ・ Zig の基本構文と `build.zig` を一度くらい触ったことがある

- ・ RISC-V については「ARM とは別物の ISA」くらいの認識でも構わない

逆に、「Zig の言語仕様そのもの」「組み込み一般の入門」「SSD1306 の API ハウツー」は本書の主題ではない。それらは適宜参考リンクで触れるに留め、本書は「**ch32fun_zig という具体的なコードベースを題材に、Zig + LLVM で MCU 向け RISC-V ファームウェアをビルドする全段を解剖する**」ことに集中する。

章構成と読み進め方

部	章	主題
I	01～03	ターゲット指定とツールチェーン (RV32EC, Zig 0.16 のクロスコンパイル)
II	04～06	リンカスクリプト、起動コード、ベクタテーブル
III	07～09	<code>build.zig</code> の歩き方、 <code>objcopy</code> 、 <code>minichlink</code> による書き込み
IV	10～12	レジスタ抽象化と HAL の作り方 (GPIO/SysTick/I2C/SSD1306)

順に読めば「ビルドが通る理由 → リンクと配置が成立する理由 → ELF からフラッシュに焼ける理由 → HAL の作法」を一気通貫で辿れる構成にしている。すでに該当章の内容を把握している読者は、興味のある章から拾い読みしてもよい。

次章では、CH32V003 のコアである **RV32EC** を Zig のターゲット指定で正しく表現するところから始める。

第2章 RV32EC というターゲットを正しく指定する

Target.Query で RV32EC を組む

学習目標

- ・ RISC-V の ISA 命名規則 (RV32IMAC のような表記) の読み方を理解する
- ・ CH32V003 が採用する **RV32EC** が、一般的な RV32I 系と何が違うのかを知る
- ・ Zig の `std.Target.Query` で「I を引き、E と C を足す」表現がどう書かれているかを読み解く
- ・ `freestanding` / `eabi` という OS/ABI 指定の意味を押さえる

RISC-V の ISA 名の読み方

RISC-V の ISA は、いくつかの基本セット + 拡張の組み合わせで表される。

- ・ **基本セット**
 - ・ **I** — 32 個の汎用整数レジスタ。RISC-V の標準。
 - ・ **E** — **Embedded**。汎用レジスタを 32 → **16 本** に削った組み込み向け派生。 **I** と排他。
- ・ **代表的な拡張**
 - ・ **M** — 整数の乗除算
 - ・ **A** — アトミック命令 (LR/SC など)
 - ・ **F** / **D** — 単精度 / 倍精度浮動小数点
 - ・ **C** — **圧縮命令**。16-bit 幅のショート形を追加し、コード密度を稼ぐ
 - ・ **Zicsr** — CSR (制御/状態レジスタ) 命令
 - ・ **Zifencei** — `fence.i` 命令

RV32GC などに使われる **G** は「`IMAFD_Zicsr_Zifencei`」のショートハンドで、Linux クラスの環境向け。一方、本書で扱う CH32V003 はそんなに豪華ではなく、最小限の **RV32EC** である。

表記	含意
RV32	32 ビット整数
E	レジスタ x0~x15 までの 16 本のみ (組み込み派生)
C	16-bit 圧縮命令を許可

つまり CH32V003 は「32-bit RISC-V のうち、レジスタ半分・C 拡張あり」という、非常にコンパクトなコアだ。乗除算もアトミックも浮動小数点もハードでは持たない。必要なら **コンパイラが `__mulsi3` などのライブラリ関数を呼び出すコードを生成する。**

🔧 Zig では、`exe.bundle_compiler_rt = true;` を立てることで、これらランタイム補助関数 (`compiler_rt`) が静的にリンクされる。`build.zig` の該当行はまさにこれを担っている。

Zig 0.16 でのターゲット指定

本プロジェクトの `build.zig` では、CH32V003 のターゲットを次のように記述している。

```
const query = std.Target.Query{
    .cpu_arch = .riscv32,
    .cpu_model = .{ .explicit = &std.Target.riscv.cpu.generic_rv32 },
    .cpu_features_add = std.Target.riscv.featureSet(&.{
        std.Target.riscv.Feature.c,
        std.Target.riscv.Feature.e,
    }),
    .cpu_features_sub = std.Target.riscv.featureSet(&.{
        std.Target.riscv.Feature.i,
    }),
    .os_tag = .freestanding,
    .abi = .eabi,
};

const target = b.resolveTargetQuery(query);
```

これは「RV32E + C、つまり RV32EC」を Zig の世界に翻訳したものだ。順に読んでいく。

`.cpu_arch = .riscv32`

ターゲット ISA が 32-bit RISC-V であることを宣言する。LLVM のターゲットトリプルでいうと `riscv32-...` の部分にあたる。

`.cpu_model = .{ .explicit = &std.Target.riscv.cpu.generic_rv32 }`

「具体的なコアモデル」を指定する場所だ。LLVM には `sifive_e31` のような実装固有モデルもあるが、ここでは **汎用の RV32 ベース** に明示的に固定し、フィーチャ追加/削除でカスタマイズする方針をとっている。メーカー独自コア (QingKe) ゆえに、LLVM 側に専用モデルが定義されているわけではない、という事情もある。

`.cpu_features_add` で C と E を足す

```
.cpu_features_add = std.Target.riscv.featureSet(&.{
    std.Target.riscv.Feature.c,
    std.Target.riscv.Feature.e,
}),
```

`featureSet` は「LLVM が認識する CPU フィーチャの集合」を作るヘルパで、ここでは:

- `Feature.c` — 圧縮命令 (RVC) を有効化。これがないと `c.add` / `c.li` などの 16-bit 命令が出ず、コードサイズがほぼ倍になる
- `Feature.e` — RV32E。レジスタ本数を 16 に絞る方の派生を選ぶ

の 2 つを ON にしている。

`.cpu_features_sub` で I を引く

```
.cpu_features_sub = std.Target.riscv.featureSet(&.{
    std.Target.riscv.Feature.i,
}),
```

E と I は同居できない。`generic_rv32` はデフォルトで I を持っているので、E を有効化するためには **明示的に I を引く** 必要がある。Zig の `Target.Query` がこういう「足す / 引く」を別フィールドで書けるおかげで、`RV32I - I + E + C = RV32EC` が宣言的に表現できている。

`.os_tag = .freestanding`

「OS は無い (= リンクすべきランタイムが無い)」という宣言。これによって、Zig は libc や OS のスタートアップに依存しないコードを吐く。文字列表記でいう `riscv32-unknown-none-eabi` の `none` 部分に対応する。

`.abi = .eabi`

EABI (Embedded ABI) を選択する。引数渡しや呼び出し規約が、デスクトップ向けの `gnu` / `musl` ABI ではなく、組み込み用の `eabi` で揃えられる。CH32V003 のような MCU では実質これ一択である。

なぜこの組み合わせが重要か

仮に `Feature.c` を外すと、生成バイナリは 2 倍近くまで膨らみ、16KB FLASH に収まらなくなる可能性が出てくる。逆に `Feature.e` を外して `I` のままにすると、コンパイラは「x16~x31 も自由に使える」前提でレジスタ割り付けを行ってしまうが、**CH32V003 の実機にはそのレジスタが物理的に存在しない**ので、命令が走った瞬間にハードフォールトする。

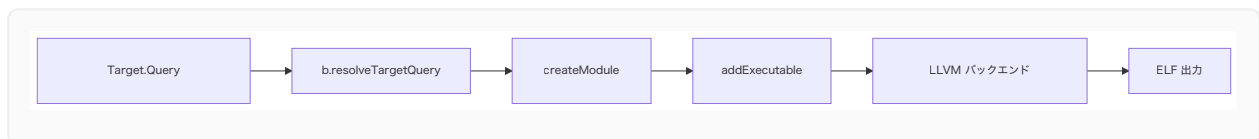
つまりこの数行は、

- ・ **コードサイズに直接効く** (`C` 拡張の有無)
- ・ **そもそも正しく動く / 動かないが決まる** (`E` を選ぶか `I` を選ぶか)

という、非常にクリティカルな宣言になっている。

ターゲット指定がどこに効くか

Zig + LLVM のパイプラインでは、ここで作った `target` は次の場所に伝播していく。



`createModule` でモジュールに紐づき、そのモジュールから `addExecutable` で実行ファイルを作ると、LLVM の RISC-V バックエンドが「RV32EC + 圧縮命令あり、ABI は eabi」というセッティングでアセンブリを生成する。

結果として `.elf` は:

- ・ ELF クラス: 32-bit
- ・ マシンタイプ: `EM_RISCV`
- ・ ABI: EABI
- ・ フラグ: `EF_RISCV_RVC` (圧縮命令) / `EF_RISCV_RVE` (組み込み派生) などが立つ

といった具合に、CH32V003 用の正しいメタデータを持つ形になる。実物は次章以降で `readelf` 系のコマンドで覗いていく。

まとめ

- ・ CH32V003 のコアは **RV32EC** — レジスタ 16 本 + 圧縮命令拡張

- ・ Zig 0.16 の `Target.Query` では、`generic_rv32` を起点に `I` を引いて `E` と `C` を足すことで RV32EC を表現
- ・ `freestanding` + `eabi` で OS なし / 組み込み ABI に揃える
- ・ このターゲット指定が正しくないと、コードサイズが膨らむか、最悪「動かないバイナリ」が出来上がる

次章では、こうして指定したターゲットを **Zig が単一バイナリのまま処理しきれ理由** — つまり、追加の RISC-V GCC を要なくしている Zig のクロスコンパイル基盤について見ていく。

第3章 Zig 0.16 のクロスコンパイル基盤

LLVM + lld + compiler_rt の役割

学習目標

- ・「Zig が `riscv-none-elf-gcc` 無しで RISC-V バイナリを吐ける」のは何故かを説明できる
 - ・LLVM バックエンド / lld / compiler_rt が Zig の中でどう連携しているかを掴む
 - ・本プロジェクトで `linkage = .static` / `link_libc = false` / `bundle_compiler_rt = true` を選んでいる理由を理解する
 - ・セクションごとの GC (`link_gc_sections` 系) がコードサイズに効く仕組みを知る
-

ふつうの組み込みクロス開発との違い

C/C++ で CH32V003 を開発する場合、典型的にはこんな依存が必要だ。

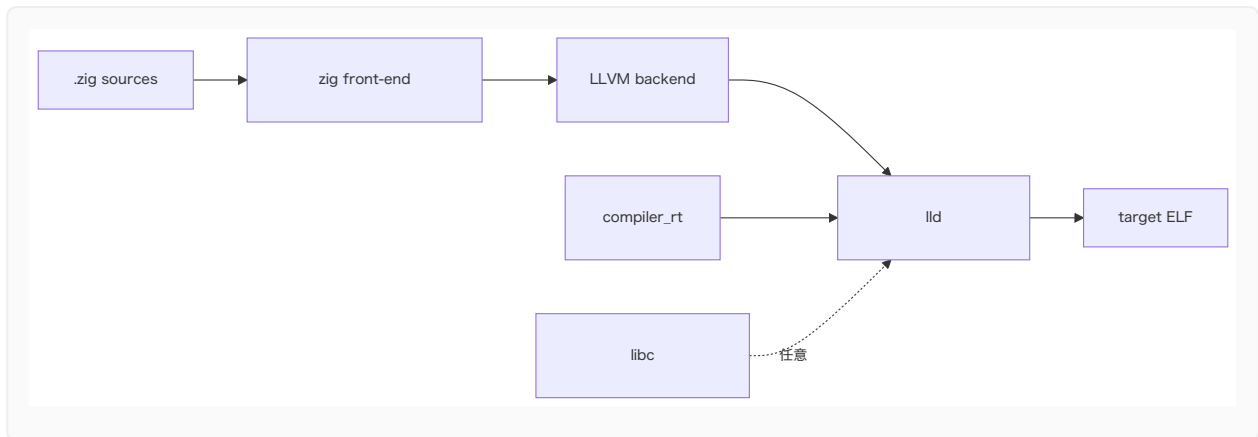
- ・ `riscv-none-elf-gcc` (もしくは `riscv64-unknown-elf-gcc` などの multilib 版)
- ・ `riscv-none-elf-binutils` (`ld` , `objcopy` , `nm` , `objdump` , `size`)
- ・同じツールチェーン用の `newlib` / `libgcc`

これらを揃えてはじめて、ホスト OS (Mac/Linux/Windows) 上で CH32V003 向けの ELF を作れるようになる。つまり、開発機ごとに「数百 MB のクロスツールチェーン」を入れる必要がある。

一方、本プロジェクトは **zig 単体しか要求しない** (書き込みのための `minichlink` を除けば)。これは Zig という言語の処理系設計に由来する性質だ。

Zig のコンパイラ構成

`zig` バイナリは、雑に分解するとこういう部品で構成されている。



ここで効いてくるポイントは:

1. **LLVM が Zig コンパイラに同梱**されている。外部の `clang` / `gcc` を呼び出すのではなく、`zig` プロセス内部で LLVM の IR を直接生成 → コード生成する
2. **lld も同梱**。GNU `ld` を呼ばずに、Zig の中でリンクまで完結する
3. **compiler_rt は Zig 自前実装**。32-bit 乗除算ヘルパ (`__mulsi3`, `__divsi3`) などを **Zig で書き直したもの**を、ターゲットに合わせて `--target=riscv32-...` でコンパイルして埋め込む
4. **libc は任意**。OS のあるターゲットなら `musl` / `glibc` / `mingw` を埋め込めるが、`link_libc = false` ならそもそもリンクしない

結果として、ホスト OS 用の `zig` を 1 本入れさえすれば、

- ・ macOS 用の `x86_64-macos`
- ・ Linux 用の `aarch64-linux-musl`
- ・ Windows 用の `x86_64-windows-gnu`
- ・ MCU 用の `riscv32-freestanding-eabi` ← **今回これ**

…のいずれもクロスビルドできる。CH32V003 向けに特別なクロスツールチェーンを入れなくてよい根拠はここにある。

RISC-V バックエンドが選ばれる流れ

第 2 章で作った `Target.Query` は、最終的に LLVM のターゲットトリプル + 機能フラグに変換される。

```
riscv32-unknown-none-eabi -target-feature +c,+e,-i
```

これが LLVM の RISC-V バックエンドに渡ると、内部で:

- ・ 命令選択 (Instruction Selection) を RV32EC 用に切替

- ・レジスタアロケータの「使える整数レジスタ集合」を `x0..x15` に制限
- ・圧縮命令 (`c.*`) を可能な箇所で出力

といった処理が行われる。Zig 側から見るとこれら全ては「ターゲットフラグを正しく作って渡すだけ」で済む。

`build.zig` で本プロジェクトが行う追加宣言

ターゲットを作ったあとの実行ファイル組み立て部分を抜き出すと、こうなっている。

```
const exe = b.addExecutable(.{
    .name = selected.name,
    .root_module = root_module,
    .linkage = .static,
});

exe.step.dependOn(&mkdir_step.step);
exe.bundle_compiler_rt = true;
exe.link_gc_sections = true;
exe.link_function_sections = true;
exe.link_data_sections = true;
exe.setLinkerScript(b.path("src/runtime/linker.ld"));
```

それぞれが MCU 開発でなぜ重要かを 1 行ずつ見ていく。

`linkage = .static`

動的リンクは MCU には存在しない。ELF はすべて静的にリンクされ、`.so` / `.dylib` の概念は無い。

`link_libc = false` (モジュール側で指定)

`root_module` を作る時に `.link_libc = false` を指定している。freestanding ターゲットなので、`printf` も `malloc` も無く、標準 C ライブラリの依存は完全に切る。

`bundle_compiler_rt = true`

第 2 章で触れた通り、CH32V003 は乗除算命令を持たない。そのため `a * b` のような Zig コードがあると、LLVM は `__mulsi3` ヘルパへの呼び出しに展開する。このヘルパ群が `compiler_rt` で、Zig が自前で持っている。 `bundle_compiler_rt = true` を指定することで、これらのヘルパが ELF にスタティックに同梱される。

実際、`zig build -Dexample=blinky` 後の ELF の中を覗くと、`__mulsi3` のような関数シンボルが (使われていれば) 含まれている。GCC 系で言う `libgcc` の役割をここで Zig が肩代わりしている。

`link_function_sections` / `link_data_sections` / `link_gc_sections`

これらはセクション粒度の **デッドコード/データ削除** を有効化する。

- `link_function_sections = true` → 関数ごとに別セクション (`.text.func_name`) として出力する
- `link_data_sections = true` → グローバル変数ごとに別セクション (`.data.var_name`) として出力する
- `link_gc_sections = true` → リンカに「参照されていないセクションは捨てて良い」と教える

3 つを揃えてはじめて、リンカは「最終的に使われていない関数 / 変数」単位でファームウェアから取り除ける。16KB の FLASH に対しては効果が大きく、SSD1306 のフォント 2KB を含んだ `oled` サンプルでも実装によっては fits in できるのは、ある程度この GC が効いているおかげだ。

`setLinkerScript`

LLD に対し、独自のリンカスクリプト (`src/runtime/linker.ld`) を渡している。これが無いと、LLD はデフォルトのスクリプトで `.text` を `0x0000_0000` から配置しようとしてしまい、CH32V003 の FLASH 配置 (`0x0800_0000` 開始) と合わない。リンカスクリプトについては第 4 章で詳しく見る。

ホスト依存を極力減らす設計

それでも `build.zig` 内に残っているホスト依存は次の通り:

用途	呼び出し先	必須か
<code>mkdir -p zig-out/firmware</code>	システムの <code>mkdir</code>	実質必須 (どの POSIX 環境にもある)
<code>.lst</code> 生成	<code>llvm-objdump</code> or <code>riscv-none-elf-objdump</code>	任意 (生成しなくてもファームウェアは作れる)
<code>.map</code> 生成	<code>llvm-nm</code> or <code>riscv-none-elf-nm</code>	任意
<code>size</code> ターゲット	<code>llvm-size</code> or <code>riscv-none-elf-size</code>	任意

用途	呼び出し先	必須か
<code>flash</code> ターゲット	<code>tools/flash.sh</code> → <code>minichlink</code>	書き込み時のみ必須

`.elf` / `.bin` / `.hex` を生成する **本筋のビルドには Zig しか要らない**。`disasm` / `mapfile` / `size` は **オプトインのステップ** として切り出しており、LLVM ツール群が無くてもデフォルトビルドは失敗しないように `build.zig` を構成している。

まとめ

- ・ Zig は LLVM と lld を同梱しており、追加の RISC-V GCC 系ツールチェーンを必要としない
- ・ 乗除算など RV32EC が持たない命令の補完は、Zig 製の `compiler_rt` を `bundle_compiler_rt` で同梱することで賄う
- ・ `link_function_sections` + `link_data_sections` + `link_gc_sections` で「使われていない部品」を削れるようにしてある
- ・ ホスト依存は「書き込み用の minichlink」と「任意の LLVM/GNU ツール」だけに絞っており、ELF 生成までは `zig` 単体で完結する

次章では、LLD に渡している `linker.ld` の中身を読み解き、FLASH と RAM の物理アドレスにコードとデータがどう配置されるかを追っていく。

第II部 リンクと起動

FLASH/RAM 配置と起動シーケンスを解剖する

第4章 メモリマップとリンカスクリプト

VMA/LMA と .data トリック

学習目標

- ・ CH32V003 の物理メモリ配置 (FLASH / SRAM の番地) を把握する
- ・ `src/runtime/linker.ld` の `MEMORY` / `SECTIONS` がそれぞれ何を決めているのか説明できる
- ・ `.data` を「FLASH に置きつつ RAM の番地で解決する」テクニック (AT >) を読み解ける
- ・ `_sdata` / `_edata` / `_sbss` / `_ebss` / `_sidata` / `_stack_top` のシンボルが何のために露出されているのかを理解する

CH32V003 の物理メモリマップ (要点)

領域	番地	サイズ	用途
FLASH	0x0800_0000 ~	16 KB	プログラムコード / 定数 / <code>.data</code> の初期値
SRAM	0x2000_0000 ~	2 KB	スタック / グローバル変数 (<code>.data</code> / <code>.bss</code>)
周辺レジスタ	0x4000_0000 ~	—	RCC, GPIO, I2C, ...
コア内蔵周辺	0xE000_0000 ~	—	PFIC, SysTick

このうち、リンカスクリプトが直接決めるのは **FLASH と SRAM の使い方** だ。周辺レジスタは MMIO として MCU の物理配線で決まっているので、ソフト側からはアドレス即値として参照するだけになる (詳細は第 10 章)。

FLASH 領域より前 (0x0000_0000 ~) には何があるのか

第 1 章のメモリマップでは「FLASH は 0x0800_0000 から」と書いた。だが RISC-V のリセットベクタは 0x0000_0000 から始まるのが普通だ。では 0x0000_0000 ~ 0x07FF_FFFF の 128 MB 弱はいったい何のために空けてあるのか — これを知らずにメモリマップを眺めると、「リセットしたのになぜ 0x0800_0000 のコードが走るのか」が説明できない。

CH32V003 (および STM32 / GD32 系) では、この領域は「ブートエイリアス」という仕組みで使われている。

ブートエイリアスとは

リセット直後の CPU : PC = 0x0000_0000 から fetch
 ↓ ハードウェアが透過的にリダイレクト
 実際にアクセスされる場所 : FLASH (0x0800_0000) または BootROM (0x1FFF_F000)
 ↑ どちらにエイリアスするかは BOOT0 ピンで決まる

つまり 0x0000_0000 の領域は、**それ自体には物理的な記憶素子が無く**、別の領域への「鏡像 (alias)」として現れる。リセット時に CPU は 0x0000_0000 を読みに行くが、アドレスバスに 0x0000_0000 が出た瞬間、メモリコントローラがそれを「BOOT0 が L なら 0x0800_0000 (FLASH)」 「BOOT0 が H なら 0x1FFF_F000 (BootROM)」 にすり替えてアクセスを返す。

CH32V003 の典型的な配置 (BOOT0 = L) では以下ようになる。

領域	番地レンジ	サイズ	役割
Boot alias	0x0000_0000 ~ 0x0000_3FFF	16 KB	リセットベクタからの fetch を FLASH または BootROM にエイリアス
予約 (未マップ)	0x0000_4000 ~ 0x07FF_FFFF	≒ 128 MB	アクセスするとバスフォールト
FLASH	0x0800_0000 ~ 0x0800_3FFF	16 KB	本書で扱うコード/データの本体
予約	0x0800_4000 ~ 0x1FFF_EFFF	—	未マップ
System Memory (BootROM)	0x1FFF_F000 ~ 0x1FFF_F7FF	1.92 KB	WCH が工場で書き込んだ ISP ブートローダ。USART などからの再書き込み経路を持つ
Option bytes	0x1FFF_F800 ~ 0x1FFF_F80F	16 B	リードプロテクション (RDPR)、ユーザフラグ、WDG モード
Factory trim 等	0x1FFF_F7D4	1 B	HSI クロックの工場校正値 (本プロジェクトは CFG0_PLL_TRIM でこの番地を読んでいる)
予約	0x1FFF_F810 ~ 0x1FFF_FFFF	—	未マップ

BOOT0 ピンと「どこへエイリアスされるか」

BOOT0	エイリアス先	何が起きるか
L (GND)	FLASH (0x0800_0000)	通常運用。リセットすると FLASH のユーザコードから起動
H (VCC)	System Memory (0x1FFF_F000)	WCH BootROM が起動。USART 等経由で FLASH を書き換える ISP モードに入れる

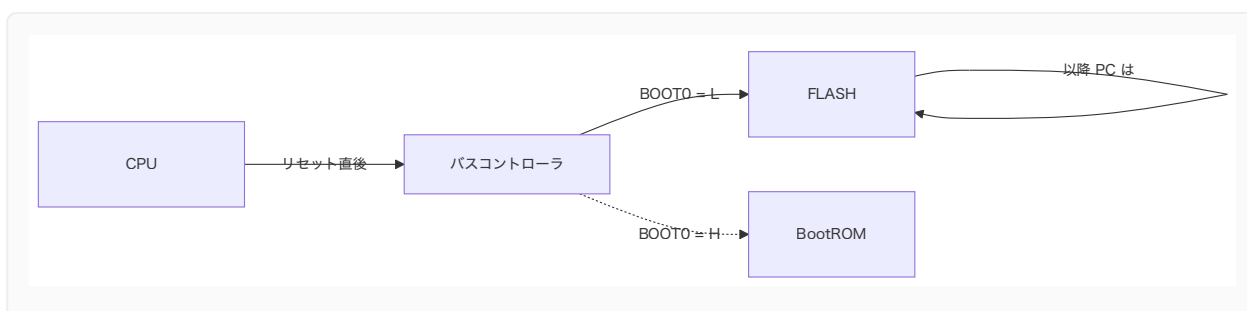
minichlink を使う書き込みでは BOOT0 を H にする必要が無い。SWIO 経由でデバッグ機能を握って FLASH を直接書き換えるためだ。一方、SWIO アダプタが無い環境で「最初の書き込み」を行いたい場合は BOOT0 を H にして BootROM を起こし、シリアル ISP を使う、という選択肢が残されている。

startup から見た風景

第 5 章で見る startup コードは、これらを意識せず常に 0x0800_xxxx の番地で動いているように振る舞う。これはリンクスクリプトが:

```
FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 16K
```

と宣言し、全てのリンク解決が 0x0800_0000 基準で行われるからだ。リセット時の 0x0000_0000 fetch はあくまでハードの透過処理であって、ソフトウェアから見えるアドレスは最初から 0x0800_xxxx になる。



「予約領域」を踏むとどうなる

0x0000_4000 ~ 0x07FF_FFFF や 0x0800_4000 ~ 0x1FFF_EFFF のような未マップ領域に load/store すると、CH32V003 では **バスフォールト (例外)** が発生する。ベクタテーブルの 3 番 (Exception) に飛ぶので、本プロジェクトのデフォルトでは _default_irq_entry 経由で wfi ループに落ちる (第 6 章)。

つまり、ヌルポインタ参照 (`*ptr = 0` で `ptr == null`) のような事故は、アドレス的には未マップ領域の `0x0000_0000` 近傍を踏みに行くため、「無事故では済まない」形で必ず観測できる、ということでもある。デバッグ時の保険として悪くない挙動。

工場 trim を使っている例

CH32V003 は内蔵 HSI (高速 RC オシレータ) のキャリブレーション値を `0x1FFF_F7D4` に持っている。本プロジェクトの `src/system/system.zig` がそれを読み出して `RCC.CTLR` に書き戻すコードを含む:

```
const trim: *volatile u8 = @ptrFromInt(regs.CFG0_PLL_TRIM);
if (trim.* != 0xFF) {
    const old = rcc.CTLR;
    rcc.CTLR = (old & ~(@as(u32, 0x1F) << 3)) | (@as(u32, trim.* & 0x1F) << 3);
}
```

`CFG0_PLL_TRIM` は:

```
pub const CFG0_PLL_TRIM: usize = 0x1FFFF7D4;
```

として `src/periph/registers.zig` に定義されている。ここはまさに「Option/Factory 領域」 — FLASH 本体 (`0x0800_0000`) ではなく BootROM 近傍にある工場ROMで、ユーザは読み取り専用で使う番地だ。

`src/runtime/linker.ld` 全体像

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 16K
    RAM (rwx)  : ORIGIN = 0x20000000, LENGTH = 2K
}

SECTIONS
{
    .vector_table :
    {
        . = ALIGN(4);
        KEEP(*(.vector_table))
        . = ALIGN(4);
    } > FLASH

    .text :
    {
        . = ALIGN(4);
```

```

        *(.text .text.*)
        *(.rodata .rodata.*)
        . = ALIGN(4);
    } > FLASH

    .data :
    {
        . = ALIGN(4);
        _sdata = .;
        *(.data .data.*)
        *(.sdata .sdata.*)
        . = ALIGN(4);
        _edata = .;
    } > RAM AT > FLASH

    _sidata = LOADADDR(.data);

    .bss (NOLOAD) :
    {
        . = ALIGN(4);
        _sbss = .;
        *(.bss .bss.*)
        *(.sbss .sbss.*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .;
    } > RAM

    PROVIDE(__global_pointer$ = ORIGIN(RAM) + 0x800);
    PROVIDE(_stack_top = ORIGIN(RAM) + LENGTH(RAM));

    /DISCARD/ :
    {
        *(.eh_frame*)
        *(.comment)
        *(.note*)
    }
}

```

ブロックごとに読み解いていく。

MEMORY — 物理領域の宣言

```

MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 16K
}

```

```
RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 2K
}
```

- ・ FLASH は **読み・実行可能** (rx)。書き込みはアプリ実行時には行わない。
- ・ RAM は **読み・書き・実行可能** (rwx) — CH32V003 の SRAM は実行可能領域でもあるが、本プロジェクトでは RAM 上にコードを置く構成は採っていない。
- ・ サイズが嘘 (LENGTH の指定が実機より大きい) だとリンカが超過を見逃すので、ここは正確に書く必要がある。

LENGTH = 16K を超えるとリンクが失敗するので、これは「16K に収まっているか」のセーフティネットとしても働く。

.vector_table — ベクタテーブルを FLASH 先頭に置く

```
.vector_table :
{
    . = ALIGN(4);
    KEEP(*(.vector_table))
    . = ALIGN(4);
} > FLASH
```

- ・ KEEP(...) を付けないと、第 3 章で有効化した `--gc-sections` が「どこからも呼ばれていない」と判断して 削除してしまう。ベクタテーブルはハードが暗黙に参照するため、リンカに「これは残せ」と指示する必要がある。
- ・ 配置順序の通り、`.text` より前にあるので、`.vector_table` は **FLASH の先頭 0x0800_0000** に居座る。
- ・ これは CH32V003 のリセット時の挙動とも整合する。リセット時、コアは `mtvec` から間接的にハンドラを引くため、ベクタテーブルがどこに居るかは「`mtvec` に書いたアドレス」で決まるのだが、配置を予測しやすくする意味でも先頭固定にしている。

.text — コードと定数

```
.text :
{
    . = ALIGN(4);
    *(.text .text.*)
    *(.rodata .rodata.*)
}
```

```
. = ALIGN(4);  
} > FLASH
```

- ・ `.text` (実行コード) と `.rodata` (読み取り専用データ、定数文字列など) をまとめて FLASH に積む。
- ・ `*(.text.*)` というワイルドカードは、第 3 章で有効化した `function_sections` の出力 (`.text.main`, `.text.delayMs`, ...) を全部すくい上げるためのもの。
- ・ ここに居るバイトは、書き込み後そのままの番地で CPU が読みに行く。

`.data` — 「FLASH に置いて RAM の番地でリンクする」 トリック

```
.data :  
{  
    . = ALIGN(4);  
    _sdata = .;  
    *(.data .data.*)  
    *(.sdata .sdata.*)  
    . = ALIGN(4);  
    _edata = .;  
} > RAM AT > FLASH  
  
_sidata = LOADADDR(.data);
```

ここが最初に見ると混乱しやすいので順を追って解く。

何が起きているか

`.data` セクションには「初期値を持ったグローバル変数」が入る。たとえば `var x: u32 = 0x12345678;` のように初期値があるもの。

このセクションは、**実行時には RAM に存在しないといけない** (書き換え可能なので)。しかし **初期値そのものは FLASH に置いておかなければならない** (電源を切ったら RAM は消えるので)。

これを解決するのが `> RAM AT > FLASH` だ。

- ・ `> RAM` は **VMA (Virtual Memory Address)**。「実行時にこの番地にあるものとしてリンクする」 — つまりコードは `0x2000_0000` 系の番地を見に行く。
- ・ `AT > FLASH` は **LMA (Load Memory Address)**。「実体は FLASH に書き込まれる」 — つまり ELF ロードヤ書き込みツールは `0x0800_xxxx` の場所にバイトを置く。

実機リセット直後では「コードは RAM 番地を期待するが、その RAM 番地はまだ未初期化」という状態になる。そこで起動時に **FLASH 上の初期値を RAM にコピーする** 必要がある (これが startup の `copyData()` の仕事 — 第 5 章で詳述)。

露出されるシンボル

シンボル	意味
<code>_sdata</code>	<code>.data</code> の RAM 上の開始番地
<code>_edata</code>	<code>.data</code> の RAM 上の終端番地
<code>_sidata</code>	<code>.data</code> の FLASH 上の開始番地 (LOADADDR が返す LMA)

startup コードはこの 3 つを参照して、`[_sidata, _sidata + (_edata - _sdata))` の FLASH 内容を `[_sdata, _edata)` の RAM へコピーする。

`.bss` — 初期値ゼロの領域

```
.bss (NOLOAD) :
{
    . = ALIGN(4);
    _sbss = .;
    *(.bss .bss.*)
    *(.sbss .sbss.*)
    *(COMMON)
    . = ALIGN(4);
    _ebss = .;
} > RAM
```

- ・「初期値が 0」の変数は ELF にバイト列として書く必要がない。ELF 上では `.bss` は領域だけ確保し、中身は持たない (`NOLOAD` の意味)。
- ・実行時には RAM 上に存在する必要があるので、startup でこの範囲を **0 で埋める** 仕事を行う (`zeroBss()`)。
- ・`_sbss` / `_ebss` がそのために露出される。

グローバルポインタとスタックトップ

```
PROVIDE(__global_pointer$ = ORIGIN(RAM) + 0x800);
PROVIDE(_stack_top = ORIGIN(RAM) + LENGTH(RAM));
```

`__global_pointer$`

RISC-V には、グローバル変数アクセスをコンパクトな命令にエンコードするための **gp レジスタ最適化** がある (Linker Relaxation の一種)。gp レジスタを「RAM の先頭 + 0x800」に向けると、そこから ±2KB の範囲のグローバル変数を 12-bit オフセット 1 命令でアクセスできるようになる。

2KB の RAM をフルに使うため、RAM 先頭から +0x800 (= 2048) を指して中心に据えるのが慣例。startup の `_start` で:

```
.option norelax
la gp, __global_pointer$
.option pop
```

として、自身の参照を Linker Relaxation から除外しつつロードしている。(`.option norelax` を挟まないと、「gp を gp 相対でロードする」という循環論法になりかねないため)

`_stack_top`

スタックは RAM の最後尾から下に伸びる慣例なので、RAM の末尾を指すシンボルを `_stack_top` として露出する。起動時の `la sp, _stack_top` でここを `sp` に積めば、以降のスタック push がアドレスを減らしながら使われる。

`/DISCARD/` — 要らないものを捨てる

```
/DISCARD/ :
{
    *(.eh_frame*)
    *(.comment)
    *(.note*)
}
```

例外ハンドリング情報やビルド時のメタデータは、ファームウェアバイナリに乗せる必要がない。明示的に `/DISCARD/` に流して、FLASH を節約する。

配置のイメージ

```
FLASH (0x0800_0000):
+-----+ 0x0800_0000
|      .vector_table      |
+-----+
```

```

|      .text      |
|      .rodata    |
+-----+
| .data の初期値 (LMA) |
+-----+ <- 16KB 以内に収まる
               < LOADADDR(.data) == _sdata

SRAM (0x2000_0000):
+-----+ 0x2000_0000 <- _sdata
|      .data      |
+-----+ <- _edata
|      .bss       |
+-----+ <- _ebss
|      ...        |
|      (free space)
|      ...        |
|      ↑ stack ↓  |
+-----+ 0x2000_0800 <- _stack_top

```

まとめ

- ・ `MEMORY` で FLASH / RAM の物理配置を宣言し、`SECTIONS` で各セクションをそこに割り当てる
- ・ `.vector_table` は `KEEP` でリンカ GC から守り、FLASH 先頭に配置
- ・ `.data` は **`VMA = RAM / LMA = FLASH`** で「初期値は FLASH、本体は RAM」を表現
- ・ `.bss` は `NOLOAD` で実体を持たず、起動時に 0 クリア
- ・ `__global_pointer$` と `_stack_top` を露出し、起動コードが `gp` / `sp` を立てる足場にする

次章では、これらシンボルを実際を使って「リセット直後に C/Zig コードが動ける状態を作るまで」の起動コードを読んでいく。

第5章 起動コードとランタイム初期化

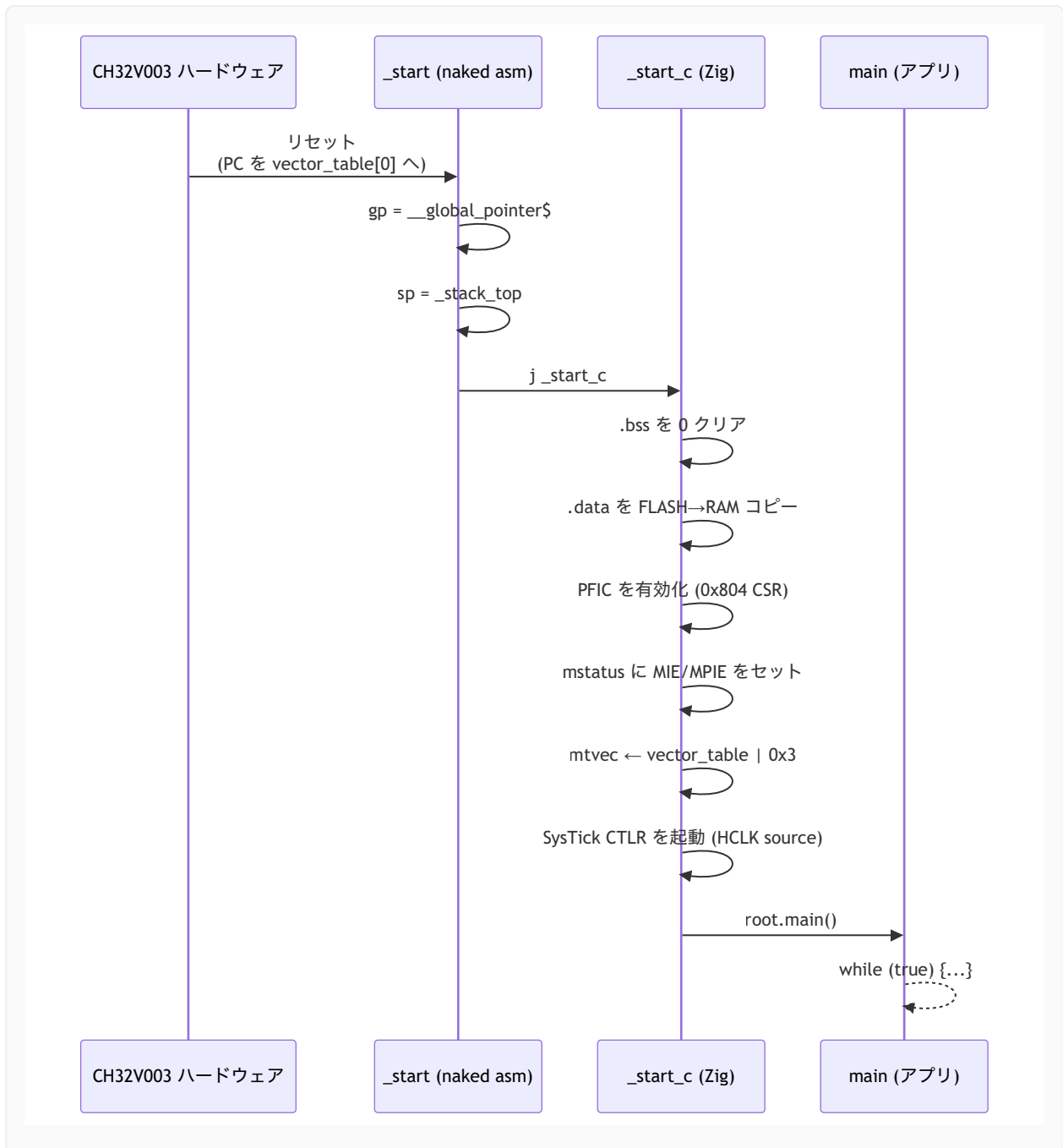
_start から main() までの段取り

学習目標

- ・ `src/runtime/startup.zig` の `_start` / `_start_c` がそれぞれ何を担当しているかを読み解ける
 - ・ `callconv(.naked)` と `callconv(.c)` の使い分けを理解する
 - ・ リセット直後に `gp` / `sp` / `.data` / `.bss` / `mtvec` / `mstatus` をどう整えていくか順を追える
 - ・ なぜ `extern var _sbss: u32;` のような外部宣言で良いのかを理解する
 - ・ 最終的にユーザの `main()` に制御が渡るまでの一連の流れを把握する
-

起動シーケンスの全体像

CH32V003 がリセットされてから、`examples/blink/main.zig` の `main()` が走り出すまでには、大きく次のステップが挟まる。



ハードウェアそのものが面倒を見てくれるのは「PC を vector_table[0] にセットする」 (= `_start` に飛ばす) ところまで。そこから先は全部ソフトウェアの仕事である。

`_start` — naked エントリポイント

```

pub export fn _start() callconv(.naked) noreturn {
    asm volatile (
        \\ .option push
        \\ .option norelax
        \\ la gp, __global_pointer$
  
```

```

        \\option pop
        \\la sp, _stack_top
        \\j _start_c
    );
}

```

callconv(.naked) が要る理由

通常関数を生成すると、コンパイラはプロローグ (フレームポインタの保存、スタック確保) を必ず挟む。ところが「sp がまだ立っていない時点で走る関数」では、スタックに何かを push する自動コードが入った瞬間に死ぬ。

callconv(.naked) は「プロローグ / エピローグを一切生成するな」という指示で、関数の中身は完全に開発者が書く **生のアセンブリ** になる。

gp を立てる — .option norelax

```

.option push
.option norelax
la gp, __global_pointer$
.option pop

```

la は擬似命令で、通常はリンカが「auipc + addi」へ展開する。さらに RISC-V のリンカは、gp 相対でアクセスできるシンボルは auipc/addi ではなく **addi gp, gp, offset 1 命令に縮める** リラックス処理を行う。

ところがここで縮めると、「gp の値を gp 相対で求める」という循環が発生してしまう。そこでこのブロックだけ .option norelax で囲い、通常の auipc + addi 展開のままにしておく。

ペアになっている .option push / .option pop は、オプションのスコープを「この命令ペアの間だけ」に閉じ込めるための定石。

sp を立てる

```

la sp, _stack_top

```

リンクスクリプトで露出した _stack_top (= RAM 末尾) を sp にロードする。これでスタックが「RAM の上端から下に伸びる」状態になり、以降は普通のスタック push が安全になる。

j_start_c

ここで初めて普通の C/Zig 関数 (`_start_c`) にジャンプする。`call` ではなく `j` (= 単純なジャンプ) を使うのは、戻る必要がない (= スタックフレームを残す必要がない) からだ。ここで戻り先を `ra` に積むと、スタックフレームの整合性に気をを使う必要が生じる。

_start_c — Zig 側の初期化本体

```
export fn _start_c() callconv(.c) noreturn {
    zeroBss();
    copyData();

    setupMachineState();

    // Start free-running SysTick (HCLK source) for delay API.
    regs.systick().CTLR = regs.SYSTICK_CTLR_STE | regs.SYSTICK_CTLR_STCLK;

    root.main();
}
```

`callconv(.c)` を付けているので、通常通り Zig コンパイラがプロローグ / エピローグを生成する。`sp` がもう立っているので、ローカル変数を取っても問題ない。

順に追っていく。

zeroBss() — .bss を 0 にする

```
extern var _sbss: u32;
extern var _ebss: u32;

fn zeroBss() void {
    var p: [*]volatile u32 = @ptrCast(&_sbss);
    const end: [*]volatile u32 = @ptrCast(&_ebss);
    while (@intFromPtr(p) < @intFromPtr(end)) : (p += 1) {
        p[0] = 0;
    }
}
```

ここで使っているテクニックは:

- `extern var _sbss: u32;` — リンカが定義するシンボルは「型は問わずアドレスだけ欲しい」場合が多い。Zig では `extern var` として **適当な型** で宣言し、`&_sbss` で「そのシンボルの番地」を取り出す。値そのものは読まないなので、`u32` でも何でも良い。

- ・ **[*]volatile u32** — 「volatile u32 の不定長配列ポインタ」を表す Zig の型。C で言う `volatile uint32_t *`。メモリへの書き込みが最適化で消えないようにするために `volatile` を付ける。
- ・ **4 バイト単位ループ** — リンカで `_sbss` / `_ebss` をどちらも 4 アラインに揃えているので、1 ワードずつ書き込むだけで取りこぼしがない。

copyData() — .data の初期値を RAM に展開

```
extern var _sdata: u32;
extern var _edata: u32;
extern var _sidata: u32;

fn copyData() void {
    var src: [*]const u32 = @ptrCast(&_sidata);
    var dst: [*]volatile u32 = @ptrCast(&_sdata);
    const end: [*]volatile u32 = @ptrCast(&_edata);
    while (@intFromPtr(dst) < @intFromPtr(end)) : ({
        src += 1;
        dst += 1;
    }) {
        dst[0] = src[0];
    }
}
```

第 4 章の > RAM AT > FLASH で作った「FLASH 上の `.data` 実体」を RAM 上の `.data` 番地にコピーする。リンカが用意したシンボルがそのまま実行時アドレスとして使える。

これが終わるまでは、初期値付きグローバル変数 (`var counter: u32 = 100;` のようなもの) は **未定義値** を持つことになる。つまり、ユーザの `main()` で `counter` を読むより前にこの関数が走り終わっている必要がある。順序を守るのが `_start → _start_c → main` の構成の役目。

setupMachineState() — マシン状態レジスタを整える

```
fn setupMachineState() void {
    const mtvec_addr: usize = @intFromPtr(&vector_table) | 0x3;

    asm volatile ("li t0, 0x03; csrw 0x804, t0" ::: .{ .t0 = true });
    asm volatile ("csrs mstatus, %[bits]"
        :
        : [bits] "r" (@as(u32, 0x88)),
        : .{ .memory = true });
    asm volatile ("csrwr mtvec, %[vec]"
        :
        : [vec] "r" (@as(u32, @truncate(mtvec_addr))),
```

```
        : .{ .memory = true });  
    }
```

3 つの CSR (Control and Status Register) を順に弄っている。

`csrw 0x804, 0x03` — WCH 拡張 INTSYSCR の有効化

`0x804` は標準 RISC-V には存在しない、**WCH (CH32V) 独自の CSR (INTSYSCR)**。ビット 0/1 を立てることで、PFIC のハードウェアスタッキングや割り込みネスティングなど、ベンダ固有の挙動を有効化している。ここはチップマニュアルを読まないと意味が取れない領域なので、おまじないとして覚えておけば十分だ。

`csrs mstatus, 0x88` — MIE と MPIE をセット

- ・ビット 3 (MIE) — 機械モード割り込み有効
- ・ビット 7 (MPIE) — `mret` 復帰時に MIE に積み戻される値

両方立てておくことで、リセット後から割り込みを受け取れる状態になる。ただし PFIC 側の有効化は個別の周辺ドライバが行う。

`csrw mtvec, vector_table | 0x3`

`mtvec` は「割り込み / 例外発生時に PC を飛ばす先」のベース。下位 2 ビットはモードビットで、`0b11` は **ベクター + ハードウェアスタッキング** モード (WCH 拡張)。

`mtvec = &vector_table | 0x3` とすることで、割り込みベクタごとに別ハンドラへ飛ばすモードが選ばれる。ベクタテーブル本体は次章で見える。

SysTick を起動

```
regs.systick().CTRL = regs.SYSTICK_CTRL_STE | regs.SYSTICK_CTRL_STCLK;
```

- ・`STE` — SysTick 有効
- ・`STCLK` — クロックソースを HCLK (システムクロック)に

これで SysTick の `CNT` が自走を始める。割り込みを使わない (`delayMs` のような) 用途ではこれだけで十分で、周期割り込みが欲しければ `hal/time.zig` の `systick.init` を別途呼ぶ。

root.main() — ユーザコードへ

```
const root = @import("root");
...
root.main();
```

Zig の `@import("root")` は **ルートソースファイル** (= `addExecutable` で渡した `.root_source_file`、つまり `examples/<name>/main.zig`) を指す。そこから `main` を呼べば、アプリ側のループに制御が移る。

各 example は次のような `main` シグニチャを持っている。

```
// examples/blink/main.zig
pub export fn _start() noreturn {
    main();
}

pub fn main() noreturn {
    fun.system.init(.{});
    fun.gpio.enableAllClocks();
    const led = fun.gpio.pin(.D, 0);
    led.configure(.output_pp_10mhz);
    while (true) {
        led.toggle();
        fun.time.delayMs(250);
    }
}
```

⚠ ここで `pub export fn _start() noreturn` がアプリ側にも出てくるのが少しややこしい。これは startup の `_start` (naked) とは別物で、各 example の `main` を呼ぶための **薄いラッパ** として書かれている。実際に呼ばれるのは `src/runtime/startup.zig` の `_start` 側 (`vector_table[0]` が向いている方) であり、アプリ側の `_start` は使われない。過去経緯で残っている薄皮なので、整理対象の余地はある。

なぜこの順序が大事なのか

ステップ	これより前にやれない仕事
gp を立てる	gp 相対アクセスを含むコード全般
sp を立てる	スタックフレームを持つ関数呼び出し全般

ステップ	これより前にやれない仕事
<code>.bss</code> を 0 クリア	「初期値ゼロ」を前提にする静的変数
<code>.data</code> を RAM にコピー	初期値付き静的変数の読み出し
<code>mtvec</code> 設定	割り込みを発生させても安全に飛べる
PFIC / <code>mstatus</code> 設定	割り込み許可後の周辺ドライバ初期化
SysTick 起動	<code>delayMs</code> ベースのタイミング処理

つまり、ハードウェアからソフトウェアへの「橋渡し」を 1 ステップずつ整えていくのが startup の役目だ。順番を 1 個飛ばすと、たいてい「とりあえず動くけど時々おかしい」という最悪のバグ群に出会うことになる。

まとめ

- ・ `_start` は `callconv(.naked)` で書かれた純粋なアセンブリ。 `gp` と `sp` を立てて、すぐに `_start_c` にジャンプする
- ・ `_start_c` は Zig の通常関数として、`.bss` ゼロクリア → `.data` コピー → CSR セットアップ → SysTick 起動 → `root.main()` を順番に行う
- ・ `extern var _sbss: u32;` のような Zig の宣言で、リンカが用意したシンボルのアドレスを取り出す
- ・ 起動コードは「ハードウェアが整えていない前提を、ソフトで埋めていく」ための階段

次章では、`mtvec` から参照されるベクタテーブルがどう作られていて、SysTick 割り込みのコンテキスト退避が何バイトの何で構成されているかを見ていく。

第6章 ベクタテーブルと割り込みエントリ

PFIC と SysTick ハンドラ

学習目標

- ・ CH32V003 (QingKe) のベクタテーブル構造を把握する
 - ・ `makeVectorTable()` が `comptime` 配列で組まれている理由を理解する
 - ・ `_systick_irq_entry` の汎用レジスタ退避がなぜあれだけ並んでいるのかを読み解ける
 - ・ ベクタ番号 12 が SysTick、14 がソフトウェアといった「PFIC 側の番号付け」を知る
-

ベクタテーブルとは

CPU から見ると、割り込み / 例外が起きると `mtvec` の指すテーブルからハンドラのアドレスを引き、**PC を飛ばす** ことになる。「テーブル」の中身は普通、1 エントリ = 1 ワード (32-bit) の関数ポインタ列だ。

CH32V003 (QingKe RV32EC) では PFIC (Programmable Fast Interrupt Controller) が割り込みを管理する。ベクタ番号は以下のように決まっている。

番号	意味
0	(リセットエントリ。実質ここに <code>_start</code> が来るよう <code>vector_table[0]</code> を仕込む)
2	NMI
3	Hard Fault / Exception
12	SysTick
14	Software (PendSV 相当)
16～	周辺割り込み (各 IRQ 番号)

本プロジェクトでは、38 までを並べた長さ 39 の関数ポインタ配列を作っている。

makeVectorTable() — comptime でテーブルを組む

```
fn makeVectorTable() [39]?*const anyopaque {
    var table = [_]?*const anyopaque{null} ** 39;
    table[2] = &_default_irq_entry;    // NMI
    table[3] = &_default_irq_entry;    // Exception
    table[12] = &_systick_irq_entry;   // SysTick
    table[14] = &_default_irq_entry;   // Software

    var i: usize = 16;
    while (i < table.len) : (i += 1) {
        table[i] = &_default_irq_entry;
    }

    return table;
}

pub export const vector_table linksection(".vector_table") = makeVectorTable();
```

ポイント

- **?*const anyopaque** — 「不透明な関数ポインタ、または null」。ベクタは関数ポインタとしてだけ意味があるので、型は緩めにして良い。
- **comptime 配列構築** — Zig 0.16 では、グローバル **const** の初期化式は **comptime** で評価される。つまり **makeVectorTable()** の呼び出しはランタイムには走らず、**コンパイル時にテーブルが組み上がる**。結果として ELF に静的データとして焼き込まれる。
- **linksection(".vector_table")** — リンカスクリプトの **.vector_table** セクションに置く指示。第 4 章で **KEEP*(.vector_table)** していたのに対応する。
- **長さ 39** — 「16 (システム例外) + 23 (CH32V003 が使う最大の周辺 IRQ 番号 + α)」というつものの数。余分なエントリも **&_default_irq_entry** で埋めて、万一不意の割り込みが来てもベクタアドレスとして null を踏まないようにしている。

_default_irq_entry

```
pub export fn _default_irq_entry() callconv(.naked) void {
    asm volatile (
        \\j _default_irq_body
    );
}

export fn _default_irq_body() callconv(.c) noreturn {
    defaultInterruptBody();
    unreachable;
}
```

```

}

fn defaultInterruptBody() callconv(.c) void {
    while (true) {
        asm volatile ("wfi");
    }
}

```

「予期しない割り込みが来た時のフォールバック」。単に `wfi` (Wait For Interrupt) を回し続けて寝る。デバッグ時にはここに来ていることがブレークで分かるので、「何かよくわからん割り込みで死んでいる」状態の検知ポイントになる。

SysTick 割り込みの作法

```

pub export fn _systick_irq_entry() callconv(.naked) void {
    asm volatile (
        \\addi sp, sp, -128
        \\sw ra, 124(sp)
        \\sw gp, 120(sp)
        \\sw tp, 116(sp)
        \\sw t0, 112(sp)
        ... (caller-saved + callee-saved 全保存)
        \\call _systick_irq_body
        ... (全部 lw で復元)
        \\addi sp, sp, 128
        \\mret
    );
}

export fn _systick_irq_body() callconv(.c) void {
    time.systickInterruptBody();
}

```

なぜ全レジスタを退避するのか

第 5 章で `mtvec` に `0x3` (= ベクタード + ハードウェアスタッキング) モードを設定したものの、ハードがスタッキングしてくれる対象は限られている。加えて Zig 側で呼ばれる `_systick_irq_body` は普通の `callconv(.c)` 関数で、内部で任意のレジスタを破壊する可能性がある。

そのため、ハンドラの先頭で `t0~t6 / a0~a7 / s0~s11 / ra / gp / tp` を全て退避し、末尾でまとめて復元する形にしている。

種別	レジスタ
呼び出し規約上 caller-saved (壊しても良いが今は退避が要る)	<code>t0 ~ t6</code> , <code>a0 ~ a7</code> , <code>ra</code>

種別	レジスタ
callee-saved (Zig の通常関数が壊さない)	s0 ~ s11, gp, tp

callee-saved 側は、厳密には `_systick_irq_body` 側で必要な分だけ保存される。だが「割り込みハンドラからの戻りで `mret` するだけ」なら、ハンドラ内で **全部を退避する方が安全** で、ステートマシンのように分かりやすい。

mret

`ret` ではなく `mret` で戻るのが割り込みハンドラの作法。 `mret` は:

- ・ `mstatus.MIE` を `mstatus.MPIE` の値で復元
- ・ 特権レベルを `mstatus.MPP` で復元
- ・ PC を `mepc` の値に戻す

を一気に行う命令で、まさに「割り込みから戻る」ことの本体になる。

SysTick ボディの実装

```
export fn _systick_irq_body() callconv(.c) void {
    time.systickInterruptBody();
}
```

```
// src/hal/time.zig
pub fn systickInterruptBody() callconv(.c) void {
    const st = regs.systick();

    st.CMP += systick_ticks_per_irq;
    st.SR = 0;
    systick_ticks += 1;

    if (tick_handler) |handler| {
        handler();
    }
}
```

- ・ `CMP` を次の発火地点に進める
- ・ `SR` を 0 にしてラッチをクリア
- ・ ティックカウンタをインクリメント
- ・ 登録されたユーザコールバックがあれば呼ぶ

「割り込みハンドラの中で重い処理をしない」のが定石どおりで、実体は HAL に閉じ込めている。

ベクタ番号と `pficEnableIrq`

ベクタ番号に対応する IRQ ラインを **PFIC 側で個別に有効化**しない限り、当該割り込みは入ってこない。その仕事をしているのが:

```
// src/periph/registers.zig
pub fn pficEnableIrq(irqn: u8) void {
    const reg_index: usize = irqn / 32;
    const bit: u32 = @as(u32, 1) << @as(u5, @intCast(irqn % 32));
    const addr = PFIC_BASE + 0x100 + (reg_index * 4);
    const reg: *volatile u32 = @ptrFromInt(addr);
    reg.* = bit;
}
```

- ・ IRQ 番号を 32 で割って、PFIC の有効化レジスタ群 (`IENR0` , `IENR1` , ...) のインデックスを得る
- ・ 32 で割った余りでビット位置を計算
- ・ そのビットだけ立てる (PFIC の有効化レジスタは「1 を書くと立つ」セットレジスタなので、RMW しなくて良い)

`SysTick` だけは少し特殊で、`vector_table` から直接ハンドラに飛ぶようになっているので、通常用途では明示的に `pficEnableIrq(12)` を呼ぶ必要が無く、`mstatus.MIE` と `SysTick` の `STIE` を立てれば走り始める。

まとめ

- ・ ベクタテーブルは長さ 39 の関数ポインタ配列で、`comptime` に組まれて `.vector_table` セクションに焼かれる
- ・ 不要なエントリを含め全部 `_default_irq_entry` で埋めることで、予期しない割り込みでも `wfi` ループに入って暴走しないようにしている
- ・ `SysTick` ハンドラはレジスタ全退避 + `mret` で、通常 Zig 関数からの呼び出しが安全になるよう作られている
- ・ 周辺 IRQ を有効化するには `pficEnableIrq(n)` で個別ビットを立てる

ここまでで「リセットからユーザコードまで走る経路」「割り込みが正しく入る経路」が揃ったので、次章からは **ビルド側のパイプライン** に視点を移し、`build.zig` がこれらを ELF にまとめ上げる手順を見ていく。

第III部 ビルドパイプラインと書き込み

ソースから実機 FLASH までの一直線の経路

第7章 build.zig をひと通り歩く

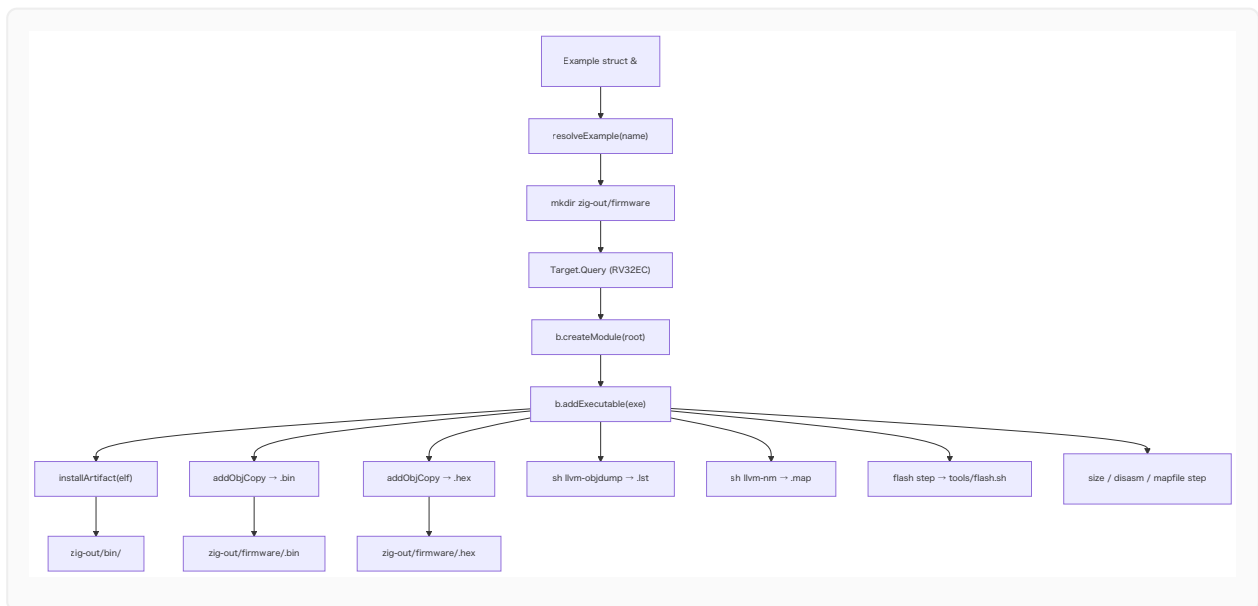
Examples 切替とステップ DAG

学習目標

- ・ 本プロジェクトの `build.zig` の全体構造をざっくり把握する
- ・ `-Dexample=<name>` でサンプルを切り替える機構の作りを読み解く
- ・ `b.createModule` → `b.addExecutable` → `b.installArtifact` の流れを追える
- ・ ステップグラフの依存関係 (`mkdir` → `exe` → `objcopy` → `install` → `flash`) を理解する

ファイル全体マップ

`build.zig` は概ね以下のブロックで構成されている。



このうち、第 2/3 章で **D~F** (ターゲット指定とコンパイラ呼び出し) を扱ったので、本章では残りの周辺整備 (A~C と G~Q) を読んでいく。

サンプル選択の作り

```
const Example = struct {
    name: []const u8,
    path: []const u8,
```

```
};

const examples = [_]Example{
    .{ .name = "blinky", .path = "examples/blinky/main.zig" },
    .{ .name = "gpio_input", .path = "examples/gpio_input/main.zig" },
    .{ .name = "timer_irq", .path = "examples/timer_irq/main.zig" },
    .{ .name = "oled", .path = "examples/oled/main.zig" },
};

fn resolveExample(name: []const u8) ?Example {
    inline for (examples) |example| {
        if (std.mem.eql(u8, name, example.name)) return example;
    }
    return null;
}
```

- `examples` を `build.zig` 内のコンパイル時データとして持つことで、サンプル追加時の手当ては「ここに 1 行足す」だけで済む。
- `inline for` は `comptime` でループを展開する。 `name` は `[]const u8` (ランタイム値) なので、展開された各 `if` の `std.mem.eql` はランタイムに走るが、ループ自体は静的に展開される。

```
const example_name = b.option([]const u8, "example", "Example to build") orelse
    "blinky";
const selected = resolveExample(example_name) orelse {
    std.debug.print("Unknown example '{s}'. Available: blinky, gpio_input, timer_irq,
oled\n", .{example_name});
    @panic("invalid example");
};
```

- `b.option([]const u8, "example", ...)` で `-Dexample=<name>` を受け取る。
- 指定が無ければ `blinky` がデフォルト。
- 未知の名前のときは `@panic` で即座に止める。これは「ビルド設定ミスサイレントに通さない」ための積極的な失敗。

`mkdir` ステップ

```
const mkdir_step = b.addSystemCommand(&.{ "mkdir", "-p", "zig-out/firmware" });
```

`zig-out/firmware/` ディレクトリを事前に作っておくためのステップ。後続の `addInstallFileWithDir(..., .{ .custom = "firmware" }, ...)` はディレクトリが無くても普通は作れるが、並列で複数の `addInstallFileWithDir` が走ったり、`sh llvm-objdump > foo.lst` のシェルリダイレクトでファイルを作るときに、親ディレクトリの存在が前提になっているケースがあるので、明示的に先行作成しておく方が嬉しい。

`exe.step.dependOn(&mkdir_step.step);` で「exe をビルドする前に mkdir」という順序を貼っている。

モジュール → 実行ファイル

```
const root_module = b.createModule({
    .root_source_file = b.path(selected.path),
    .target = target,
    .optimize = optimize,
    .link_libc = false,
});

const exe = b.addExecutable({
    .name = selected.name,
    .root_module = root_module,
    .linkage = .static,
});
```

- ・ `root_module` は「ルートソース」「ターゲット」「最適化レベル」「libc を引かない」を持つ Zig モジュールの単位。
- ・ それを `addExecutable` の `.root_module` に渡す。Zig 0.16 のビルド API は「モジュールを作って、それから実行ファイルを作る」というワンクッションを挟む形になっている。
- ・ `linkage = .static` で動的リンクを禁ずる。

ch32fun モジュールの import

```
const ch32fun_mod = b.createModule({
    .root_source_file = b.path("src/ch32fun.zig"),
});
root_module.addImport("ch32fun", ch32fun_mod);
```

これがあるおかげで、example 側で:

```
const fun = @import("ch32fun");
```

と書けるようになる。名前解決の正体は「`ch32fun` という名前を `src/ch32fun.zig` に紐付ける」という単純なマッピングだ。

ELF を install する

```
b.installArtifact(exe);

const elf_install = b.addInstallFileWithDir(
    exe.getEmittedBin(),
    .{ .custom = "firmware" },
    b.fmt("{s}.elf", .{selected.name}),
);
b.getInstallStep().dependOn(&elf_install.step);
```

- ・ `installArtifact(exe)` は デフォルトの場所 `zig-out/bin/<name>` に `exe` をコピーする標準ステップ。
- ・ それとは別に、**ELF を `zig-out/firmware/<name>.elf` にも複製**するために `addInstallFileWithDir` を使っている。 `.custom = "firmware"` でサブディレクトリを指定する形式。
- ・ 結果として、ファームウェア関連の成果物は全て `zig-out/firmware/` 配下にまとまる。

objcopy で `.bin` と `.hex` を生成

```
const bin = exe.addObjCopy(.{
    .format = .bin,
    .basename = b.fmt("{s}.bin", .{selected.name}),
});
const bin_install = b.addInstallFileWithDir(
    bin.getOutput(),
    .{ .custom = "firmware" },
    b.fmt("{s}.bin", .{selected.name}),
);
b.getInstallStep().dependOn(&bin_install.step);
```

- ・ `exe.addObjCopy({ .format = .bin })` は Zig が内部で `objcopy` 相当の処理を行うステップを作る。外部の `llvm-objcopy` を呼んでいるわけではなく、Zig が ELF を読んで Intel HEX や raw binary に変換する処理を持っている。
- ・ `.bin` (Raw) は `minichlink` への書き込みで使う。
- ・ `.hex` (Intel HEX) は他のツール (ISP プログラマ等) と互換させたいとき用。

詳しい話は次章 (objcopy) に回す。

`.lst` と `.map` のオプション生成

```
const lst_cmd = b.addSystemCommand(&.{
    "sh",
    "-c",
    "if command -v llvm-objdump >/dev/null 2>&1; then llvm-objdump -d \"$1\" > \"$2\"; else riscv-none-elf-objdump -d \"$1\" > \"$2\"; fi",
    "sh",
});
lst_cmd.addFileArg(exe.getEmittedBin());
const lst_file = lst_cmd.addOutputFileArg(b.fmt("{s}.lst", .{selected.name}));
const lst_install = b.addInstallFileWithDir(lst_file, .{ .custom = "firmware" },
b.fmt("{s}.lst", .{selected.name}));
lst_install.step.dependOn(&lst_cmd.step);
```

- `addSystemCommand` で外部コマンドをステップ化する。ここでは `sh` から `llvm-objdump` を呼んで逆アセンブルを取る。
- `addFileArg` は入力ファイル (`$1`)。
- `addOutputFileArg` は出力ファイル (`$2`) の **placeholder**。Zig は出力先のパスを内部で割り当て、後続の `addInstallFileWithDir` でそのパスを参照できる。シェル上では「Zig が決めたテンポラリパス」が `$2` に入る形になる。

`disasm` / `mapfile` ステップとして公開してあるので、「LLVM ツールが無いと失敗する」性質を持ち込まずに済んでいる:

```
const disasm = b.step("disasm", "Generate disassembly (.lst) - requires llvm-objdump or riscv-none-elf-objdump");
disasm.dependOn(&lst_install.step);

const mapfile = b.step("mapfile", "Generate symbol map (.map) - requires llvm-nm or riscv-none-elf-nm");
mapfile.dependOn(&map_install.step);
```

通常の `zig build` (デフォルトの `install` ターゲット) はこれらに依存していないため、LLVM ツール無しでも成功する。`zig build disasm` を明示したときだけ走る。

flash ステップ

```
const flash = b.step("flash", "Flash selected example using minichlink");
flash.dependOn(b.getInstallStep());

const flash_cmd = b.addSystemCommand(&.{ "sh", "tools/flash.sh" });
flash_cmd.addArg(selected.name);
```

```
flash_cmd.step.dependOn(b.getInstallStep());
flash.dependOn(&flash_cmd.step);
```

- ・ `flash` ステップは「install 一式」 + 「flash.sh 実行」の両方に依存する。
- ・ `flash.sh` は次の章 (第 9 章) で詳しく見るが、要は `minichlink -w zig-out/firmware/<name>.bin flash -b` を呼びだけのシンスクリプト。

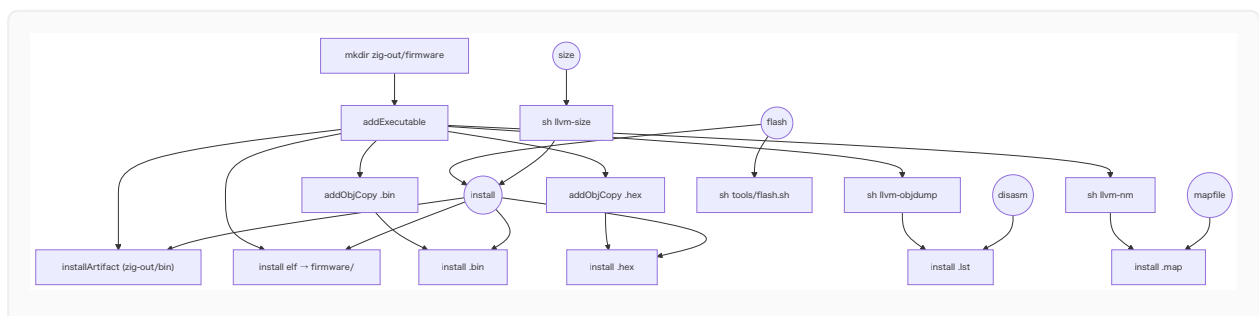
size ステップ

```
const size_cmd = b.addSystemCommand(&.{
    "sh",
    "-c",
    "if command -v llvm-size >/dev/null 2>&1; then llvm-size \"$1\"; else riscv-none-elf-size \"$1\"; fi",
    "sh",
});
size_cmd.addFileArg(exe.getEmittedBin());
size_cmd.step.dependOn(b.getInstallStep());
const size = b.step("size", "Show firmware size - requires llvm-size or riscv-none-elf-size");
size.dependOn(&size_cmd.step);
```

ELF を `llvm-size` (もしくは `riscv-none-elf-size`) に通して、`.text` / `.data` / `.bss` のサイズを表示する。「16K に収まっているか」のさっくり確認に便利。`size` ステップも明示的に呼んだときだけ走る。

ステップグラフ全体

Zig のビルドシステムは内部的に DAG (有向非巡回グラフ) でステップを管理している。本プロジェクトのグラフはおおむね次のようになる:



ポイント:

- ・ `install` の依存先には `.lst` / `.map` が含まれない。デフォルトビルドはこの 4 つ (`bin/<name>`, `firmware/<name>.elf`, `firmware/<name>.bin`, `firmware/<name>.hex`) を作れば成功。
- ・ `disasm` / `mapfile` / `size` は **オプティン**。LLVM ツールに依存する箇所を切り出してある。
- ・ `flash` は `install` を必ず先に動かす。

まとめ

- ・ `examples` 配列 + `-Dexample=<name>` で、ビルドのソースとなる `main.zig` を切り替えている
- ・ 1 つの `exe` から `installArtifact` / `addObjCopy(bin)` / `addObjCopy(hex)` / 外部コマンドステップ群を派生させて、各種成果物を生成
- ・ LLVM ツール群に依存する `disasm` / `mapfile` / `size` はオプティンのステップに分離してある
- ・ ステップは DAG で管理されており、`install` / `flash` 系の関係性は明示的な `dependOn` 呼び出しで構築される

次章では、本章で軽く触れた `addObjCopy` の中身、そして「ELF から `.bin` / `.hex` を作るとはどのような変換か」を、ファイル構造のレベルで見ていく。

第8章 ELF から .bin / .hex への変換

objcopy が抜き出すもの

学習目標

- ・ ELF ファイルが「実行ファイル」ではなく **メタデータ付きコンテナ** であることを理解する
 - ・ なぜ MCU への書き込みには「ELF そのまま」ではなく **.bin** や **.hex** が必要なのかを言える
 - ・ Raw binary と Intel HEX のフォーマットの違いをざっくり把握する
 - ・ `exe.addObjCopy({ .format = .bin })` が内部で何をしているのか想像できる
-

ELF はそのままでは焼けない

`zig build` が生成する `zig-out/bin/<name>` や `zig-out/firmware/<name>.elf` は **ELF (Executable and Linkable Format)** ファイルだ。これには以下のような情報が詰まっている。

- ・ ELF ヘッダ (マシン種別 = `EM_RISCV`、ABI 種別 = `EABI`、エントリポイントアドレスなど)
- ・ プログラムヘッダ (どの領域をどの仮想/物理番地にロードするか)
- ・ セクションヘッダ (`.text`, `.data`, `.bss`, `.vector_table`, ...)
- ・ シンボルテーブル / 文字列テーブル
- ・ デバッグ情報 (DWARF) — 本プロジェクトのリリースビルドではほぼ無い
- ・ 各種ノートセクション (`.note.gnu.build-id` 等)

これらの大半は **PC 上のローダ向けの情報** であって、MCU のフラッシュメモリには「ELF をそのままダンプ」しても動かない。必要なのは「FLASH `0x0800_0000` から始まる連続バイト列」だ。

`objcopy` の役目は、ELF のプログラムヘッダを辿って、**「フラッシュに焼くべき本体だけ」を抜き出して所定のフォーマットに変換すること**にある。

addObjCopy の中身

`build.zig` でこう呼ばれているステップ:

```
const bin = exe.addObjCopy({
    .format = .bin,
    .basename = b.fmt("{s}.bin", .{selected.name}),
});
```

これは Zig が内部に持つ `std.Build.Step.ObjCopy` ステップを作成する。中身を簡略化して書くと、おおよそ:

1. 入力 ELF を読み出す
2. プログラムヘッダのうち `PT_LOAD` (= ロード対象セグメント) を順に走査
3. それらを **VMA ではなく LMA に従って並べ直す**
4. セグメント間の隙間は 0 で埋めるか、HEX なら省略するか
5. 指定フォーマットで書き出す

第 4 章で扱った `.data` セクションが「VMA = RAM、LMA = FLASH」だった理由がここで効いてくる。objcopy は **LMA** を見て出力アドレスを決めるので、`.data` の中身は FLASH のしかるべき場所に並ぶ。その並んだ FLASH 像を MCU に書き込み、起動コードがその初期値を `_sdata → _sdata` でコピーする。

`.bin` (Raw binary) フォーマット

`.bin` は **アドレス情報を一切持たない、単なるバイト列**。

```
[FLASH 0x0800_0000 の中身] [FLASH 0x0800_0001 の中身] ... [FLASH 0x0800_xxxx の中身]
```

ファイルの先頭が、FLASH 上の **最初に内容を持つ番地** に対応する。本プロジェクトでは `.vector_table` が `0x0800_0000` から始まるので、`.bin` の先頭はベクタテーブルの最初のエントリ (= リセット時に飛ぶ `_start` のアドレス) になる。

利点と欠点

- ・ **シンプル極まりない**。ファイルサイズ ≒ 焼くべきバイト数
- ・ `minichlink` や DAPLink、SPI フラッシュライタなど、ほとんどのプログラマが対応
- ・ **アドレスを内包しない**ので、「`0x0800_0000` から書く」前提を呼び手が知っていなければならない
- ・ **飛び地** (例: FLASH `0x0800_0000` と `0x0800_3000` にだけ内容があるが、間は空) を表現すると、中間がゼロでパディングされてしまうため、ファイルが膨らむ

本プロジェクトは FLASH を `0x0800_0000` から連続して使うので、これらの欠点は実害が無い。ゆえに書き込み経路は `.bin` を主に使う。

.hex (Intel HEX) フォーマット

Intel HEX は **テキスト形式** で、アドレス情報を含む。

例 (`zig-out/firmware/blink.hex` の先頭数行):

```
:0200000040800F2
:1000000041000000550000005500000055000000B4
:1000100055000000550000005500000055000000A4
...
:000000001FF
```

各行 (レコード) はこんな構造:

- ・ **:** — 開始マーカ
- ・ **02** — バイト数 (2 桁 16 進)
- ・ **0000** — アドレス
- ・ **04** — レコードタイプ
- ・ **00** データ
- ・ **01** 終端
- ・ **04** 拡張リニアアドレス (上位 16-bit を指定)
- ・ 残り — データ + チェックサム

第 1 行 `:0200000040800F2` は「拡張リニアアドレス = `0x0800`、つまり以降のレコードのベースアドレスを `0x0800_0000` 系にする」という指示で、これによって 16-bit アドレスフィールドだけでは表現できない `0x0800_0000` を扱えるようにしている。

利点と欠点

- ・ **● アドレスを内包**するため、「焼く側はファイルの中身を見ればどこに置くべきか判断できる」
- ・ **●** 飛び地を許容するため、ブートローダ + アプリのような非連続配置にも向く
- ・ **●** テキストで人間にも読みやすい反面、同じ内容なら `.bin` の 2~3 倍のファイルサイズになる
- ・ **●** パーサがバイナリより複雑

最近の DAPLink を使った PC マウントによる「ドラッグ&ドロップ書き込み」では Intel HEX 推奨のケースもある (ボードによる)。互換性確保のために `.hex` も並行して出力している。

どちらをいつ使うか

シナリオ	おすすめ
minichlink (このプロジェクトの flash 経路)	.bin
ベンダ純正 ISP ツール	.hex (場合による)
「とりあえずバイト列だけ欲しい」「dd で書き込む」	.bin
異なるアドレス範囲を 1 ファイルで持ちたい	.hex

本プロジェクトでは両方を `zig-out/firmware/` に置くだけ置いておいて、実際に使うのは `.bin` だけ、という運用にしている。

ELF が生きる場面

`.elf` を書き込みに使わないと言っても、ELF を出力しないと困る場面もある。

- ・ **デバッグ** — `gdb` は ELF のシンボル情報を読むので、ステップ実行や `info symbol` に必須
- ・ **逆アセンブル / シンボルマップ** — 第 7 章で見た `llvm-objdump` / `llvm-nm` の入力には ELF
- ・ **サイズ計測** — `llvm-size` も ELF を読む

そのため、`build.zig` では `.elf` も `.bin` / `.hex` と並べて `zig-out/firmware/` に install している。

実際に覗いてみる

`zig build` 後の `zig-out/firmware/blinky.bin` を `xxd` で先頭だけ見ると、たとえばこんな具合になる (実機・最適化条件で異なる):

```
$ xxd zig-out/firmware/blinky.bin | head -4
00000000: 4100 0000 5500 0000 5500 0000 5500 0000  A...U...U...U...
00000010: 5500 0000 5500 0000 5500 0000 5500 0000  U...U...U...U...
00000020: 5500 0000 5500 0000 5500 0000 5500 0000  U...U...U...U...
00000030: 5500 0000 ...                               U...
```

- ・ 先頭 4 バイト `41 00 00 00` がベクタテーブル 0 番、つまり `_start` のアドレス。
- ・ 続く `55 00 00 00 ...` は `_default_irq_entry` のアドレスが繰り返されている (ベクタテーブルの埋め草)。

- ・ `_start` が `0x0800_0041` 付近、 `_default_irq_entry` が `0x0800_0055` 付近にあることが読み取れる。

これがそのまま CH32V003 の FLASH に書き込まれて、第 5 章の起動シーケンスが回り始める、というわけ。

まとめ

- ・ ELF は「メタデータ付き実行ファイル」で、MCU への書き込みには不要な情報も多く含む
- ・ `add0bjCopy` は LMA に従って **本当に焼くべきバイト列だけ** を抜き出し、Raw binary or Intel HEX に変換する
- ・ `.bin` はサイズ最小・アドレス情報なし、`.hex` はサイズ大・アドレス情報あり
- ・ ELF 自体はデバッグ / 逆アセンブル / サイズ計測に依然必要なので、並行して `zig-out/firmware/` に置く

次章では、ここで作った `.bin` を **実機の FLASH に焼く** ところ、つまり `minichlink` と `tools/flash.sh` の中身を見ていく。

第9章 minichlink で実機に書き込む

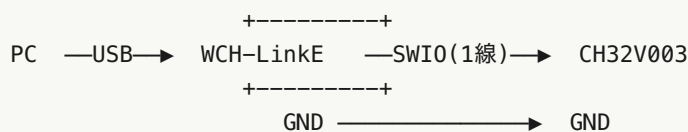
SWIO と tools/flash.sh

学習目標

- ・ CH32V003 のデバッグ / 書き込みインタフェース **SWIO** の位置付けを知る
- ・ **minichlink** がどんなツールなのか、なぜ ch32fun 側のリポジトリから持ってきているのかを理解する
- ・ 本プロジェクトの **tools/flash.sh** が何をしているかを 1 行ずつ読める
- ・ **zig build ... flash** が走ったときの呼び出し連鎖を辿れる

CH32V003 への書き込みインタフェース

CH32V003 のデバッグ・プログラミングは、**SWIO (1-wire シリアル)** で行う。ARM Cortex-M で言う SWD に似た「データ 1 本 + GND だけで全部できる」系のインタフェースだ。



PC 側のホストアダプタは **WCH-LinkE** (公式の dongle) や、「他の CH32V を WCH-LinkE 互換のプログラマに仕立てた」自作プログラマなどを使う。SWIO の物理プロトコルは公開されていて、PC ⇄ ホストアダプタ間は USB ベンダ独自プロトコルでつながる。

minichlink とは

[cnlohr/ch32fun](#) リポジトリの **minichlink/** に同梱されている、**WCH-LinkE 互換アダプタを叩く小さな CLI ツール**。

- ・ C で書かれている
- ・ 依存は **libusb-1.0** だけ
- ・ macOS / Linux / Windows でビルド可能
- ・ 動作モード: 書き込み、消去、検証、リセット、RAM ロードなど

本プロジェクトでは「**zig build flash** で MCU に焼く」ために、既存の minichlink をそのまま使う。Zig 側から minichlink を呼ぶ薄いシェルスクリプトを置くだけで、全部の経路が成立する。

minichlink のビルド

依存:

- ・ `gcc` / `clang` などの C コンパイラ
- ・ `make`
- ・ `libusb-1.0` (Mac は `brew install libusb`、Debian/Ubuntu は `libusb-1.0-0-dev`)
- ・ `pkg-config`

セットアップ:

```
cd ..
git clone https://github.com/cnlohr/ch32fun.git
make -C ch32fun/minichlink
```

成功すると `ch32fun/minichlink/minichlink` という実行ファイルが生まれる。これが本プロジェクトのデフォルトの参照先だ。

`tools/flash.sh` を読む

```
#!/usr/bin/env sh
set -eu

if [ "$#" -ne 1 ]; then
    echo "usage: tools/flash.sh <example>" >&2
    exit 2
fi

EXAMPLE="$1"
BIN="zig-out/firmware/${EXAMPLE}.bin"
MINICHLINK="../ch32fun/minichlink/minichlink"

if [ ! -x "$MINICHLINK" ]; then
    echo "minichlink not found: $MINICHLINK" >&2
    echo "Build it first: make -C ../ch32fun/minichlink" >&2
    exit 1
fi

if [ ! -f "$BIN" ]; then
    echo "missing binary: $BIN" >&2
    echo "run: zig build -Dexample=$EXAMPLE" >&2
    exit 1
fi
```

```
exec "$MINICHLINK" -w "$BIN" flash -b
```

順番に意味を取ると:

行	意味
<code>set -eu</code>	失敗したら即座に終了、未定義変数で死ぬ。シェルスクリプトの安全策。
引数チェック	サンプル名 1 つを必須にし、誤用を早期に止める
<code>MINICHLINK=../ch32fun/minichlink/minichlink</code>	「ch32fun_zig」とch32funを兄弟ディレクトリに置く」前提でパスを固定
存在チェック × 2	minichlink バイナリと <code>.bin</code> がそれぞれ無ければ親切なメッセージを出して終了
<code>exec minichlink -w "\$BIN" flash -b</code>	自分自身を minichlink で置き換えて呼び出す

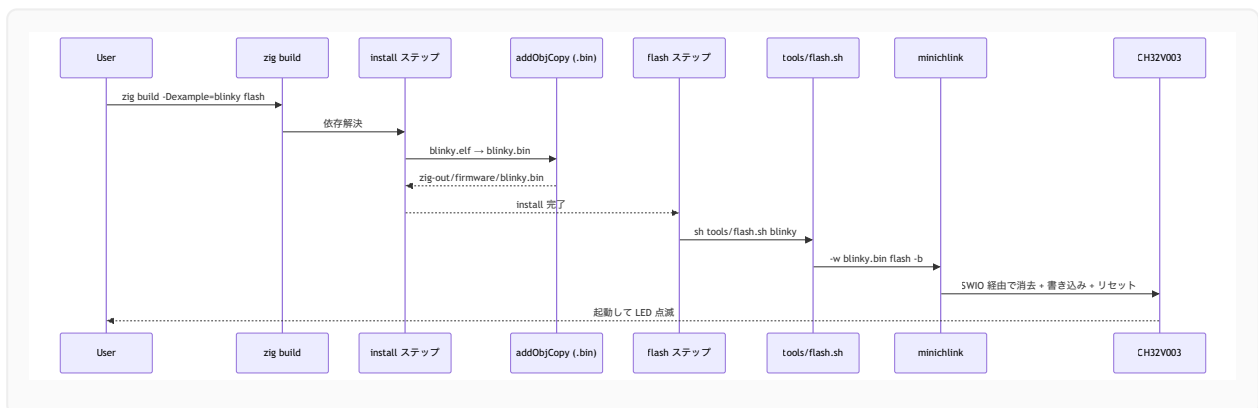
最後の `minichlink` のオプションは:

- ・ `-w <file>` — 書き込み対象ファイル
- ・ `flash` — 書き込み先 (FLASH)
- ・ `-b` — 書き込み後にリセットを発行し、動かす

これで「焼いたあとすぐ動き出す」までが 1 コマンドで完結する。

zig build flash の呼び出し連鎖

第 7 章のステップグラフでも触れたが、流れを再掲しておく。



トラブルシューティングの勘所

CH32V003 への書き込みは、周辺の小さなトラブルでつまずきがち。代表的なものを挙げておく。

1. minichlink がデバイスを見つけない

- ・ USB の権限。Linux なら `udev` ルールが必要。ch32fun リポジトリの `minichlink/49-wch.rules` を `/etc/udev/rules.d/` に入れて `udevadm control --reload-rules` し、ユーザを `plugdev` に追加。
- ・ Mac は通常そのまま使える。ただし他のアプリ (例: ベンダ純正の WCH-LinkUtility) が同じ USB をつかみっぱなしだと衝突する。

2. minichlink を見つけられない (zig build flash 時)

- ・ `tools/flash.sh` は `../ch32fun/minichlink/minichlink` を固定パスで見る。別の場所に置く場合はスクリプトを書き換えるか、シンボリックリンクで対応するのが楽。

3. 書き込みは成功するが LED が点滅しない

- ・ 第 5 章の起動シーケンスを疑う。特に `mtvec` 設定 / `.bss` クリア / `.data` コピーのいずれかが噛み合っていないと、ELF は焼けるけど挙動が変、になる。
- ・ まずは `zig build disasm` で `.lst` を見て、`_start` が `0x0800_xxxx` のどこに居て、`.vector_table` の最初のエントリがそれを指しているかを目視確認するのがおすすめ。

4. RDPR (リードプロテクション) で書き込み拒否

- ・ 既に他のファームが入っていてプロテクションが効いている個体に当たることがある。
`minichlink` には解除コマンドもあるので、ch32fun の README を参照のこと。

まとめ

- ・ 書き込みは WCH-LinkE 互換アダプタ + minichlink で行う、SWIO ベースのワイヤ 1 本構成
- ・ `tools/flash.sh` は「minichlink を `../ch32fun/minichlink/minichlink` で探して、`.bin` を焼く」だけの薄いラップで、これによって Zig 側のビルドシステムから minichlink を呼べる
- ・ `zig build flash` は `install` を必ず先行させるよう依存が貼ってあるので、ビルドし忘れて古い `.bin` を焼く事故は起きにくい

ここまでで「コード → ELF → `.bin` → MCU の FLASH」までの一直線の道筋が完全に揃ったことになる。次章からは視点を変えて、`main` から **HAL を呼ぶ側** のしくみ — つまりレジスタ抽象化と HAL の設計について見ていく。

第Ⅳ部 HAL の構造

レジスタ層からアプリ寄り HAL までを段で積む

第10章 MMIO とレジスタ抽象化

extern struct と *volatile T

学習目標

- ・ MMIO (Memory-Mapped I/O) の概念をおさらいする
- ・ `src/periph/registers.zig` がどう CH32V003 のレジスタ群を **Zig の型** に落とし込んでいるかを読み解く
- ・ `extern struct` / `@ptrFromInt` / `*volatile T` がそれぞれ何のために使われているのかを説明できる
- ・ ベースアドレス + フィールドオフセットの設計が、なぜ「うっかり間違えにくい」のかを理解する

MMIO のおさらい

ARM Cortex-M、RISC-V、SH、ほとんどの組み込み MCU は周辺機能を **MMIO** で公開している。「特定の物理番地に対する読み書きが、そのまま周辺レジスタへの操作になる」という仕組みだ。

CH32V003 では、例えば GPIO ポート D の出力レジスタは:

- ・ 番地: `0x4001_1410` (`GPIO_D_BASE` (= `0x4001_1400`) + オフセット `0x10`)
- ・ 振る舞い: ここに `u32` を書くと PD0~PD15 の出力が同時に変わる

これを生で扱うと、こうなる:

```
*(volatile uint32_t *)0x40011410 = 0x00000001; // PD0 を H
```

ポインタを `volatile` にしないとコンパイラが最適化で消したり、読み書き順序を入れ替えたりする。

このアプローチは「番地を間違えない」「フィールド名を間違えない」が **完全に開発者責任** になるので、規模が増えると間違いを生みやすい。`src/periph/registers.zig` はこれを Zig の `extern struct` で型付けし、ミスを構造化する。

ベースアドレスの宣言

```
pub const FLASH_BASE: usize = 0x08000000;  
pub const SRAM_BASE:  usize = 0x20000000;
```

```

pub const PERIPH_BASE: usize = 0x40000000;
pub const CORE_PERIPH_BASE: usize = 0xE0000000;

pub const APB2PERIPH_BASE: usize = PERIPH_BASE + 0x10000;
pub const APB1PERIPH_BASE: usize = PERIPH_BASE;
pub const AHBPERIPH_BASE: usize = PERIPH_BASE + 0x20000;

pub const GPIOA_BASE: usize = APB2PERIPH_BASE + 0x0800;
pub const GPIOC_BASE: usize = APB2PERIPH_BASE + 0x1000;
pub const GPIOD_BASE: usize = APB2PERIPH_BASE + 0x1400;
pub const I2C1_BASE: usize = APB1PERIPH_BASE + 0x5400;

pub const RCC_BASE: usize = AHBPERIPH_BASE + 0x1000;
pub const FLASH_R_BASE: usize = AHBPERIPH_BASE + 0x2000;

pub const PFIC_BASE: usize = CORE_PERIPH_BASE + 0xE000;
pub const SYSTICK_BASE: usize = CORE_PERIPH_BASE + 0xF000;

```

ここでやっていることは:

1. **メモリ空間全体のセクションベース** (FLASH / SRAM / 周辺 / コア内蔵周辺) を定数化
2. **バスごとのベース** (APB1 / APB2 / AHB) を派生
3. **個別周辺のベース** をさらに派生

CH32V003 のデータシートのメモリマップ表をそのまま Zig の `const` に写したものだ。 `usize` で書いておくことで、ポインタへの変換が型付きで通る (`@ptrFromInt` の引数として直接使える)。

レジスタブロックを `extern struct` で記述

代表例として RCC (Reset and Clock Control) のレジスタ群。

```

pub const RccRegs = extern struct {
    CTLR: u32,
    CFGR0: u32,
    INTR: u32,
    APB2PRSTR: u32,
    APB1PRSTR: u32,
    AHBPCENR: u32,
    APB2PCENR: u32,
    APB1PCENR: u32,
    RESERVED0: u32,
    RSTSCKR: u32,
};

```

ポイント:

- ・ **extern struct** は C ABI に準拠したレイアウトを保証する。Zig の通常 **struct** だとフィールド並び替えが起こる可能性があるが、**extern struct** ではフィールド順 = メモリ上の順。
- ・ **フィールド順 = データシートのオフセット表の順**。RCC の場合、オフセット 0x00, 0x04, 0x08, ... と並ぶレジスタを単に上から書いていけばよい。
- ・ **RESERVED0: u32** で予約領域を埋める。これがあるおかげで、後続フィールドのオフセットがズレない。
- ・ **アライメント** は自然に整う。**u32** だけで構成しているなら 4 バイト境界に揃う。

I2C のように 16-bit レジスタが並ぶケース

```
pub const I2cRegs = extern struct {  
    CTLR1: u16,  
    RESERVED0: u16,  
    CTLR2: u16,  
    RESERVED1: u16,  
    OADDR1: u16,  
    RESERVED2: u16,  
    OADDR2: u16,  
    RESERVED3: u16,  
    DATAR: u16,  
    RESERVED4: u16,  
    STAR1: u16,  
    RESERVED5: u16,  
    STAR2: u16,  
    RESERVED6: u16,  
    CKCFGR: u16,  
    RESERVED7: u16,  
};
```

CH32V の I2C は「16-bit レジスタが 4 バイトおきに並ぶ」という変則レイアウト。これを忠実に表現するため、各 **u16** の後ろに **RESERVED?: u16** を入れて 4 バイトに揃えている。こうすると **i2c1().CTLR2 = ...** のような自然な記述が、正しい番地への 16-bit 書き込みになる。

ベース番地を ***volatile T** に変換するヘルパ

```
pub fn rcc() *volatile RccRegs {  
    return @ptrFromInt(RCC_BASE);  
}  
  
pub fn gpioA() *volatile GpioRegs {  
    return @ptrFromInt(GPIOA_BASE);  
}
```

```

}

pub fn i2c1() *volatile I2cRegs {
    return @ptrFromInt(I2C1_BASE);
}

pub fn systick() *volatile SysTickRegs {
    return @ptrFromInt(SYSTICK_BASE);
}

```

- `@ptrFromInt(addr)` — 整数を「型付きポインタ」に変換する Zig の組み込み関数。戻り型が `*volatile RccRegs` なら、そのアドレスを RCC レジスタブロックへのポインタとみなす。
- `*volatile T` — 「`T` への volatile ポインタ」。経由する読み書きは最適化で消されたり並び替えられたりしない。

呼び出し側はこうなる:

```

const rcc = regs.rcc();
rcc.APB2PCENR |= regs.RCC_APB2_GPIOD;

```

これだけで `0x4002_1018 |= 0x00000020` 相当の書き込みが正しく発行される。番地もシフト幅も書き間違える余地が無い。

ビットマスクは別途定数で

```

pub const RCC_APB2_AFIO: u32 = 0x00000001;
pub const RCC_APB2_GPIOA: u32 = 0x00000004;
pub const RCC_APB2_GPIOC: u32 = 0x00000010;
pub const RCC_APB2_GPIOD: u32 = 0x00000020;
pub const RCC_APB1_I2C1: u32 = 0x00200000;

```

ビットマスクを **コードシンボルとして名前付け**しておくのが、数値ベタ書きとの大きな違い。これがあると `grep` や IDE のジャンプで「このビットを使っている箇所」を一発で見つけられる。命名規則は「ペリフェラル_バス_機能」とすると、同じ APB2 系のクロックを横断的に見たくなったときに揃って並ぶ。

PFIC の有効化 — 動的にアドレスを作る例

```

pub fn pficEnableIrq(irqn: u8) void {
    const reg_index: usize = irqn / 32;
    const bit: u32 = @as(u32, 1) << @as(u5, @intCast(irqn % 32));
    const addr = PFIC_BASE + 0x100 + (reg_index * 4);
}

```

```
const reg: *volatile u32 = @ptrFromInt(addr);
reg.* = bit;
}
```

PFIC の IENR (Interrupt Enable Register) は 32 ビット幅 × 複数本という構成で、IRQ 番号によって書き込むレジスタが変わる。これを毎回専用の `extern struct` フィールドに割り付けるのは面倒なので、ここだけは **アドレス計算で動的にポインタを作る** 書き方をしている。

`@as(u5, @intCast(irqn % 32))` の `u5` キャストは、RISC-V の論理シフト命令が「シフト量は下位 5 ビットだけ意味を持つ」ことを Zig の型システム上で表現するためのもの。余計な広い型でシフトしようとするとも Zig のコンパイラが文句を言うので、明示的に絞っている。

なぜこの抽象化が「ちょうど良い」のか

他言語 / 他フレームワークでは、以下のような重い抽象化もよく見る:

- ・ HAL ライブラリ (STM32CubeMX 系) — 各レジスタを構造化したうえで、さらに `HAL_GPIO_Init(GPIOA, &init_typedef)` のような high-level API を被せる
- ・ Rust の `svd2rust` — SVD XML から自動生成された型安全なレジスタアクセス API

これらは安全性と表現力で勝るが、学習コストとコードサイズで重くなる。一方、本プロジェクトはあくまで「MCU の生のレジスタ表を Zig の `extern struct` に写経した、ごく薄いレイヤ」に留めている。これにより:

- ・ データシートと 1:1 で照合できる (調査・デバッグが楽)
- ・ ビット幅・オフセットを把握しているなら、ほぼ生 C と同じ手触り
- ・ HAL レイヤを上はどう被せるかは別途自由に設計できる (本プロジェクトは `src/hal/*.zig`)

「ちょうど薄く、ちょうど型安全」という塩梅で、教育・実験目的のコードベースに向けた抽象度になっている。

まとめ

- ・ ベースアドレス + `extern struct` + `@ptrFromInt` + `*volatile T` の組み合わせで、MMIO を **データシートとほぼ 1:1 の Zig 型** に落とし込んでいる
- ・ 予約領域は `RESERVED?: u32` のようなフィールドで埋め、オフセットを保つ
- ・ ビットマスクはシンボル名で定数化することで、「どこで何のビットを使っているか」が grep 可能になる
- ・ PFIC のように動的なアドレス計算が必要な箇所は、個別関数で吸収

次章では、この薄いレジスタ層の上に被せた **HAL (GPIO / SysTick / 入力)** を読んでいき、アプリ層から見たときの API がどう作られているかを見る。

第11章 HAL — GPIO と SysTick

Pin 型と delayMs の作り

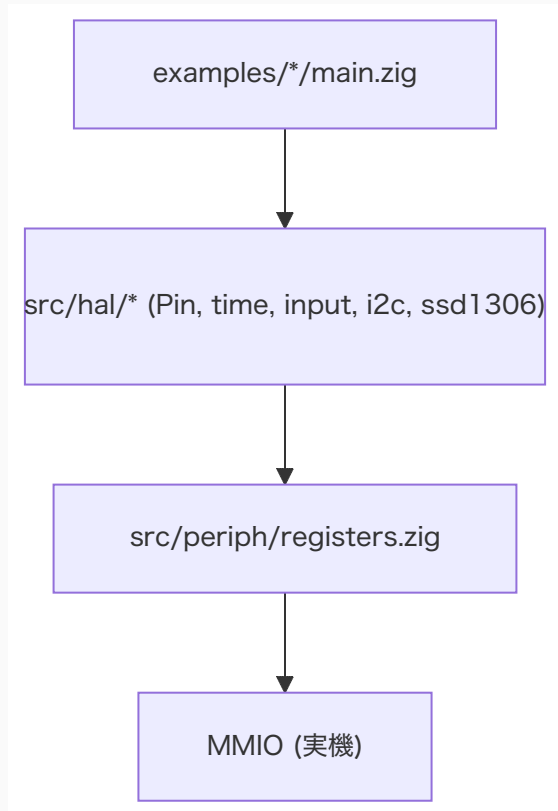
学習目標

- ・ `src/hal/gpio.zig` の `Pin` 型がどんな API を提供しているか把握する
 - ・ `src/hal/time.zig` の `delayMs` / `systick.init` がどう SysTick を使っているか読み解ける
 - ・ `src/hal/input.zig` のような薄いラップが「GPIO の使い方の例」を兼ねていることを理解する
 - ・ レジスタ層 (第 10 章) からアプリ層への抽象化の階段がどう積まれているかを掴む
-

HAL の位置付け

第 10 章で見たレジスタ層は「データシートのオフセット表を Zig 型に写経した」もので、触り心地は半分ハードに近い。そこにアプリが直接接触すると、- ビットマスク手書き、- クロック有効化忘れ、- レジスタオフセット計算ミス、が必ず起きる。

HAL レイヤーは、これらを「ポート / ピン / モード」のような **MCU 寄りのドメインモデル** に持ち上げる役目を持つ。ただし汎用ライブラリではないので、「GPIO ピンを 1 本動かす」「SysTick で時間を測る」のような最小限の道具立てに絞っている。



src/hal/gpio.zig を読む

Port / Mode 列挙

```
pub const Port = enum {
    A,
    C,
    D,
};

pub const Mode = enum(u4) {
    input_analog = 0,
    input_floating = 4,
    input_pull = 8,
    output_pp_10mhz = 1,
    output_pp_2mhz = 2,
    output_pp_30mhz = 3,
    output_od_10mhz = 5,
    output_od_2mhz = 6,
    output_od_30mhz = 7,
    output_af_pp_10mhz = 9,
    output_af_pp_2mhz = 10,
    output_af_pp_30mhz = 11,
```



```

output_af_od_10mhz = 13,
output_af_od_2mhz = 14,
output_af_od_30mhz = 15,
};

```

CH32V003 の GPIO は、CFGLR レジスタの **4 ビット = 1 ピン** で「方向 + 速度 + プルアップ/オーブンドレイン」を一括指定する設計。Mode の値は、そのまま CFGLR に流す 4-bit パターンになっている。

そして Mode を **enum(u4)** にしたのが効いていて、**@intFromEnum(mode)** でそのまま 4-bit 値として扱える。値の妥当性 (0~15 の範囲) は型レベルで保証される。

Pin 型

```

pub const Pin = struct {
    port: Port,
    index: u4,

    pub fn configure(self: Pin, mode: Mode) void {
        const g = self.gpio();
        const shift: u5 = @as(u5, self.index) * 4;
        const mask: u32 = @as(u32, 0xF) << shift;
        const value: u32 = @as(u32, @intFromEnum(mode)) << shift;

        g.CFGLR = (g.CFGLR & ~mask) | value;
    }

    pub fn write(self: Pin, value: bool) void {
        const g = self.gpio();
        const bit: u32 = @as(u32, 1) << self.index;
        if (value) {
            g.BSHR = bit;
        } else {
            g.BSHR = bit << 16;
        }
    }

    pub fn toggle(self: Pin) void {
        self.write(!self.readOut());
    }

    pub fn read(self: Pin) bool {
        const g = self.gpio();
        return ((g.INDR >> self.index) & 0x1) != 0;
    }

    pub fn readOut(self: Pin) bool {
        const g = self.gpio();
        return ((g.OUTDR >> self.index) & 0x1) != 0;
    }
}

```

```

fn gpio(self: Pin) *volatile regs.GpioRegs {
    return switch (self.port) {
        .A => regs.gpioA(),
        .C => regs.gpioC(),
        .D => regs.gpioD(),
    };
}

};

pub fn pin(comptime port: Port, comptime n: u4) Pin {
    return .{ .port = port, .index = n };
}

```

注目したい設計が 3 点ある。

1. `Pin` は値型で軽い

`Pin` は `Port` (列挙) と `index: u4` のペアだけ。関数の引数として渡しても、ほぼレジスタ 1 本分のデータしか動かない。結果として、`led.toggle()` のような呼び出しはインライン化されて生のレジスタ操作と遜色ない出力になる。

2. `configure` — リード・モディファイ・ライト

```

const mask: u32 = @as(u32, 0xF) << shift;
g.CFGLR = (g.CFGLR & ~mask) | value;

```

`CFGLR` は他のピンの設定も同居しているので、該当 4 ビットだけを差し替える RMW を行う必要がある。これを「ピンを設定する」1 メソッドに閉じ込めたのが API としての価値。

3. `write` — `BSHR` で 1 ライトのアトミック化

```

if (value) {
    g.BSHR = bit;
} else {
    g.BSHR = bit << 16;
}

```

`BSHR` (Bit Set/Reset) は「**下位 16 ビットを 1 にすると対応ピンが H、上位 16 ビットを 1 にすると L**」という仕様。つまり「他のピンに副作用を出さず、1 ライトでビットを立てる/落とす」ことができる。OUTDR を RMW すると、タイミング次第で割り込みハンドラと競合し得る (片方が古い値を書き戻す) ので、こちらの方が安全。

enablePortClock / enableAllClocks

```
pub fn enablePortClock(port: Port) void {
    const rcc = regs.rcc();
    switch (port) {
        .A => rcc.APB2PCENR |= regs.RCC_APB2_GPIOA,
        .C => rcc.APB2PCENR |= regs.RCC_APB2_GPIOC,
        .D => rcc.APB2PCENR |= regs.RCC_APB2_GPIOD,
    }
}

pub fn enableAllClocks() void {
    const rcc = regs.rcc();
    rcc.APB2PCENR |= regs.RCC_APB2_AFIO | regs.RCC_APB2_GPIOA | regs.RCC_APB2_GPIOC |
    regs.RCC_APB2_GPIOD;
}
```

CH32V003 では「使う前にクロックを供給」が必須で、そこをサボると **レジスタ書き込みが効かない (= 静かに失敗する)** という最悪のバグになる。HAL 側で「ポート単位で有効化」 / 「全 GPIO + AFIO 一括」を関数化してある。

blinky の例:

```
fun.system.init(.{});
fun.gpio.enableAllClocks(); // ← これを忘れると LED が点かない
const led = fun.gpio.pin(.D, 0);
led.configure(.output_pp_10mhz);
```

src/hal/time.zig を読む

delayMs — シンプルなビジーウェイト

```
pub fn delayMs(ms: u32) void {
    const ticks_per_ms = system.core_clock_hz / 1000;
    const wait_ticks = ms * ticks_per_ms;
    const start = regs.systick().CNT;

    while (@as(u32, regs.systick().CNT -% start) < wait_ticks) {}
}
```

- ・ **-%** は Zig のラップアラウンド付き減算。SysTick の **CNT** は 32-bit で自由走行しており、オーバーフローしうるので、ラップを許す減算で経過 tick を計算する。

- ・ `system.core_clock_hz / 1000` — クロックが 48MHz なら 48000 tick = 1 ms。
`system.init` でクロックを設定したあとに呼ぶ前提。
- ・ 何も寝かさない単純なビジーループ。省電力にはほぼ寄与しないが、タイミングのジッタは最小。

systick.init — 周期割り込み有効化

```
pub const systick = struct {
    pub fn init(tick_hz: u32) void {
        const st = regs.systick();

        systick_ticks_per_irq = system.core_clock_hz / tick_hz;
        if (systick_ticks_per_irq == 0) {
            systick_ticks_per_irq = 1;
        }

        st.CTLR = 0;
        st.CMP = systick_ticks_per_irq - 1;
        st.CNT = 0;
        st.SR = 0;
        systick_ticks = 0;

        // Keep SysTick running from HCLK without ISR binding in this stage.
        st.CTLR = regs.SYSTICK_CTLR_STE | regs.SYSTICK_CTLR_STCLK;
    }

    pub fn nowTicks() u64 {
        const st = regs.systick();
        if (systick_ticks_per_irq == 0) return 0;
        return st.CNT / systick_ticks_per_irq;
    }

    pub fn onTick(handler: *const fn () callconv(.c) void) void {
        tick_handler = handler;
    }
};
```

- ・ `tick_hz` を引数に取り、「1 tick あたり何 SysTick カウントか」を計算 (`systick_ticks_per_irq`)
- ・ CTLR を一旦 0 にしてから設定し直すことで、二重起動時の不整合を防ぐ
- ・ `onTick(handler)` で **ユーザ定義コールバック** を登録できる。中身は第 6 章で見た `_systick_irq_body` から呼ばれる

`timer_irq` サンプルではこのコールバックを設定し、「定期的に LED フラグを反転 → メインループから書き出し」という SysTick 周期処理のパターンを実装している。

systickInterruptBody

```
pub fn systickInterruptBody() callconv(.c) void {
    const st = regs.systick();

    st.CMP += systick_ticks_per_irq;
    st.SR = 0;
    systick_ticks += 1;

    if (tick_handler) |handler| {
        handler();
    }
}
```

- ・ `CMP` を「次の発火点」に進める。 `+=` はラップアラウンド加算。
- ・ `SR` を 0 にして割り込みフラグを下ろす。
- ・ グローバルカウンタをインクリメント。
- ・ 登録があれば `handler` を呼ぶ。

ハンドラ本体を `_systick_irq_body` から呼ぶことで、アセンブリの「全レジスタ退避部」と「実際の処理」を分離している (第 6 章参照)。

src/hal/input.zig — HAL の最小ラッパの例

```
pub fn initButtonPd1Pullup() void {
    gpio.enablePortClock(.D);

    const button = gpio.pin(.D, 1);
    button.configure(.input_pull);
    button.write(true);
}

pub fn isButtonPressed() bool {
    return !gpio.pin(.D, 1).read();
}
```

- ・ `PD1` を入力 + プルアップに設定し、`BSHR` 経由で `OUTDR` のビットを立てて **内部プルアップを有効化**する (CH32V003 の入力プルアップは `OUTDR` のビットで決まる)
- ・ ボタンは「押すと GND に落ちる」アクティブ Low 配線を想定。 `read()` の `false` = 押下、なので返却時に反転している

`oled` サンプルがそのまま呼んでいる:

```
fun.input.initButtonPd1Pullup();
...
const pressed = fun.input.isButtonPressed();
if (pressed and !prev_button) {
    fast_mode = !fast_mode;
}
prev_button = pressed;
```

「ピン番号と配線がアプリ側のドメイン知識として漏れない」ようにする、という HAL の本来の目的の小さな例だ。

まとめ

- ・ `Pin` 型は値型として軽く、`configure` / `write` / `toggle` / `read` で MCU のレジスタ操作を集約
- ・ `write` は副作用回避のため `BSHR` の 1 ライトを使う
- ・ `delayMs` は `SysTick` の自由走行カウンタとラップアラウンド減算で実装
- ・ `systick.init` + `onTick` で周期割り込みハンドラを差し込める設計
- ・ `input.zig` は「HAL の使い方の例」を兼ねた最薄ラップで、アプリ側にピン番号が漏れない

次章では、I2C と SSD1306 — 「複数バイト通信を行う周辺ドライバ」を読んでいく。GPIO よりも状態機械が深く、タイムアウト処理も必要になる、もうひとつ大きな抽象化レイヤだ。

第12章 HAL — I2C と SSD1306

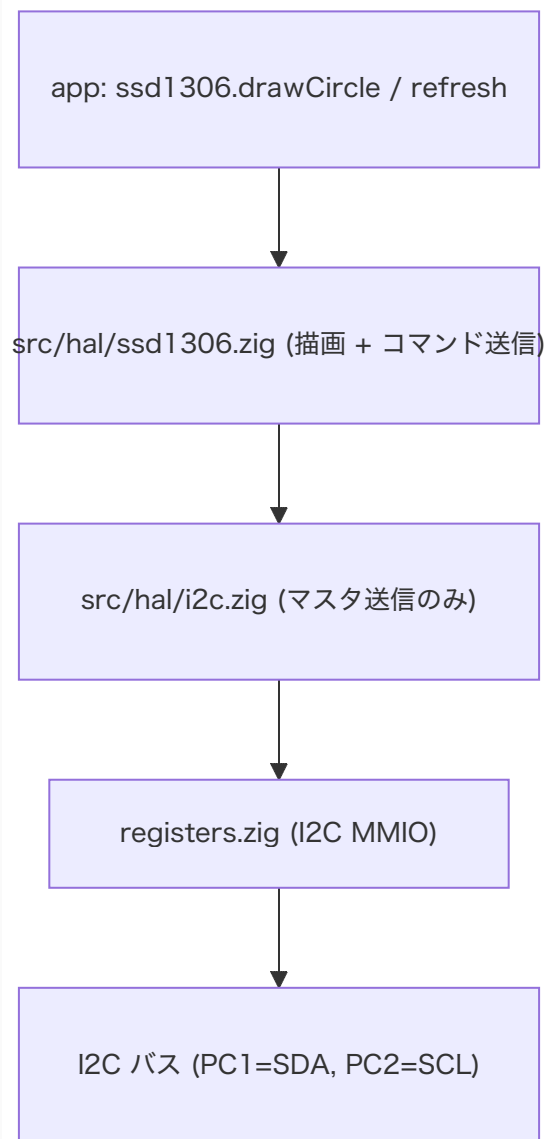
ブロッキング送信と 1024B フレームバッファ

学習目標

- ・ 本プロジェクトの I2C ドライバが「マスタ送信のみのブロッキング実装」であることを把握する
 - ・ I2C のイベントフラグ (STAR1 / STAR2) を組み合わせてステートを判定する作法を読む
 - ・ SSD1306 ドライバが **128×64 のフレームバッファを 1024 バイトの RAM** に持ち、32 バイト単位で送信する設計の理由を理解する
 - ・ フレームバッファ → I2C 経路の各ステップで RAM/ROM/CPU 時間がどう使われているかを掴む
-

なぜ I2C と SSD1306 を一緒に扱うか

`src/hal/i2c.zig` と `src/hal/ssd1306.zig` は、アプリ視点では別物だが、「下層 = I2C のブロッキング送信」 / 「上層 = ピクセル描画 + パネル送信」の縦の関係になっている。SSD1306 のドライバが大きいのは、ほとんどが「描画系の操作」で、I2C 越しの操作はほんの一部 (init + refresh) に過ぎない。



src/hal/i2c.zig を読む

設計方針

- ・ **マスタ送信のみ**。受信や複数マスタ対応は意図的に省略。 SSD1306 は単方向だけで使えるため、これでスコープが収まる。
- ・ **全部ブロッキング**。割り込みを使わない。 タイムアウトは「規定回数のループ」というスピンカウンタで実装。
- ・ **Error** 列挙で何でこけたか分かる。


```
pub const Error = error{
    BusyTimeout,
    MasterModeTimeout,
    TxModeTimeout,
    TxEmptyTimeout,
    TxDoneTimeout,
};
```

初期化

```
pub fn initI2c1FastMode() void {
    const rcc = regs.rcc();

    rcc.APB2PCENR |= regs.RCC_APB2_GPIOC;
    rcc.APB1PCENR |= regs.RCC_APB1_I2C1;

    gpio.pin(.C, 1).configure(.output_af_od_10mhz); // SDA
    gpio.pin(.C, 2).configure(.output_af_od_10mhz); // SCL

    resetAndSetup();
}
```

- ・ GPIOC と I2C1 のクロックを供給
- ・ PC1 / PC2 を **オープンドレインの代替機能** (= I2C のフィジカル割り当て) に設定
- ・ `resetAndSetup()` で I2C1 を初期化

`resetAndSetup` は CTLR1 / CTLR2 / CKCFGR を順に設定し、最終的に PE (ペリフェラル有効) と ACK を立てる典型的な手順。周波数は **1MHz Fast mode** で動かす。

```
const bus_clock_hz: u32 = 1_000_000;
const logic_clock_hz: u32 = 2_000_000;
```

- ・ `bus_clock_hz` — SCL の周波数
- ・ `logic_clock_hz` — I2C ペリフェラル内部のロジッククロック (`CTLR2.FREQ` に書く値の元)

イベントフラグの組み合わせ

```
const evt_master_mode_select: u32 = 0x00030001;
const evt_master_tx_selected: u32 = 0x00070082;
const evt_master_byte_transmitted: u32 = 0x00070084;

fn checkEvent(mask: u32) bool {
    const i2c = regs.i2c1();
    const status: u32 = @as(u32, i2c.STAR1) | (@as(u32, i2c.STAR2) << 16);
```

```

    return (status & mask) == mask;
}

```

CH32V / STM32 系の I2C はベンダドキュメントで「イベント」と呼ばれる **複数フラグの組合せ** で状態を判定する。上位 16-bit に **STAR2**、下位 16-bit に **STAR1** を積んで 1 つの 32-bit 比較に持ち込む書き方は、多くのベンダ純正コードでも見られる典型イディオム。

ブロッキング送信

```

pub fn writeBlocking7bit(addr: u7, bytes: []const u8) Error!void {
    const i2c = regs.i2c1();

    var timeout: i32 = timeout_max;
    while ((i2c.STAR2 & regs.I2C_STAR2_BUSY) != 0 and timeout > 0) : (timeout -= 1) {}
    if (timeout <= 0) return Error.BusyTimeout;

    i2c.CTLR1 |= regs.I2C_CTLR1_START;

    timeout = timeout_max;
    while (!checkEvent(evt_master_mode_select) and timeout > 0) : (timeout -= 1) {}
    if (timeout <= 0) return Error.MasterModeTimeout;

    i2c.DATAR = @as(u16, addr) << 1;

    timeout = timeout_max;
    while (!checkEvent(evt_master_tx_selected) and timeout > 0) : (timeout -= 1) {}
    if (timeout <= 0) return Error.TxModeTimeout;

    for (bytes) |b| {
        timeout = timeout_max;
        while ((i2c.STAR1 & regs.I2C_STAR1_TXE) == 0 and timeout > 0) : (timeout -=
1) {}
        if (timeout <= 0) return Error.TxEmptyTimeout;

        i2c.DATAR = b;
    }

    timeout = timeout_max;
    while (!checkEvent(evt_master_byte_transmitted) and timeout > 0) : (timeout -= 1)
{}
    if (timeout <= 0) return Error.TxDoneTimeout;

    i2c.CTLR1 |= regs.I2C_CTLR1_STOP;
}

```

順番が完全に「I2C のシーケンス図そのもの」になっている。

1. バスの BUSY が下りるのを待つ
2. START を発行

3. マスタモードになるのを待つ (`evt_master_mode_select`)
4. アドレス + W を送る (`addr << 1`)
5. 「送信器として選ばれた」状態 (`evt_master_tx_selected`) を待つ
6. 送りたいバイトをすべて DATAR に書く (TXE フラグを確認しながら)
7. 最後の 1 バイトが送信完了 (`evt_master_byte_transmitted`) するのを待つ
8. STOP を発行

各グループで `timeout` をデクリメントして「ハングしない安全網」を入れている。ハードの何かがおかしいとき (たとえばプルアップが付いてなくて SCL が H に戻らない)、`BusyTimeout` で抜けてアプリにエラーを返せるので、後追いデバッグがやりやすい。

`src/hal/ssd1306.zig` を読む

フレームバッファのデザイン

```
pub const width: u8 = 128;
pub const height: u8 = 64;
pub var buffer: [width_us * height_us / 8]u8 = [_]u8{0} ** (width_us * height_us / 8);
```

- ・ $128 \times 64 = 8192$ pixel。1 ピクセル 1 bit なので $8192 / 8 = 1024$ バイト。
- ・ SRAM は 2KB しか無く、その **半分をフレームバッファが占有する**。シングルバッファだけ持つのはこの制約のため。
- ・ 「縦 8 ピクセルが 1 バイト = 縦方向ストライプ」というレイアウト。SSD1306 のページモード GDDRAM 配置 (YYYxxxxx) にそのまま合うので、シリアル送信時に変換しなくて良い。

ピクセル描画

```
pub fn drawPixel(x: i16, y: i16, color: bool) void {
    if (x < 0 or y < 0) return;
    if (x >= width or y >= height) return;

    const xu: usize = @intCast(x);
    const yu: usize = @intCast(y);
    const addr = xu + @as(usize, width) * (yu / 8);
    const mask: u8 = @as(u8, 1) << @as(u3, @intCast(yu & 7));

    if (color) {
        buffer[addr] |= mask;
    } else {
        buffer[addr] &= ~mask;
    }
}
```

```
}
}
```

- `addr = x + 128 * (y / 8)` — ページ単位の縦ストライプにマップ
- `mask = 1 << (y % 8)` — ページ内のビット位置
- `i16` で受けてレンジ外を早めに弾く。描画コードを書くアプリ側は座標が負になりがちなので、ここでの境界チェックがあると上位コードがシンプルに保てる

コマンドとデータの送信

```
fn cmd(command: u8) Error!void {
    var pkt = [_]u8{ 0x00, command };
    try i2c.writeBlocking7bit(i2c_addr, pkt[0..]);
}

fn data(chunk: []const u8) Error!void {
    var pkt: [packet_size + 1]u8 = undefined;
    pkt[0] = 0x40;
    @memcpy(pkt[1 .. 1 + chunk.len], chunk);
    try i2c.writeBlocking7bit(i2c_addr, pkt[0 .. 1 + chunk.len]);
}
```

- SSD1306 の **Co/D/C** プロトコルそのまま。先頭バイトが `0x00` ならコマンド、`0x40` ならデータ。
- データは `packet_size = 32` バイト単位で送る。SSD1306 自体は任意長を受けられるが、切れ目を入れることでスタックやループ実装が単純になる。

画面更新

```
pub fn refresh() !void {
    try cmd(0x21);
    try cmd(0);
    try cmd(width - 1);

    try cmd(0x22);
    try cmd(0);
    try cmd(7);

    var i: usize = 0;
    while (i < buffer.len) : (i += packet_size) {
        const end = @min(i + packet_size, buffer.len);
        try data(buffer[i..end]);
    }
```

```
}
}
```

- ・ `0x21` 系コマンドで「列アドレス範囲 = 0~127」「ページアドレス範囲 = 0~7」を設定し、続けて GDDRAM への書き込みモードに入る
- ・ 1024 バイトのフレームバッファを 32 バイトずつ送信。ループ回数は 32 回固定

このループの所要時間が、描画 → 表示までの体感レイテンシそのものなので、I2C 速度を上げる (今回は 1MHz Fast mode) と気持ちよく動く。

上位 API の役割分担

- ・ 描画系 (`drawPixel` / `drawLine` / `drawCircle` / `drawRoundRect` / ...) は **フレームバッファだけ** を触る
- ・ バス側を叩くのは `initI2c` / `initPanel` / `refresh` の 3 つだけ
- ・ アプリは描画関数で構図を組み立て、最後に 1 回 `refresh()` する、という「ダブルバッファ的」な書き口になる

このおかげで、描画コードは I2C のタイムアウトや状態遷移を意識せずに済む。第 11 章で見た GPIO HAL より一段ドメインに近い API、という位置付け。

回転とビットマップ

`drawStrRot` / `drawCharRot` / `drawImageRot` は `Rotation = .deg0/.deg90/.deg180/.deg270` を受け取り、回転後の座標を `rotatePoint` 関数で計算してから `blendPixel` を呼ぶ。中間バッファを持たない (= 1024 バイト以外の絵用 RAM を使わない) のは 2KB しかない SRAM への配慮で、計算量が多い代わりにメモリは喰わない、というトレードオフを選んでいる。

```
fn rotatePoint(sx: i16, sy: i16, w: i16, h: i16, rotation: Rotation) struct { x: i16,
y: i16 } {
    return switch (rotation) {
        .deg0 => .{ .x = sx, .y = sy },
        .deg90 => .{ .x = h - 1 - sy, .y = sx },
        .deg180 => .{ .x = w - 1 - sx, .y = h - 1 - sy },
        .deg270 => .{ .x = sy, .y = w - 1 - sx },
    };
}
```

回転変換を一箇所に閉じ込めて、上位のループは「(sx, sy) について dest を求めて、そこに `blendPixel`」という統一形で書けるようにしてある。

全体を貫くデザイン原則

第 10 章のレジスタ層 → 第 11 章の GPIO/SysTick HAL → 第 12 章の I2C/SSD1306 HAL と段を上がってきて、設計指針はぶれていない:

1. **下層は MCU の語彙のままに薄く写経**。命名は基本的にデータシート通り。
2. **上層はアプリ側のドメインに寄せる**。ただし汎用化はしない。「このプロジェクトの用途」に必要な分だけ作る。
3. **割り込みやイベント駆動は最小限**。タイミングが厳しくない箇所はビジーループ + タイムアウトでよしとする。
4. **RAM は神聖**。2KB しかないので、描画バッファ 1 枚で全部こなす。中間バッファは持たない。
5. **エラーは Zig の `!T` で素直に上げる**。SSD1306 側の `Error` は I2C の `Error` をそのまま採用 (`pub const Error = i2c.Error;`)。

これらを通して読むと、「組み込みの薄い HAL を Zig で書くなら、だいたいこの粒度に落ち着くだろう」という素朴な良いリファレンスになっている。

まとめ

- ・ I2C はマスタ送信のみのブロッキング実装。タイムアウト付きスピンウェイトで安全に止まれる
- ・ SSD1306 ドライバは 1024 バイトのシングルフレームバッファ + 32 バイト単位送信
- ・ 描画 API はフレームバッファのみを操作し、バスは `refresh()` 1 箇所に集約
- ・ 回転処理はピクセル単位で計算し、中間バッファを持たない方針で 2KB SRAM に収まるようにしてある

本書の結び

ここまでで、`ch32fun_zig` が「Zig 一本で CH32V003 (RV32EC) のファームウェアを生成・書き込み」までやり切るための **全ての段** を歩いてきた。

- ・ **ターゲット指定**: `Target.Query` で RV32EC を組む (第 2 章)
- ・ **クロスコンパイル基盤**: LLVM + lld + compiler_rt が Zig に同梱 (第 3 章)
- ・ **配置**: リンカスクリプトで FLASH/RAM を割り当て (第 4 章)
- ・ **起動**: `_start` から `main()` までの整地作業 (第 5 章)
- ・ **割り込み**: ベクタテーブルと SysTick (第 6 章)
- ・ **ビルドパイプライン**: `build.zig` のステップグラフ (第 7 章)
- ・ **書き込み用フォーマット**: ELF → `.bin` / `.hex` (第 8 章)

- ・ **書き込み**: minichlink + SWIO (第 9 章)
- ・ **MMIO 抽象化**: `extern struct` と `*volatile T` (第 10 章)
- ・ **GPIO/SysTick HAL**: アプリ寄りの薄いラッパ (第 11 章)
- ・ **I2C/SSD1306 HAL**: I/O + 描画の縦の積み重ね (第 12 章)

それぞれの章は独立に読んでも価値があるが、通して読むと **「組み込み RISC-V を Zig で扱う時、各層がどれだけ薄くたためるか」** の一つの実装例として鑑賞できる構成になっている。自分のターゲット MCU が CH32V003 でなくとも、RV32IM 系の MCU や Cortex-M に Zig を持ち込みたい場合の道標になれば幸いだ。

第Ⅴ部 言語仕様と標準ライブラリ

freestanding + RV32EC で使える Zig の範囲を見極める

第13章 本プロジェクトで使える Zig 言語機能と std

使える / 使えない / 太る の見分け方

学習目標

- ・「Zig が書ける」と「この環境で動く Zig が書ける」ことの差を把握する
- ・本プロジェクト (`freestanding` + `link_libc=false` + RV32EC + RAM 2KB) で 安全に使える言語機能 / std の範囲を知る
- ・動くがコードサイズ・RAM を急に食う機能 (Debug ビルドの panic、大きい型、format) の特徴を知る
- ・「使ったらまずビルドが通らない」「ビルドは通るが実行で破綻する」を見分けられるようになる

結論を先に

本プロジェクトでファームウェアを書くとき、ざっくりこういう分類になる:

分類	代表例
♥ そのまま使ってよい	言語の型システム全般、 <code>comptime</code> 、 <code>packed struct</code> 、 <code>extern struct</code> 、 <code>inline for</code> 、 <code>@import("builtin")</code> 、 <code>std.mem</code> / <code>std.math</code> / <code>std.fmt.bufPrint</code> / <code>std.meta</code> / <code>std.heap.FixedBufferAllocator</code> / <code>std.ArrayList</code> (allocator 渡し)、インラインアセンブリ
🟡 慎重に使う	浮動小数点演算、64-bit 整数、 <code>std.AutoHashMap</code> 、Debug ビルドの panic 経路
🔴 そもそもビルドが通らない	<code>std.debug.print</code> / <code>std.fs.*</code> / <code>std.Thread</code> / <code>std.process</code> / <code>std.os</code> 系の OS API、 <code>std.heap.GeneralPurposeAllocator</code> (実体は <code>DebugAllocator</code> で、 <code>freestanding</code> では取れない要素を要求)
⚠️ 通るが事故を起こしやすい	グローバル可変状態の野放しな使用、「ヒープ」の概念を期待する API、例外/エラー系で巨大なフォーマッタが走る経路

以降、それぞれを「なぜ」「どこまで」「どう代替」の観点で見ていく。

大原則: `freestanding` には OS が無い

本プロジェクトのターゲットは:

```
riscv32-unknown-none-eabi (RV32EC, freestanding, eabi, link_libc=false)
```

`freestanding` は「標準的な OS が無い」を意味する。これに伴い:

- ・ システムコールが無い → `std.fs` / `std.process` / `std.Thread` / `std.net` は **コンパイル自体が通らない** (`@compileError` で明示的にブロックされる)
- ・ 標準入出力 (`stdout` / `stderr`) が無い → `std.debug.print` も同様にコンパイル不可
- ・ ページアロケータが無い → `std.heap.page_allocator` を要求するアロケータは使えない

これらは「freestanding でも動くように Zig 側が用意してくれている逃げ道」を意識して回避することになる:

- ・ 「ヒープが要る」 → `FixedBufferAllocator` (任意のバイト配列を裏に持つ)
- ・ 「ログを吐きたい」 → 自前で UART/SWO/SWD-Printf を実装、または ITM。本プロジェクトには現状その経路は無い (`fmt.bufPrint` でバッファに書いて自前送信するパターン)
- ・ 「並行処理が欲しい」 → SysTick + 割り込み、もしくは協調マルチタスクを自前で組む

言語機能 — そのまま使えるもの

`comptime` / ジェネリクス

これは本プロジェクトの骨子で多用されている。例:

```
// build.zig 内で「known examples」を inline for で走査
inline for (examples) |example| {
    if (std.mem.eql(u8, name, example.name)) return example;
}

// ベクタテーブルは comptime に組み上がって ELF に焼き込まれる
fn makeVectorTable() [39]?*const anyopaque {
    var table = [_]?*const anyopaque{null} ** 39;
    table[2] = &_default_irq_entry;
    // ...
    return table;
}

pub export const vector_table linksection(".vector_table") = makeVectorTable();
```

`comptime` で式が解決できる限り、**ランタイムコストはゼロ**。MCU 向けにこれほど嬉しい言語機能はない。

`packed struct(u8)` と `extern struct`

MMIO レイアウトの記述に欠かせない。本プロジェクトでは `extern struct` をフィールド並びの保証として、一部のフラグ表現には `packed struct(uN)` を使える。

```
const Flags = packed struct(u8) {
  a: bool,
  b: bool,
  rest: u6,
};

const raw: u8 = @bitCast(Flags{ .a = true, .b = true, .rest = 0 });
```

- `extern struct` — フィールド順 = メモリ順、自然アライン。C 互換。
- `packed struct(uN)` — ビット幅を明示し、ビットフィールドのように扱える。`@bitCast` で生のスカラに戻せる。

`optional` / `error union` / `tagged union`

普通に使える。ただし「`!T` のエラーを `format` 越しに文字列化する」と、内部的に複雑なフォーマッタが引きずられる可能性があるので注意 (後述)。

```
const T = union(enum) { a: u8, b: u16 };
const t: T = .{ .b = 100 };
switch (t) {
  .a => |v| ...,
  .b => |v| ...,
}
```

インラインアセンブリ

`asm volatile (...)` が使え、CSR 操作 / 割り込み制御 / 起動コードに必須。第 5・6 章で詳説した通り。

`@import("builtin")` で得られる情報

```
const builtin = @import("builtin");

if (builtin.os.tag == .freestanding) {
```

```

    // ターゲット依存の分岐
}
if (builtin.mode == .Debug) {
    // Debug ビルド時だけのコード
}
const endian = builtin.cpu.arch.endian();

```

`builtin` 経由でターゲット情報を取り、`comptime` 分岐すれば、ビルド時にコード分岐が解決されてランタイムコストがかからない。

`@intCast` / `@truncate` / `@bitCast` / `@as`

整数の幅・型の変換が明示的に書ける。0.16 では「黙って広がる/縮む」キャストは原則禁止になっており、思った通りの命令が出る確認になる。

```

const a: u16 = 0x1234;
const lo: u8 = @truncate(a); // 下位 8-bit
const sg: i16 = @intCast(@as(i32, -3)); // 範囲チェック付き
const flags: u8 = @bitCast(my_packed_struct);

```

言語機能 — 注意して使うもの

浮動小数点

CH32V003 には FPU が無い。`f32` / `f64` の計算は、Zig が `compiler_rt` のソフトウェア浮動小数点ルーチン を呼ぶコードに展開する (第 3 章で触れた `bundle_compiler_rt`)。

- ・ **ビルドは通る** — `compiler_rt` が同梱されているため
- ・ **動くは動く** — ただし 1 演算で数百クロック消費するレベル
- ・ **コードサイズが目に見えて増える** — 浮動小数点を 1 か所でも使うと、`__addsf3` / `__mulsf3` / `__divsf3` などが ELF に乗ってくる

組み込み的には「**整数演算と固定小数点で済むなら、そのほうがずっと幸せ**」。SSD1306 の円描画もプレゼンハム整数で書いてある (第 12 章) のはこの方針。

64-bit 整数 (`u64` / `i64`)

CH32V003 は 32-bit コア。64-bit 演算は **二命令以上 + キャリーチェーン** として展開される。さらに乗除算は `compiler_rt` の `__muldi3` などを使う。

- ・ **ビルドは通る**

- ・ 動く
- ・ 1 操作あたりのコストは 32-bit の 2~10 倍

時刻 (u64 で tick) のように 32-bit が直近で溢れる用途では正当。単に「広い方が安心」で 64-bit を選ぶのは、このターゲットでは贅沢に分類される。

std.AutoHashMap / std.ArrayList

- ・ ビルドは通り、freestanding でも動く (アロケータを引数で受け取るため)
- ・ だが ハッシュ機構やリサイズが意外と重い。2KB の RAM では 数十エントリで枯れる
- ・ 「組み込み向けの薄い HAL」を作る本プロジェクトの趣旨では、出番はかなり限定的

Debug ビルドと panic

Zig は Debug ビルドで:

- ・ 整数オーバーフロー、配列境界、タグ不整合などをランタイムで検査
- ・ 失敗すると panic ハンドラを呼び
- ・ 標準の panic ハンドラは「ファイル名・行番号・スタックトレースを整形してプリント」しようとする

freestanding ターゲットでも、「panic 時のフォーマッタとスタックトレース展開コード」は ELF に乗ってきてしまい、数百 KB ~ MB 単位で .text が膨らむことがある。

検証用の最小コードで実測すると:

構成	ELF サイズ (概算)
-O ReleaseSmall + 自前 panic ハンドラ	1~2 KB
-O Debug + デフォルト panic	~2 MB ELF (=リンク後の .text が無視できないサイズに膨らむ)

本プロジェクトでも、デフォルト最適化は ReleaseSmall にしてある (build.zig の optimize のデフォルト)。Debug でビルドするときは:

- ・ 必ず自前の panic ハンドラ を root モジュールに置く `zig pub fn panic(_: []const u8, _: ?*std.builtin.StackTrace, _ : ?usize) noreturn { // LED 点滅 + wfi など、最低限の合図だけにする while (true) {} }`
- ・ それでも Debug ビルドの方が大きいので、16K FLASH に収まらないことがある

「ReleaseSmall でビルド、デバッグは観測 LED と SWD でやる」が現実的な落とし所。

言語機能 — 使えないもの

`std.debug.print` 系

```
std.debug.print("hi\n", .{});  
// → error: struct 'posix.system__struct_837' has no member named 'getrandom'  
// → コンパイルが通らない
```

`debug.print` は内部で `stderr` に書き込むため、freestanding では参照先で `@compileError` が立つ。ログを取りたいければ:

- ・ `std.fmt.bufPrint(&buf, "x={d}", .{x})` でバッファに整形
- ・ そのバイト列を UART / SWO など自前送信

の 2 段にする。本プロジェクトには UART HAL は同梱されていないので、必要なら自分で追加する立場になる。

`std.fs` / `std.process` / `std.os` 系

```
std.fs.cwd().openFile(...)  
// → error: root source file struct 'fs' has no member named 'cwd'
```

freestanding ターゲットでは `std.fs.cwd` 自体が存在しない。「ファイル」「プロセス」「環境変数」という概念がそもそも無いのだから、これは正しい挙動。

`std.Thread`

```
_ = std.Thread.spawn(.{}, worker, .{}) catch unreachable;  
// → @compileError("Unsupported operating system freestanding");
```

明示的に `@compileError` で止められる。「並行処理が欲しいなら、割り込み or 自前のタスクスケジューラを書け」という設計判断。

`std.heap.GeneralPurposeAllocator` / `DebugAllocator`

`GeneralPurposeAllocator` は 0.16 で `DebugAllocator` に改名されているが、いずれにせよホスト環境のページロケータや OS の確保 API を要求するため、freestanding では使えない。代替は `FixedBufferAllocator`。

```
var pool_bytes: [256]u8 = undefined;

pub fn main() void {
    var fba = std.heap.FixedBufferAllocator.init(&pool_bytes);
    const a = fba allocator();
    // a を ArrayList / AutoHashMap に渡せる
}
```

「ヒープ」を `[N]u8` の固定配列に閉じ込めるイメージ。RAM の使用上限が静的に見える、ある意味 MCU フレンドリな方法。

std.log

`std.log` は内部で `std.debug.print` 系に依存しているため、設定しないと freestanding で死ぬ。どうしても使いたければ `pub const std_options = std.Options{ .logFn = myLog };` で自前の `logFn` を差し込み、その中で `bufPrint` + 自前送信に流す形になる。ただし本プロジェクトには現状その配線は無い。

std で使える「組み込み向きの便利な部品」

freestanding でも問題なく動き、役立つものの一覧:

機能	用途
<code>std.mem.eql</code> / <code>indexOf</code> / <code>sliceTo</code> / <code>copyForwards</code>	バイト/スライス操作
<code>std.mem.byteSwap</code> / <code>nativeToLittle</code> / <code>nativeToBig</code>	エンディアン変換 (通信プロトコル)
<code>std.math.maxInt</code> / <code>minInt</code> / <code>clamp</code> / <code>mod</code> / <code>divCeil</code>	安全な整数演算
<code>std.fmt.bufPrint</code> / <code>bufPrintZ</code>	文字列整形 (UART ログ作る時)
<code>std.fmt.parseInt</code> / <code>parseFloat</code>	受信した ASCII を数値に
<code>std.meta.fields(T)</code> / <code>tagName</code> / <code>stringToEnum</code>	enum / struct の reflection
<code>std.hash.Crc32</code> / <code>Adler32</code> / <code>XxHash32</code>	CRC・チェックサム
<code>std.crypto.hash</code> のいくつか	ハッシュ。ただし重い
<code>std.heap.FixedBufferAllocator</code>	スタティック配列を裏に持つアロケータ

機能	用途
<code>std.ArrayList(T)</code> / <code>BoundedArray(T, N)</code>	可変長配列 / 上限固定可変長配列
<code>std.io.fixedBufferStream</code>	バイト列を Writer/Reader に変換

「Reader/Writer + bufPrint + Crc32」の組合せだけで、シリアルプロトコルの実装はかなり書ける。

設計判断: いつ std を使い、いつ自前で書くか

このリポジトリのスタンスは、「ハードに触る部分は自前 / アルゴリズム的な部分は std」。例えば:

- ・レジスタアクセス、割り込みハンドラ、起動 → 自前 (`src/runtime`, `src/periph`, `src/hal`)
- ・バイト列のスライス操作、整数の最大/最小、ビット演算ヘルパ → std を使ってよい

本プロジェクトのソースは現状 `std` を一切 import していない (HAL の規模が小さいので、std を引かなくても困らないため)。ただし「std 禁止」ではない。アプリ側で `std.fmt.bufPrint` を使った UART ロガーを書くなどは正当な選択肢。

チートシート

「迷ったらこれを見る」用の一覧:

- ✔ Always OK
 - 言語機能全般 (`comptime`, `generics`, `packed/extern struct`, `asm`, `optional`, `error union`)
 - `@import("builtin")`
 - `std.mem.*` (`eql`, `indexOf`, `sliceTo`, `copy`, `swap`)
 - `std.math.*` (整数系)
 - `std.fmt.bufPrint` / `bufPrintZ`
 - `std.meta.*` (`fields`, `tagName`, `stringToEnum`)
 - `std.hash.*` / `std.crypto.hash` 系の整数ハッシュ
 - `std.heap.FixedBufferAllocator`
 - `std.ArrayList(T)` / `BoundedArray(T, N)` (allocator は FBA を渡す)
 - `std.io.fixedBufferStream`
- △ Use carefully
 - f32/f64 演算 (`soft-float`, サイズと速度に影響)
 - u64/i64 演算 (32-bit 2 命令展開 + `libcall`)
 - `std.AutoHashMap` (RAM 不足になりやすい)
 - `std.fmt.allocPrint` (要 allocator、フォーマット重め)
 - Debug ビルドの panic 経路 (`.text` が大幅に膨らむ)
- ✖ Won't compile / Don't use
 - `std.debug.print` / `std.log` (内部で OS 依存)


```
- std.fs.* / std.process.* / std.os.* / std.posix.*  
- std.Thread (@compileError "freestanding")  
- std.heap.GeneralPurposeAllocator / DebugAllocator  
- std.heap.page_allocator  
- std.net.* / std.http.*
```

まとめ

- ・ `freestanding` + `link_libc=false` + RV32EC + RAM 2KB の制約上、「**Zig 言語そのものはほぼフル機能、std はホスト/OS に依存しない部分だけ**」という線引きになる
- ・ 言語機能 (`comptime` / `packed struct` / 任意ビット幅整数 / インラインアセンブリ) は組み込みに非常に強く、これを積極的に使うのが本プロジェクトのスタイル
- ・ `std.mem` / `std.math` / `std.fmt.bufPrint` / `std.meta` / `std.heap.FixedBufferAllocator` / `std.ArrayList` あたりは「**OS が無くても動く部品**」として安心して使える
- ・ `std.debug.print` / `std.fs` / `std.Thread` / `GeneralPurposeAllocator` などは **コンパイル時点で弾かれる**。これは仕様であって不具合ではない
- ・ 浮動小数点と 64-bit は「ビルドは通るが MCU には重い」という典型例。必要性を見極めて使う
- ・ Debug ビルドは panic 関連で `.text` が大きく膨らむ。デフォルトの `ReleaseSmall` で書き、デバッグは LED と SWD で観測するのが現実的

これで、本プロジェクト上で「**これを書いたら通る/通らない/動くが太る**」が見通せる状態になる。各章で見てきた MCU 寄りの薄い HAL と、ここで挙げた「freestanding で使える Zig の道具立て」を組み合わせれば、CH32V003 上で達成できる事の幅はずっと広がる。

第Ⅵ部 永続化

内蔵 FLASH への書き込みで設定/スコアを残す

第14章 データの永続化 — 内蔵 FLASH に書く HAL

Slot(T) で 1 構造体 = 1 ページ

学習目標

- ・ CH32V003 にはなぜ EEPROM が無く、永続化はどう実現するのかを知る
- ・ 本プロジェクトの `src/hal/flash.zig` が提供する API を読んで、アプリから使えるようになる
- ・ 「ページ消去 → ページプログラム」のシーケンスをハード視点で説明できる
- ・ リンカで「ユーザデータ用ページ」を予約する仕組みを理解する
- ・ 寿命 (≒10,000 回) と単位 (64B/ページ) を意識した使い方ができる

CH32V003 の永続化事情

CH32V003 には専用の EEPROM が **無い**。永続化したいなら、ファームウェアが入っているのと同じ 16KB の **内蔵 FLASH** に自分で書き込む。

観点	内容
単位	64 バイト = 1 ページ 。これより小さい粒度では消せない / 書けない
書き換え寿命	約 10,000 サイクル/ページ (TRM の典型値)
1 ページ消去にかかる時間	約 3 ms (busy ループで待つ)
1 ページ書き込みにかかる時間	約 3 ms
書ける方向	「 1 のビットを 0 にする」のみ。逆方向はページ消去で全ビット 1 に戻す
アクセス方法	読みは通常のメモリロード命令で OK。書きは FLASH 周辺レジスタを通して unlock + erase + program
同じ FLASH をコードも使っている	ユーザデータ用に領域を分けないと .text が侵食して使えなくなる

「EEPROM ライク」な使い方をしたければ、FLASH の最後のページなどを「ユーザデータ用」として確保し、そこだけ書き換える形にする。本プロジェクトはまさにそれをやっている。

設計判断

本プロジェクトの永続化 HAL は次の方針で設計してある:

1. **物理アドレス指定の低レベル API** (`erasePage` / `programPage` / `writePage`) を素直に公開する
2. その上に **薄い KV 風ラッパ Slot(T)** を提供して、アプリは「1 構造体 = 1 ページ」のシンプルなモデルだけで永続化できるようにする
3. **リンカスクリプトでユーザデータページ (64B) を予約**して、アプリのコードが誤って侵食したらリンクエラーで気付けるようにする

これにより、「ハードを直接叩きたい」用途と「ミニゲームのスコアを保存したい」用途が、同じモジュールから別レベルでアクセスできる。

リンカスクリプトでの予約

`src/runtime/linker.ld` を 2 つの FLASH 領域に分けてある:

```
MEMORY
{
    FLASH      (rx)  : ORIGIN = 0x08000000, LENGTH = 16K - 64
    USER_DATA  (r)   : ORIGIN = 0x08003FC0, LENGTH = 64
    RAM        (rwx) : ORIGIN = 0x20000000, LENGTH = 2K
}
```

- ・ **FLASH** : コード + `.rodata` + `.data` の LMA 用、16KB - 64B
- ・ **USER_DATA** : 永続データ用、64B = 1 ページ

`.text` / `.rodata` が **FLASH** を超えた場合、リンカが「**USER_DATA** を覆ってしまう」とは見なさず単純に **region 'FLASH' overflowed** で落ちる。つまり「アプリが太ってきても、ユーザデータ領域が静かに侵食される事故は起きない」という安全側のレイアウトになっている。

リンカは **USER_DATA** の物理アドレスをシンボルとして HAL に渡す:

```
PROVIDE(_user_data_start = ORIGIN(USER_DATA));
PROVIDE(_user_data_size  = LENGTH(USER_DATA));
```

`src/hal/flash.zig` の `userDataAddr()` がこれを参照している:

```
extern var _user_data_start: u8;

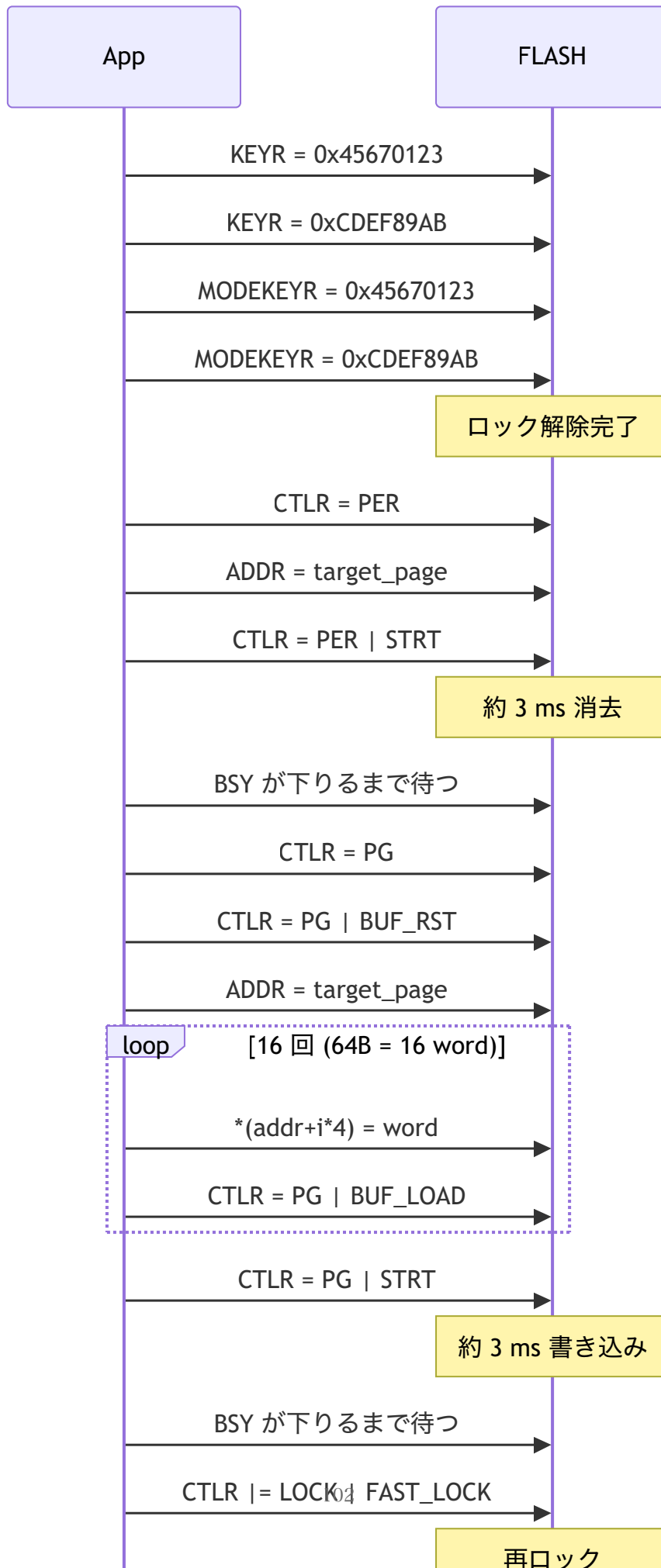
pub fn userDataAddr() usize {
```

```
return @intFromPtr(&_user_data_start);  
}
```

第 5 章の `extern var _sbss: u32` と同じイディオムで、「リンカが用意したシンボルのアドレスだけが欲しい」ケースだ。

ハード視点の手順 — Fast Program モード

CH32V003 の FLASH は「Fast Page Erase」「Fast Page Program」という 64B 単位の操作を持つ。操作シーケンスは TRM (Technical Reference Manual) に従って:



要点:

- ・ **2 段ロック**: KEYR (主ロック) と MODEKEYR (fast モード) の両方を 0x45670123 → 0xCDEF89AB の順で解除する
- ・ **消去とプログラムは別フェーズ**: erase は PER ビット、program は PG ビット (TRM では FTPG/FTER と呼ばれる)
- ・ **ページバッファ**: program 時はまず CPU が普通のメモリ書き込み命令で 4 バイトずつ書き、直後に BUF_LOAD を立てて「FLASH 側の内部バッファに 1 ワード積んだよ」と教える。64B 分積んだあと STRT で実 FLASH に焼き込む
- ・ **STATR の BSY ビット**: 操作中は立ちっぱなしになるので、完了まで spin する
- ・ **書き込み後は必ず再ロック**: 何かの暴走で意図しないアドレスを書き換ええないため

HAL の API

src/hal/flash.zig を @import("ch32fun").flash 経由で使う。

低レベル API

```
pub const page_size: usize = 64;
pub const Page = [page_size]u8;

pub fn unlock() Error!void;
pub fn lock() void;
pub fn erasePage(addr: usize) Error!void;
pub fn programPage(addr: usize, src: *const Page) Error!void;
pub fn writePage(addr: usize, src: *const Page) Error!void;
pub fn readPage(addr: usize, dst: *Page) Error!void;
pub fn userDataAddr() usize;
```

writePage は erase + program + verify を 1 関数にしたコンビニエンス。通常はこれで足りる。

エラー型:

```
pub const Error = error{
    UnlockFailed,    // KEYR シーケンスが効かなかった
    BusyTimeout,     // BSY が下りない
    WriteProtected,  // STATR.WRPRTERR が立った
    Misaligned,      // addr % 64 != 0
    OutOfRange,      // addr が FLASH レンジ外
    Verify,          // 書いた後の読み返しが不一致
};
```

高レベル API — Slot(T)

「1 ページに 1 つの構造体を載せる」 簡易 KV 風ラップ:

```
pub fn Slot(comptime T: type) type;
```

レイアウト:

-----+-----+-----+-----+-----
2B magic 2B ver 4B reserved sizeof(T) バイトの値
0xCAFE u16
-----+-----+-----+-----+-----

- ・ magic でデータの有無を判定 (未書き込みの FLASH は `0xFF` で埋まっている)
- ・ version で「保存形式が変わったらマイグレーション」ができる

呼び出し:

```
const Counter = extern struct {
    boots: u32,
    last_score: u32,
};

const slot = fun.flash.Slot(Counter).default(1); // version = 1

// 読み出し (無ければ null)
const maybe = slot.load();

// 保存
try slot.save(.{ .boots = 7, .last_score = 1024 });

// 「無ければ初期値で保存し、必ず値を返す」
const c = try slot.loadOrInit(.{ .boots = 0, .last_score = 0 });
```

Slot は内部で `unlock()` → `writePage()` → `lock()` を行う。1 回の `save()` で 1 サイクル消費される。

制約

- ・ T のサイズは 56 バイト (= 64 - 8) 以下。 `@compileError` でビルド時に弾く
- ・ T は `extern struct` などの **メモリエイアウトが固定された型** にすること (Zig の通常 `struct` だとフィールド並び替えが起きうるので、別のビルドで読めなくなる可能性あり)

サンプル: `examples/persistent_counter/`

電源 ON のたびに **起動回数を FLASH に保存し、その回数だけ LED を点滅させる** デモ。

```
const fun = @import("ch32fun");

const Counter = extern struct {
    boots: u32,
    last_score: u32,
};

pub fn main() noreturn {
    fun.system.init(.{});
    fun.gpio.enableAllClocks();

    const led = fun.gpio.pin(.D, 0);
    led.configure(.output_pp_10mhz);

    const slot = fun.flash.Slot(Counter).default(1);
    var c = slot.loadOrInit(.{ .boots = 0, .last_score = 0 }) catch Counter{
        .boots = 0, .last_score = 0,
    };
    c.boots += 1;
    slot.save(c) catch { /* LED 高速点滅でエラー通知 */ };

    // boots 回ぶん LED を点滅 → ゆっくり点滅ループ
    var n: u32 = 0;
    while (n < c.boots) : (n += 1) {
        led.write(true); fun.time.delayMs(200);
        led.write(false); fun.time.delayMs(200);
    }
    fun.time.delayMs(1500);
    while (true) {
        led.toggle();
        fun.time.delayMs(1000);
    }
}
```

ビルド & 書き込み:

```
zig build -Dexample=persistent_counter flash
```

電源をいったん落として入れ直すたびに、LED の点滅回数が **1, 2, 3 ...** と増えるはず。リセットボタンでも同様に増える。

ミニゲームのスコア保存に使うときの設計

「LED を点滅させる」程度の話で、たとえば SSD1306 (oled サンプル) ベースで作ったミニゲームのハイスコアに置き換えるなら、 こうなる:

```
const HighScore = extern struct {
    score: u32,
    last_played_boots: u32,
};

const hs_slot = fun.flash.Slot(HighScore).default(1);

pub fn main() noreturn {
    // 起動時に読み出し
    var hs = hs_slot.loadOrInit({ .score = 0, .last_played_boots = 0 }) catch ...;

    while (true) {
        const result = playOneRound(hs); // ゲーム本体
        if (result.score > hs.score) {
            hs.score = result.score;
            hs_slot.save(hs) catch {
                showError();
            };
        }
    }
}
```

設計上の注意

1. **書き込み頻度を絞る** — 寿命 10,000 回は意外と少ない。1 ゲーム終了ごとに 1 回保存くらいが現実的。「フレームごとに保存」は数日でフラッシュを潰す可能性がある
2. **ハイスコアを更新したときだけ書く** — 上の例のように差分があるときだけ save
3. **読み書き時にユーザにフィードバック** — 書き込み中は 3~6ms 動作が止まる。OLED 描画中なら一瞬カクつくので、「セーブ中…」アイコンを出すと UX が良い
4. **電源断耐性** — `writePage()` の途中で電源が切れると、そのページは中途半端な状態になる。magic フィールドが意図せず壊れていれば `load()` は null を返すので、アプリは「無ければ初期値」のロジックさえ書いておけば回復はする。中途半端な「半分書かれた T」が返ることはない (verify が走り `Error.Verify` が返る)
5. **複数を保存するなら 1 つの大きな struct** — `Slot(T)` は 1 ページ = 1 値なので、「スコア」と「設定」を別ページで保存したいなら、USER_DATA を 64B 単位で増やす (linker.ld で領域を増やす) か、1 つの struct にまとめる

寿命を伸ばしたい場合 — Wear Leveling

10,000 サイクル制約はゲーム用途なら大体足りるが、「秒ごとに保存したい」ような用途では困る。そういう時は **ページ内 wear leveling** を実装するのが定石:

USER_DATA を 64B のリングバッファとして扱い、
8B ずつ区切って 8 スロット (record 0..7) とする。

各レコード:
+---- 1B seq番号 (毎回インクリメント) ----+---- 7B データ ----+

書き込み時:
- 最新の seq を持つレコードの「次のスロット」に書く
- 全スロットが 1 ページに収まるので、ページ消去は 8 回に 1 回で済む

→ 実効寿命が 8 倍に

これは本プロジェクトには未実装。上の構造に従って Slot の拡張版を書くと **80,000 サイクル相当** に伸ばせる。必要に応じてアプリ側で書く立ち位置になる。

制約と落とし穴 まとめ

やってはいけないこと	結果
アライメント無視で <code>programPage(0x0800_3FC1, ...)</code>	<code>Error.Misaligned</code>
FLASH レンジ外 (<code>0x2000_0000</code> など) を指定	<code>Error.OutOfRange</code>
Unlock を忘れて <code>programPage</code>	<code>Error.UnlockFailed</code> (or <code>WriteProtected</code>)
通常 <code>struct</code> (非 extern) を <code>Slot(T)</code> に使う	コンパイルは通るがレイアウト保証無し
1 ループで毎回 save	フラッシュ寿命が 100 時間で尽きる
<code>T</code> のサイズが 57 バイト以上	<code>@compileError</code> でビルド失敗

推奨

`T` は `extern struct` で書き、サイズを 4 アラインに揃える

バージョン番号を付けて、レイアウト変更時に上げる (古いデータは無視される)

推奨

「書き込みに失敗したら LED 高速点滅で通知」のような最低限のフィードバックを付ける

開発中はフラッシュサイクルを浪費しないよう、「保存はリリース時のみ」など分岐させても良い

まとめ

- ・ CH32V003 には EEPROM が無く、永続化は **内蔵 FLASH の末尾 1 ページ** に書く形で実現する
- ・ HAL は **ハード手順を忠実に踏む低レベル関数** と、**slot(T)** という **KV 風ラッパ** の 2 層構成
- ・ リンカが **USER_DATA** 領域を明示的に切り出しているため、アプリ側のコードが侵食する事故をビルド時に検出できる
- ・ **examples/persistent_counter** がエンドツーエンドの動作確認用サンプル
- ・ 10,000 サイクル制約と 64B 単位 / 3ms ストールという特性は常に意識する。寿命を伸ばしたければ wear leveling を上位で実装する

これで「ミニゲームのスコアを電源 OFF を跨いで覚えておく」程度のことは、アプリ側数行で書けるようになった。

第Ⅶ部 便利な周辺機能

ファームウェアでよく使う 5 機能を HAL に揃える

第15章 周辺機能の HAL — UART / log / PWM / Tone / ADC / EXTI

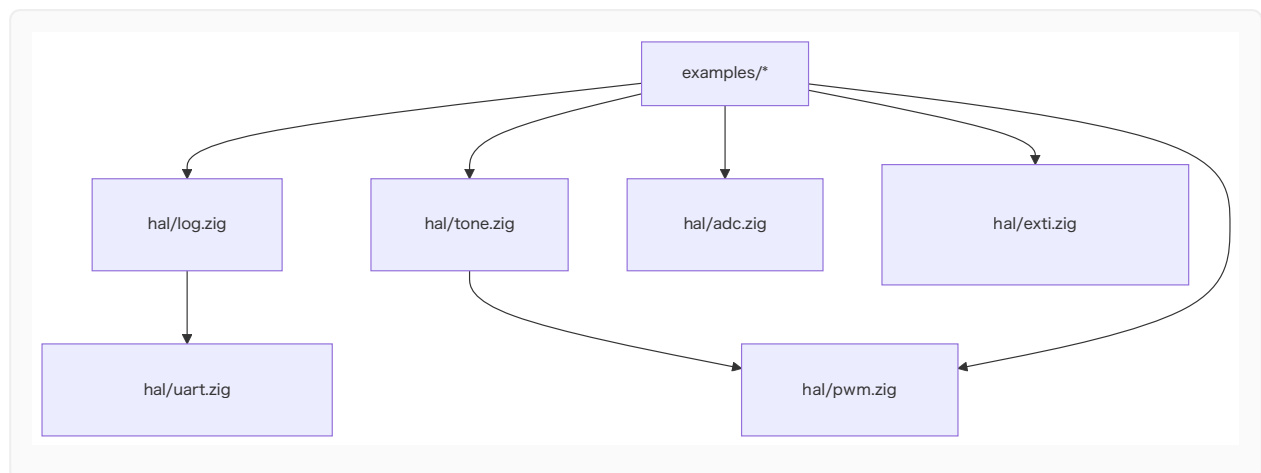
ログ・LED調光・音・アナログ・割り込み入力

学習目標

- ・「ファームウェアでよくある 5 機能」を本プロジェクトのスタイルで使えるようになる
- ・ **UART** で **printf** 風ログ を出す
- ・ **PWM** で **LED** を呼吸させる
- ・ **PWM** の上に **トーン API** を被せて圧電ブザーで音を出す
- ・ **ADC** でアナログ電圧を読む
- ・ **EXTI** でボタン押下を割り込み駆動にする
- ・ これらの HAL がどう「薄く」設計されているかを掴む
- ・ 各 example の動作と必要な配線を把握する

全体マップ

これまでの章で扱った GPIO / SysTick / I2C / SSD1306 / FLASH に加え、第V部の HAL ファミリーには次の 6 モジュールが新たに加わった:



設計方針はこれまでと同じ — **MCU の語彙にできるだけ薄く、アプリ側ですぐ使える形だけ整える。**

UART (`fun.uart`)

仕様

- ・ ペリフェラル: **USART1**
- ・ TX: **PD5** (リマップ無し)
- ・ 受信: 未サポート (CTRL1.RE は立てない)
- ・ ボーレート: コア (HCLK) を整数分周。48MHz / **BRR**
- ・ フォーマット: 8N1 固定

API

```
pub fn init(baud: u32) void;  
pub fn writeByte(b: u8) void;  
pub fn writeAll(bytes: []const u8) void;  
pub fn flush() void;
```

配線

```
PC USB-Serial:  RXD <----- PD5 (CH32V003 TX)  
                  GND <----- GND
```

USB-Serial 変換 (CP2102 等) の RXD を PD5 に、GND を GND に。端末を **115200bps 8N1** で開いて待ち受ける。

log (`fun.log`)

仕様

UART の上に薄く被せた **printf 風ラッパ**。内部に **[96]u8** の静的バッファを 1 本持ち、**std.fmt.bufPrint** で整形して UART に流す。

API

```
pub fn init(baud: u32) void;           // 内部で uart.init(baud) を呼ぶ  
pub fn info(comptime fmt: []const u8, args: anytype) void;
```

```
pub fn warn(comptime fmt: []const u8, args: anytype) void;
pub fn err(comptime fmt: []const u8, args: anytype) void;
pub fn raw(bytes: []const u8) void;
```

注意点

- ・ **割り込みコンテキストから呼ぶのは非推奨**。UART がブロックしている間、割り込みハンドラが寝てしまう
- ・ バッファあふれは無視 (96B でフォーマット結果が打ち切られる)。長文を吐きたければ複数回に分ける
- ・ `\r\n` は自動付与

第13章で「使えない」と言ったログ系との関係

第13章で `std.debug.print` は freestanding でコンパイル不可だと言った。本モジュールはまさに「**OS が無くてもログを取りたい**」を埋める層。中身は `std.fmt.bufPrint` (これは freestanding でも使える) + 自前 UART 送信。

サンプル: `examples/uart_hello`

```
const fun = @import("ch32fun");

pub fn main() noreturn {
    fun.system.init(.{});
    fun.log.init(115200);

    var n: u32 = 0;
    while (true) : (n += 1) {
        fun.log.info("hello from CH32V003! tick={d}", .{n});
        fun.time.delayMs(1000);
    }
}
```

書き込み:

```
zig build -Dexample=uart_hello flash
```

端末側で 1 秒おきに `[I] hello from CH32V003! tick=N` が増えていく。

PWM (fun.pwm)

仕様

CH32V003 の 2 つの汎用タイマを PWM 出力に使えるよう包んだ。

ペリフェラル	クラス	デフォルトチャネルピン
TIM1	Advanced (MOE 必要)	CH1=PD2, CH2=PA1, CH3=PC3, CH4=PC4
TIM2	General	CH1=PD4, CH2=PD3, CH3=PC0, CH4=PD7

両方とも **PWM mode 1 (アクティブ Hi)** で固定。デューティ比は `0 .. period` の整数。

API

```
pub const Channel = enum(u2) { ch1, ch2, ch3, ch4 };
pub const Config = struct {
    period: u16 = 1000,
    prescaler: u16 = 0,
};

pub const tim1 = struct {
    pub fn init(cfg: Config) void;
    pub fn enableChannel(ch: Channel) void;
    pub fn setDuty(ch: Channel, value: u16) void;
    pub fn setPeriod(period: u16) void;
    pub fn stop() void;
};
pub const tim2 = struct { /* 同じ shape */ };
```

実効カウントクロック = `HCLK / (prescaler + 1)`、PWM 周波数 = カウントクロック / `period`。

サンプル: `examples/led_fade`

```
fun.gpio.pin(.D, 2).configure(.output_af_pp_10mhz);
fun.pwm.tim1.init({ .prescaler = 47, .period = 1000 }); // 48MHz/48/1000 = 1kHz
fun.pwm.tim1.enableChannel(.ch1);
// duty を 0 → 1000 → 0 と往復させて呼吸を作る
```

LED アノードを PD2、カソードを GND (抵抗経由) で繋ぐと、LED が明るくなったり暗くなったりする。

Tone (`fun.tone`)

仕様

`pwm.tim2` の上に、「周波数 (Hz) と長さ (ms) で 1 音鳴らす」API を被せたもの。デューティは 50% 固定。内部プリスケアラは 256 分周で、おおよそ 30Hz～数 kHz の範囲をカバーする。

API

```
pub fn init(channel: pwm.Channel) void;
pub fn play(freq_hz: u32, duration_ms: u32) void; // ブロッキング
pub fn stop() void;
```

配線

```
ブザー (+) ——— PD4 (TIM2_CH1)
ブザー (-) ——— GND
```

圧電ブザー (パッシブタイプ) 推奨。アクティブブザーは内部発振器を持つので周波数制御が効かない。

サンプル: `examples/tone_song`

ドレミファソラシド (C4 → C5) を 200ms ずつ鳴らしてループ。 `Note = struct { freq, ms }` のテーブル駆動でアプリ側を簡潔に保てる。

ADC (`fun.adc`)

仕様

- ・ ADC1、10-bit、単一チャネル変換、ソフトウェアトリガ
- ・ チャネルマップ (CH32V003):
- ・ ch0=PA2, ch1=PA1, ch2=PC4, ch3=PD2,
- ・ ch4=PD3, ch5=PD5, ch6=PD6, ch7=PD4
- ・ サンプル時間は最大 (約 241 cycles) に統一

API

```
pub const max_value: u16 = 1023;
pub fn init() void;
pub fn readSingle(channel: u8) u16;           // 0..1023
pub fn readAveraged(channel: u8, samples: u8) u16;
```

注意点

- ・ 使う前に 対応 GPIO ピンを `.input_analog` に設定 すること
- ・ Vref は Vdd (3.3V) なので、mV 換算は `raw * 3300 / 1023`
- ・ ノイズが気になるなら `readAveraged` で 4~16 回平均を取るのが現実解

サンプル: `examples/adc_meter`

```
fun.gpio.pin(.D, 2).configure(.input_analog); // ch3
fun.adc.init();
while (true) {
    const raw = fun.adc.readAveraged(3, 8);
    const mv: u32 = (@as(u32, raw) * 3300) / 1023;
    fun.log.info("ADC ch3 raw={d} ~{d}mV", .{ raw, mv });
    fun.time.delayMs(200);
}
```

可変抵抗を `3.3V - POT - GND` で繋ぎ、中点を PD2 に。つまみを回すと UART に 0~3300 mV が流れる。

EXTI (`fun.exti`)

仕様

GPIO ピンのエッジ検出で割り込みを発生させる。

- ・ ライン番号 = ピン番号 (0..7)。ライン 8..15 は別 IRQ なので今回は未対応
- ・ AFIO.EXTICR でポート (A/C/D) を選ぶ
- ・ トリガ: `rising` / `falling` / `both`
- ・ ハンドラはライン別に登録 (内部に `[8]?Handler`)
- ・ CH32V003 では ライン 0..7 は全部 `EXTI7_0_IRQn = 20` に集約されてくるので、1 つのエントリで分配

API

```
pub const Trigger = enum { rising, falling, both };
pub const Handler = *const fn (line: u8) callconv(.c) void;
pub const Config = struct {
    port: gpio.Port,
    line: u4,
    trigger: Trigger,
    handler: Handler,
};

pub fn config(cfg: Config) void;
pub fn enable(line: u4) void; // INTENR + PFIC のラインを立てる
pub fn disable(line: u4) void;
```

起動側との結合

src/runtime/startup.zig の vector_table[16 + 20] が `_exti7_0_irq_entry` を指す。その先で `exti.handleInterrupt()` が呼ばれ、立っているラインだけハンドラを叩く。第 6 章の SysTick ハンドラと同じ「全レジスタ退避 + mret」パターン。

サンプル: examples/exti_button

```
fn onButtonPress(line: u8) callconv(.c) void {
    _ = line;
    fun.gpio.pin(.D, 0).toggle();
}

pub fn main() noreturn {
    fun.system.init(.{});
    fun.gpio.enableAllClocks();
    fun.gpio.pin(.D, 0).configure(.output_pp_10mhz);
    fun.input.initButtonPd1Pullup();

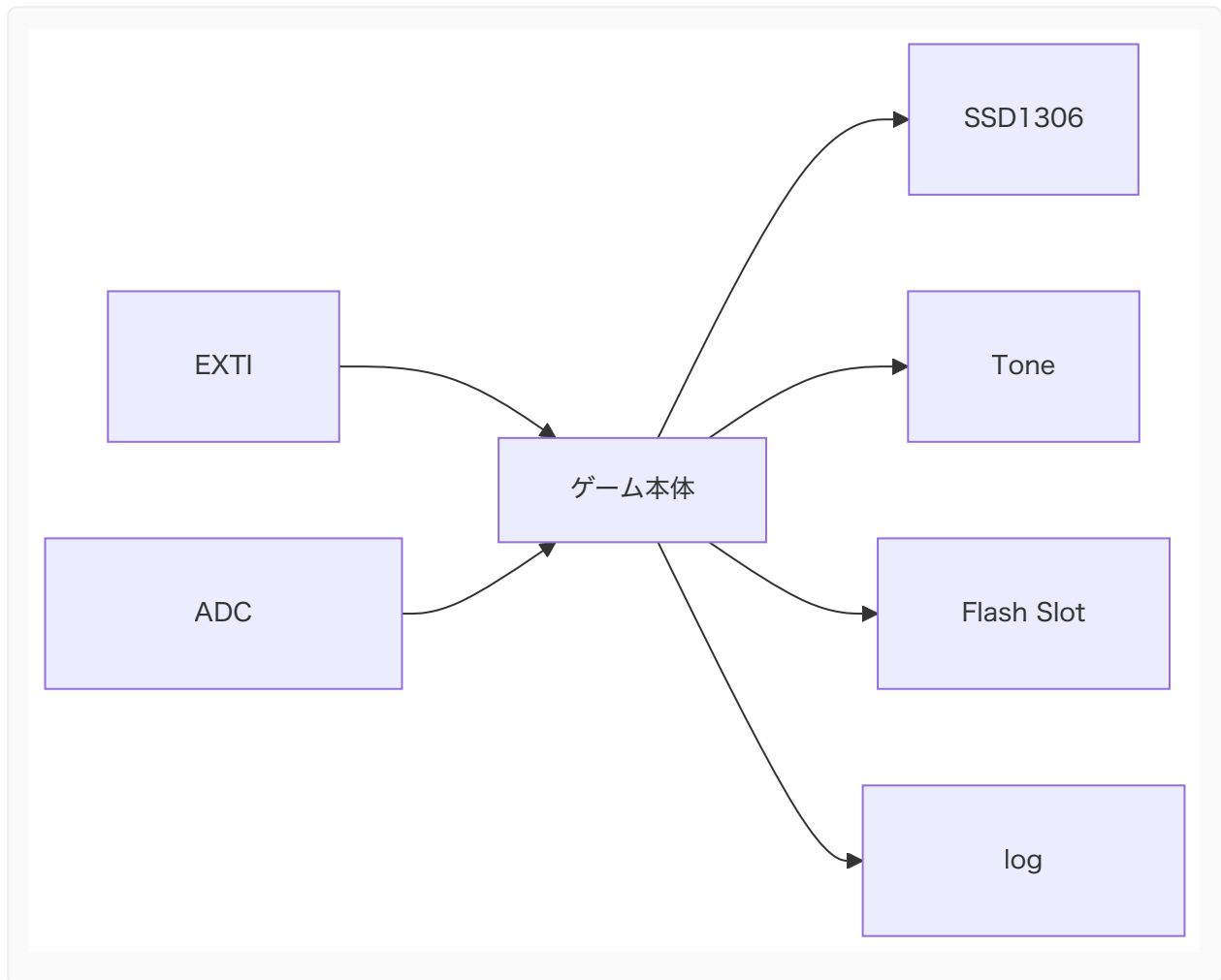
    fun.exti.config({
        .port = .D, .line = 1, .trigger = .falling, .handler = onButtonPress,
    });
    fun.exti.enable(1);
    fun.system.enableInterrupts();

    while (true) fun.system.wfi();
}
```

メインループは `wfi` で寝るだけ。ボタン押下 (= PD1 立ち下がり) があるたびに割り込みで起きて LED がトグルする。ポーリング版 (`examples/gpio_input`) と動作は同じだが、**CPU の稼働時間がほぼゼロ** になるので電池駆動に向く。

モジュール間の組み合わせ

これらは単独で使えるが、組み合わせると `examples/` 1 つでミニゲーム規模のことが作れる:



`examples/oled` + `examples/persistent_counter` + 本章の HAL を組み合わせれば、「POT で照準を動かして、ボタンで撃って、当たったらブザーが鳴り、ハイスコアが FLASH に残り、UART でデバッグ情報が見える」という構成がそのまま書ける。

制約と注意点 まとめ

モジュール	主な制約
UART	TX のみ、PD5 固定、ブロッキング (busy wait)
log	バッファ 96B 切り詰め、割り込みからは呼ばない

モジュール	主な制約
PWM	mode は PWM1 (アクティブ Hi) 固定、4 ch のみ、出力反転は未対応
Tone	50% デューティ固定、1 音ずつブロッキング
ADC	サンプル時間最大 241cyc 固定、連続変換/シーケンス変換は未対応
EXTI	ライン 0..7 のみ、1 ライン 1 ハンドラ、デバウンスは無し

それぞれ「**もう一段豪華に**」したい場合の拡張ポイントは明確。必要が出たタイミングで HAL を継ぎ足していけば良い。

まとめ

- ・第V部までで「Zig で安全に書ける範囲」を整理したが、本章では実機の「便利な周辺」を HAL に落とし込み、アプリから 1～数行で使えるようにした
- ・UART + log があると、デバッグの一手目が「**LED 点滅**」から「**log.info** で文字列を吐く」に進化する
- ・PWM + Tone + ADC + EXTI が揃うと、ミニゲームのインタラクショナー式が CH32V003 上で完結する
- ・いずれも実装は 50～120 行程度の薄い HAL。不足分は同じスタイルで継ぎ足せる

第VIII部 Zig 言語機能を活かす

comptime / tagged union / packed struct で書くファーム

第16章 Zig 言語機能を活かしたファームウェアパターン

4 つのサンプルで Zig らしさを実装で示す

学習目標

- ・ `comptime` がコードサイズ・RAM 使用量にどう効くかを 4 つのサンプルで実感する
- ・ `tagged union` + 網羅 `switch` でステートマシンを書くと、何が嬉しいのか分かる
- ・ `packed struct(uN)` + `@bitCast` で「ビット幅を意識した型」を実用化する
- ・ 13 章のチェックリスト的な解説を、動くコードで補強する

4 つのサンプル

#	サンプル	主な Zig 機能	bin サイズ
A	<code>compile_time_morse</code>	<code>comptime</code> 文字列展開、 <code>tagged union</code>	0.5 KB
B	<code>state_machine_game</code>	<code>tagged union</code> + 網羅 <code>switch</code> 、飽和加算 <code>+ \ =</code>	6.0 KB (OLED 込み)
C	<code>packed_settings</code>	<code>packed struct(u32)</code> 、 <code>@bitCast</code> 、Flash Slot	1.7 KB
D	<code>comptime_lookup</code>	<code>comptime</code> で sin テーブル生成、 <code>.rodata</code> 配置	0.8 KB

それぞれは独立して動く小品。「Zig だから書けた」部分を意識して読むと教材価値が高い。

A: `compile_time_morse` — 文字列 → モールス符号をコンパイル時に展開

何が嬉しいか

- ・ 「文字 → モールス変換ロジック」は **コンパイル時に消える**
- ・ 実機で走るのは「事前に並んだ `Element` 配列を順に出すループ」だけ
- ・ 変換コストは 0 命令 / 0 バイトの RAM。一方、普通の C で書こうとすると、ランタイムの `switch` か LUT 引きが残ってしまう

キーになる部分

```
const Element = union(enum) { dit, dah, letter_gap, word_gap };

fn morseEncode(comptime text: []const u8) []const Element {
    comptime {
        var elems: []const Element = &.{};
        for (text, 0..) |ch, idx| {
            const code = morseFor(ch) orelse continue;
            for (code) |c| {
                elems = elems ++ &[_]Element{switch (c) {
                    '.' => .dit,
                    '-' => .dah,
                    else => unreachable,
                }};
            }
            if (idx + 1 < text.len and text[idx + 1] != ' ') {
                elems = elems ++ &[_]Element{.letter_gap};
            }
        }
        return elems;
    }
}

const message = morseEncode("HELLO ZIG "); // ← ROM に焼かれる配列
```

- ・ `morseEncode` は `comptime` ブロックで全部評価される。 `text` は `comptime []const u8` (= 文字列リテラル) として渡ってくる。
- ・ 配列の連結 `++` は `comptime` に許される操作。結果は固定長の `[]const Element` リテラルとして `message` に紐付く。
- ・ 実機上の `for (message) |e| emit(led, e);` は、ROM 上の配列要素を `switch` で分岐するだけの単純なループに翻訳される。

改変アイデア

- ・ `Element` に「周波数」を持たせて、LED ではなくブザーで送信する版に
 - ・ `text` を別のメッセージに差し替えて、自分の名前を SOS 風にする
-

B: `state_machine_game` — tagged union でステートを表現

何が嬉しいか

C で書くと「`int state;` + `enum { MENU, PLAYING, GAME_OVER }`」 + 「ステートごとに別の構造体」を別々のグローバルとして並べることになりがち。Zig の `union(enum)` はそれらを 1 つの「ステート」型として束ね、

```
const State = union(enum) {
    menu: Menu,
    playing: Playing,
    game_over: GameOver,
};
```

と書ける。そして `switch (state.*)` を書くと、**すべてのバリエーションを処理しないとコンパイルエラー** になる。新しいステート (`paused` など) を追加した瞬間、「`render / step` の両方を直してくれ」と Zig が怒ってくれる。これは大規模化したファームのバグ予防に効く。

キーになる部分

```
fn step(state: *State, btn_pressed: bool) ?State {
    switch (state.*) {
        .menu => |*m| { /* ... */ },
        .playing => |*p| {
            if (btn_pressed) {
                if (in_sweet) {
                    p.score += 1; // 飽和加算: u16 オーバーフローでパニックしない
                    // ...
                }
            }
            return null;
        },
        .game_over => |*g| { /* ... */ },
    }
}
```

- ・ `.playing => |*p|` で **そのバリエーションの中身へのポインタ** が取れる。これでスコアやミス数をその場で書き換えられる
- ・ `+=` (飽和加算) は Zig の組み込み演算子。 `u16` の上限 65535 を超えても勝手にラップ/パニックしない。ゲームスコアのような「定義域を絶対超えてほしくない」値に向く
- ・ ステート遷移は `return State{ .game_over = .{ ... } };` の形で **値を返す** スタイル。グローバルを書き換えずに済むので、振る舞いを読みやすい

サイズ

OLED ドライバ込みで 6KB。これは `oled` サンプルとほぼ同じで、ステートマシン部分そのものの追加コストはごくわずか。

改変アイデア

- ・ `Paused` ステートを追加してみる → コンパイラに何箇所怒られるかを観察するとよい教材
- ・ スイートスポットの幅を `Playing` の中に持たせ、段階的に難しくする

C: `packed_settings` — `packed struct(u32)` を Flash に保存

何が嬉しいか

設定一式 (動作モード / 明るさ / ブザーオン・オフ / ボリューム / 保存回数 / レビジョン) を **u32 1 ワード** にビット幅指定で詰める:

```
const Settings = packed struct(u32) {
    mode: LedMode,      // 3 bit
    brightness: u4,     // 4 bit
    buzzer: bool,       // 1 bit
    volume: u4,         // 4 bit
    save_count: u12,    // 12 bit
    revision: u8,       // 8 bit
};

comptime {
    if (@sizeOf(Settings) != 4) @compileError("Settings must fit in u32");
}
```

- ・ `packed struct(u32)` の `(u32)` は「全フィールド合計でちょうど 32-bit になることをコンパイラに検証させる」。ビット幅の積み上げを 1 ビットでも間違えるとコンパイルエラー
- ・ `comptime { ... @compileError }` で「サイズ ≠ 4 ならビルド失敗」とダメ押しの保険
- ・ `@bitCast(u32, settings)` でスカラに、逆方向もキャスト 1 回 ↔ struct

Flash 永続化との相性が良く、第14章の `Slot(T)` に `T = Settings` を渡すだけで設定保存が完了する。

キーになる部分

```
const initial: Settings = .{
    .mode = .slow_blink,
```

```

        .brightness = 8,
        .buzzer = true,
        .volume = 4,
        .save_count = 0,
        .revision = 1,
    };

    fn loadSettings() Settings {
        return settingsSlot().load() orelse initial;
    }

    // ボタン押下が落ち着いたタイミングだけ Flash に書く (寿命対策)
    if (dirty and debounce == 0) {
        saveSettings(s);
        dirty = false;
    }

```

`packed struct` で必要なビット幅だけを宣言しているので、構造的に不正な値 (`mode = 99` のような) が **入りようがない**。これは C のビットフィールドにありがちな「コンパイラ依存のレイアウト」を排除した、Zig の良いところ。

改変アイデア

- ・ フィールドを増やしてもサイズが 32-bit に収まり続けるか、`@compileError` で検査
- ・ 8-bit MCU 向けに `packed struct(u16)` 版にして比較してみる

D: `comptime_lookup` — `sin` テーブルをコンパイル時に作る

何が嬉しいか

LED の呼吸 (フェード) には `sin` 関数が向いているが、CH32V003 には FPU が無く、`sin` 計算はソフトウェア float で重い。Zig の `comptime` なら、**テーブルそのものをビルド時に生成** して `.rodata` に焼ける。

```

fn buildBreathTable() [table_len]u16 {
    @setEvalBranchQuota(200_000);
    var out: [table_len]u16 = undefined;
    var i: usize = 0;
    while (i < table_len) : (i += 1) {
        const phase: f32 = @as(f32, @floatFromInt(i)) / @as(f32, table_len);
        const s = @sin(phase * std.math.pi);
        const lin = s * s;
        const corrected = std.math.pow(f32, lin, gamma); // ガンマ補正
        const v = corrected * @as(f32, @floatFromInt(pwm_period));
        out[i] = @intFromFloat(@max(0.0, @min(@as(f32, @floatFromInt(pwm_period)), v))
    );
}

```

```

    }
    return out;
}

const breath_table: [table_len]u16 = buildBreathTable();

```

ポイント:

- ・ `buildBreathTable()` の中で `@sin` / `std.math.pow` を呼んでいるが、これらはすべて **ホスト (PC) 側の Zig コンパイラ** が実行する。MCU 側のコードには浮動小数点の `calc` は 1 命令も出ない
- ・ `@setEvalBranchQuota(200_000)` は「comptime での命令実行回数の上限」を上げる指示。
`pow` 内部のループが多くてデフォルト 20,000 では足りないため
- ・ 結果は `[256]u16 = 512` バイトの ROM 定数。ループ側は `breath_table[idx]` でインデックス引きするだけ

サイズ

最終 bin は **0.8KB**。512 バイトのテーブルが入っているとは思えないコンパクトさ。これは:

- ・ ReleaseSmall のリンクが似たビットパターンを共通化している
- ・ ガンマ補正済みのテーブルは「同じ値が連続する区間」が多く、デッドコードを含めても `.text` が短い

ことの合わせ技。

改変アイデア

- ・ ガンマを 1.0 / 2.2 / 3.0 で切り替えて、視覚的な呼吸感の違いを観察
- ・ `sin` の代わりに「三角波」や「ノコギリ波」でテーブルを作って比較

4 つを通して見えてくる「Zig らしさ」

観点	言語機能	例
ランタイムを comptime に追い出す	<code>comptime</code> 関数 / ブロック	A の符号化、D の <code>sin</code> テーブル
不正な状態を表現できないようにする	<code>tagged union</code> + 網羅 <code>switch</code> 、 <code>packed struct(uN)</code>	B のステート、C の設定
数値の安全な操作		B のスコア、C の各フィールド

観点	言語機能	例
	<code>+\\ =</code> (飽和)、 <code>@truncate</code> / <code>@intCast</code> 、ビット幅指定整数	
OS なし環境の特性を活かす	<code>freestanding</code> でも <code>comptime</code> + <code>std.math</code> は使える	A / D は <code>std</code> を <code>import</code> しているが OS 機能には触らない

第13章では「使える / 使えない」を表で整理した。本章はそれを **動く 4 つのコード** で裏付けたものの、と捉えたと章間の繋がりが分かりやすい。

使い方

```
zig build -Dexample=compile_time_morse flash
zig build -Dexample=state_machine_game flash    # OLED + ボタン (PD1) が必要
zig build -Dexample=packed_settings flash      # LED + ボタン
zig build -Dexample=comptime_lookup flash      # PWM LED (PD2 = TIM1_CH1)
```

それぞれの配線は各 `examples/<name>/main.zig` の冒頭コメントに書いてある。

まとめ

- `comptime` を使うと、計算結果 (配列 / テーブル / 文字列展開) が ROM の `.rodata` に焼かれ、実機の RAM/CPU を消費しない
- `tagged union` + 網羅 `switch` でステートマシンを書くと、「ステートを増やすたびにハンドラの抜けが `compile error` で見つかる」という安全な拡張ができる
- `packed struct(uN)` + `@bitCast` + `@compileError` の組み合わせで、「永続化したい設定を 1 ワードに収める」ような場面が表現的に書ける
- これらは ARM Cortex-M でも RISC-V でも同じように使えるので、ターゲットを変えても再利用可能なパターン

付録

補足資料

付録A. 用語集

本書で繰り返し登場する用語の要約集。

用語	説明
CH32V003	WCH 製の RISC-V (RV32EC) MCU。FLASH 16KB / SRAM 2KB / 48MHz。本書のターゲット。
RV32E	RISC-V 32-bit の組み込み派生。汎用レジスタが <code>x0 ~ x15</code> の 16 本に削られた版。
RV32EC	RV32E + 圧縮命令 (C) 拡張。CH32V003 が採用。
C 拡張 (RVC)	16-bit 幅の短縮命令を導入する RISC-V 拡張。コード密度に効く。
freestanding	OS が無い環境向けターゲット。libc などの標準ランタイムを前提にしない。
EABI	Embedded ABI。組み込み向けの呼び出し規約。
MMIO	Memory-Mapped I/O。周辺レジスタを物理メモリ番地経由で読み書きする方式。
MTVEC	RISC-V の機械モード trap vector。割り込み / 例外発生時の PC 飛び先。
PFIC	CH32V (QingKe) の Programmable Fast Interrupt Controller。NVIC 風の割り込みコントローラ。
SWIO	CH32V のデバッグ / 書き込み用 1-wire シリアル。SWD 相当。
WCH-LinkE	WCH 公式の SWIO ホストアダプタ。
minichlink	ch32fun 同梱の USB-SWIO ブリッジ CLI。 <code>-w <file></code> で書き込み。
objcopy	ELF を <code>.bin</code> / <code>.hex</code> に変換するツール。Zig の <code>addObjCopy</code> が同等処理を内包。
LMA / VMA	リンカ用語。Load Memory Address は「書き込み先」、Virtual Memory Address は「実行時アドレス」。
<code>extern struct</code>	C ABI 互換のメモリレイアウトを保証する Zig の構造体宣言。
<code>*volatile T</code>	volatile な型付きポインタ。最適化で読み書きが省略・並び替えされない。
<code>compiler_rt</code>	乗除算ヘルパなど、CPU が持たない算術を補う関数群。Zig は自前実装を持つ。
lld	LLVM 系のリンカ。Zig に同梱され、GNU <code>ld</code> を使わずにリンクが完結する。

用語	説明
PFIC IENR	PFIC の Interrupt Enable Register。IRQ 番号ごとにビットを立てて有効化する。
SysTick	コア標準の周期タイマ。 <code>delayMs</code> / <code>systick.init</code> が使う。
SSD1306	128×64 OLED コントローラ。I2C 経由で叩く。
GDDRAM	SSD1306 内部のグラフィクス用 DRAM。1024 バイトのフレームバッファそのもの。

付録B. RV32 アセンブリ チートシート

本書の起動コードやベクタテーブル章で出てきた RV32 命令の要点まとめ。

レジスタ別名

ABI 名	物理	用途
zero	x0	常に 0
ra	x1	リターンアドレス
sp	x2	スタックポインタ
gp	x3	グローバルポインタ (Linker Relaxation で使用)
tp	x4	スレッドポインタ
t0 ~ t2	x5~x7	一時 (caller-saved)
s0 ~ s1	x8~x9	保存 (callee-saved) — RV32E では x8 までしか無いので注意
a0 ~ a7	x10~x17	引数・戻り値 — RV32E では a0~a5 (x10~x15) まで
s2 ~ s11	x18~x27	保存 — RV32E には無い
t3 ~ t6	x28~x31	一時 — RV32E には無い

⚠ RV32E は x0 ~ x15 までしか持たない。本書の SysTick ハンドラの退避コードに s2 ~ s11 / t3 ~ t6 が並んでいるのは、RVE であっても LLVM の RISC-V バックエンドが安全側に倒して命令を出すので、ハンドラ側も対応して退避している、という事情。

よく出てきた命令

命令	意味
la rd, sym	rd ← &sym。擬似命令で auipc + addi に展開
li rd, imm	rd ← imm。擬似命令で lui + addi などに展開

命令	意味
<code>j label</code>	$PC \leftarrow \text{label}$ (無条件ジャンプ)
<code>call sym</code>	$ra \leftarrow PC+4; PC \leftarrow \text{sym}$
<code>addi rd, rs, imm</code>	$rd \leftarrow rs + \text{imm}$
<code>sw rs, offset(rs1)</code>	$\text{MEM}[\text{rs1}+\text{offset}] \leftarrow rs$ (4 バイト書き込み)
<code>lw rd, offset(rs1)</code>	$rd \leftarrow \text{MEM}[\text{rs1}+\text{offset}]$ (4 バイト読み出し)
<code>csrw csr, rs</code>	$csr \leftarrow rs$
<code>csrs csr, rs</code>	$csr \leftarrow csr \mid rs$
<code>csrci csr, imm</code>	$csr \leftarrow csr \& \sim \text{imm5}$
<code>mret</code>	機械モード trap からの復帰。 <code>mstatus.MIE</code> を <code>MPIE</code> で復元、 $PC \leftarrow \text{mepc}$
<code>wfi</code>	割り込みが来るまで停止

起動 asm の意味の取り方

```
.option push
.option norelax
la gp, __global_pointer$
.option pop
```

- ・ `la gp, __global_pointer$` は通常 `auipc + addi` に展開され、さらに RISC-V リンカは `gp` 相対の命令に "relax" する。
- ・ ところが `gp` 自身の値を `gp` 相対で求めるのは無意味。そこで `.option norelax` で囲い、`auipc + addi` の素の展開を保つ。

```
la sp, _stack_top
j _start_c
```

- ・ スタックを RAM 末尾に立てる。
- ・ `_start_c` には `j` で飛ぶ — 戻る必要がないので `call (= ra を積む)` より軽い。

SysTick ハンドラの caller-saved 退避

```
addi sp, sp, -128
sw ra, 124(sp)
sw gp, 120(sp)
sw tp, 116(sp)
sw t0, 112(sp)
...
sw t6, 8(sp)
call _systick_irq_body
lw t6, 8(sp)
...
addi sp, sp, 128
mret
```

- `sp` を 128 バイト下に降ろし、32 個 (4 バイト × 32 = 128) のレジスタ分の枠を確保。
- `ra` と `gp` / `tp` も含めて退避するのは、`mret` 後の戻り先と Linker Relaxation 用ベースを守るため。
- `mret` で `PC ← mepc`、`mstatus.MIE ← mstatus.MPIE` が同時に行われ、元のコンテキストへ戻る。

付録C. CH32V003 レジスタ早見表

`src/periph/registers.zig` で扱っている主要ベース番地とオフセットの要約。データシートの該当章を引くときの索引として使う。

メモリマップ全体

領域	番地レンジ
Boot alias (BOOT0 で FLASH / BootROM にエイリアス)	<code>0x0000_0000</code> ~ <code>0x0000_3FFF</code> (16 KB)
予約	<code>0x0000_4000</code> ~ <code>0x07FF_FFFF</code>
FLASH	<code>0x0800_0000</code> ~ <code>0x0800_3FFF</code> (16 KB)
予約	<code>0x0800_4000</code> ~ <code>0x1FFF_EFFF</code>
System Memory (BootROM, USART ISP)	<code>0x1FFF_F000</code> ~ <code>0x1FFF_F7FF</code> (1.92 KB)
Factory trim (HSI 校正值などの工場 ROM)	<code>0x1FFF_F7D4</code> ほか
Option bytes (RDPR / USER)	<code>0x1FFF_F800</code> ~ <code>0x1FFF_F80F</code>
予約	<code>0x1FFF_F810</code> ~ <code>0x1FFF_FFFF</code>
SRAM	<code>0x2000_0000</code> ~ <code>0x2000_07FF</code> (2 KB)
予約	<code>0x2000_0800</code> ~ <code>0x3FFF_FFFF</code>
周辺 (APB1/APB2/AHB)	<code>0x4000_0000</code> ~ <code>0x5FFF_FFFF</code>
予約	<code>0x6000_0000</code> ~ <code>0xDFFF_FFFF</code>
コア内蔵周辺 (PFIC, SysTick, ...)	<code>0xE000_0000</code> ~ <code>0xFFFF_FFFF</code>

リセット直後の CPU は `PC = 0x0000_0000` から fetch するが、ハードウェアが BOOT0 ピンの状態に応じて `0x0800_0000` (FLASH) または `0x1FFF_F000` (BootROM) に透過的にエイリアスする。ソフトから見たアドレスは最初から `0x0800_xxxx` 系。

バスごとのベース

バス	ベース	主な乗物
APB1	0x4000_0000	TIM2, I2C1, USART1 など
APB2	0x4001_0000	AFIO, GPIOA/C/D, EXTI, TIM1, ADC1 など
AHB	0x4002_0000	RCC, FLASH 制御, DMA, CRC など

個別ペリフェラル

名前	ベース	主なオフセット
RCC	0x4002_1000	CTLR=+0x00 , CFGR0=+0x04 , APB2PCENR=+0x18 , APB1PCENR=+0x1C
FLASH 制御	0x4002_2000	ACTLR=+0x00 , KEYR=+0x04 , OBKEYR=+0x08 , STATR=+0x0C , CTLR=+0x10 , ADDR=+0x14 , OBR=+0x1C , WPR=+0x20 , MODEKEYR=+0x24 , BOOT_MODEKEYR=+0x28
GPIOA	0x4001_0800	CFGLR=+0x00 , INDR=+0x08 , OUTDR=+0x0C , BSHR=+0x10
GPIOC	0x4001_1000	同上
GPIOD	0x4001_1400	同上
I2C1	0x4000_5400	CTLR1=+0x00 , CTLR2=+0x04 , DATAR=+0x10 , STAR1=+0x14 , STAR2=+0x18 , CKCFGR=+0x1C
PFIC	0xE000_E000	ISR=+0x00 , IENR=+0x100 系
SysTick	0xE000_F000	CTLR=+0x00 , SR=+0x04 , CNT=+0x08 , CMP=+0x10

主要ビットマスク

RCC.APB2PCENR

ビット	内容
0	AFIO

ビット	内容
2	GPIOA
4	GPIOC
5	GPIOD

RCC.APB1PCENR

ビット	内容
21	I2C1

I2C CTLR1

名前	ビット
PE	0
START	8
STOP	9
ACK	10

SysTick CTLR

名前	ビット
STE	0
STIE	1
STCLK	2

FLASH CTLR (書き込み用)

名前	ビット
PG (FTPG)	0
PER (FTER)	1

名前	ビット
STRT	6
LOCK	7
FAST_LOCK	15
BUF_LOAD	18
BUF_RST	19

FLASH STATR

名前	ビット
BSY	0
WRPRTERR	4
EOP	5

FLASH KEYR / MODEKEYR シーケンス

0x45670123 → 0xCDEF89AB の順で 2 回書き込むと、それぞれ主ロック / fast プログラムロックが外れる。

付録D. トラブルシューティング

「ビルドはできたのに動かない」「書き込みで詰まる」系の症状ごとのチェックポイント。

ビルド系

zig build で「Unknown example 'xxx'」

- ・ `build.zig` の `examples` 配列に該当 example が登録されているか確認
- ・ 名前のタイポ。 `-Dexample=` の値はディレクトリ名と一致が必要

zig build disasm / mapfile / size でコマンドが見つからない

- ・ LLVM ツールが入っていないため。 `brew install llvm` や `sudo apt install llvm` で導入
- ・ もしくは GNU 系の `riscv-none-elf-*` をインストール (本プロジェクトの sh が両方フォールバックする)

zig version が 0.16.0 でない

- ・ README の手順に従って `0.16.0` を導入。Homebrew が `0.15.x` をインストールするケースは tarball で上書きする

バイナリ生成系

16K に収まらない

- ・ `zig build size` でセクションサイズを確認
- ・ 多くの場合、`.text` 過大: 第 3 章で扱った `link_function_sections` / `link_gc_sections` が有効か確認
- ・ `Feature.c` が外れていないか (圧縮命令が無いとサイズ倍増)

ELF はできるが .bin が極端に大きい

- ・ `.bin` は LMA 上のレンジを Raw に出すため、「FLASH の途中にだけデータがある」と途中の隙間が全部 0 で埋められる
- ・ 第 4 章のリンクスクリプトで `.data` を `> RAM AT > FLASH` にしているか確認 (`AT >` を書き忘れると FLASH に詰めない)

書き込み系

minichlink not found

- ・ `tools/flash.sh` は `../ch32fun/minichlink/minichlink` を見るので、ディレクトリ配置と `make` 済みかを確認
- ・ `which minichlink` が PATH 上にあるなら、スクリプトの `MINICHLINK` 行を書き換えても良い

libusb 関連で minichlink のビルドが通らない

- ・ macOS: `brew install libusb pkg-config`
- ・ Debian/Ubuntu: `sudo apt install libusb-1.0-0-dev pkg-config`
- ・ Arch: `sudo pacman -S libusb pkgconf`

USB デバイスが見えない (Linux)

- ・ `49-wch.rules` を `/etc/udev/rules.d/` に置いて `sudo udevadm control --reload-rules`
- ・ ユーザを `plugdev` (Ubuntu) に追加して再ログイン
- ・ `lsusb` で WCH のベンダ ID (`1a86`) が見えるか確認

書き込みに成功してもチップが動かない

1. LED 等の物理配線を見直す
2. `zig build disasm` で `.lst` を見て、`_start` が `.vector_table` の先頭ワードが指している番地と一致しているか
3. `_start_c` の最後で `root.main()` を呼んでいるか
4. クロックが上がっていないと SysTick の `delayMs` の体感が極端に遅くなる: `system.init` を呼んでいるか確認

実行時の挙動が変

起動直後に固まる

- ・ 多くは `mtvec` 設定前に割り込みが入って `_default_irq_entry` で `wfi` ループ
- ・ `_start_c` の `setupMachineState()` の順序を確認
- ・ `Feature.e` が外れたまま (= RV32I のレジスタ x16+ を出力するコードが走る) 可能性も。ターゲット設定を再確認

`.data` のグローバルが期待値で初期化されない

- ・ `copyData()` が呼ばれているか
- ・ リンカスクリプトの `_sidata = LOADADDR(.data);` 行があるか (これが無いと FLASH 上のロードアドレスが解決できない)

ボタンが反応しない

- ・ `gpio.enablePortClock(.D)` 呼んでいるか
- ・ 配線が GND プルダウンになっていないか (本プロジェクトはアクティブ Low + 内部プルアップ前提)

I2C / SSD1306 系

`initI2c` が `BusyTimeout` で帰ってくる

- ・ SDA/SCL に **外部プルアップ抵抗** (4.7k~10kΩ) があるか確認。内部プルアップに頼ると Fast mode 1MHz では持たない
- ・ バスに何も繋がってなくても `STAR2.BUSY` は基本下りるはず。立ちっぱなしならハード起因 (配線 / 電源)

画面が真っ黒のまま

- ・ I2C アドレス (`0x3C`) が個体と一致しているか
- ・ `setbuf(false) → refresh()` の順で 1 度書いてみて、何も出なければハード側
- ・ `oled` サンプルから流用して動作確認するのが手っ取り早い